Health Data Science: AI and Knowledge Translation

M. Ehsan. Karim

2026-05-14

Table of contents

The Project

Welcome to the course notes for **Health Data Science: AI and Knowledge Translation**.

This platform bridges the gap between traditional epidemiology and modern data science. Unlike standard textbooks that focus on a single language, this resource adopts a **"Polyglot"** approach—teaching R and Python side-by-side—and integrates **Artificial Intelligence** directly into the workflow.

Whether you are a health researcher looking to modernize your skills or a data scientist seeking to understand epidemiological rigor, you are in the right place.

Here, we offer:

- **"Zero to AI Co-Pilot" Training**: Learn to use Large Language Models (LLMs) to generate, debug, and translate code while critically auditing them for bias and hallucinations.
- **Real-World Data**: Guides on navigating "messy" open data sources such as NHANES and WHO, rather than sanitized "toy" datasets.
- **Knowledge Translation**: Step-by-step walkthroughs on building interactive dashboards (Shiny/Dash) and dynamic reports to communicate findings to policymakers.

What We Aim to Achieve

We are on a mission to:

- **Democratize Compute Access**: Eliminate hardware barriers by using cloud-native environments (Google Colab, Kaggle) that run on any device.

- **Bridge the "Polyglot" Gap**: Provide "Rosetta Stone" tutorials where concepts are implemented in both R and Python, allowing learners to be fluent in both.
- **Teach "AI-Resilience"**: Shift the focus from manual syntax generation to **rigorous code auditing**, security validation, and reproducibility.
- **Enable Professional Portfolios**: Help students transform static analyses into public-facing data science portfolios.

Dive into Our Modules

Embark on a journey through our structured modules, designed to take you from data literacy to advanced AI workflows.

> **Note**
>
> The tutorials are designed with a consistent structure to provide a cohesive learning experience. Here is what you can expect in each chapter:
>
> 1. **Overview**: A concise summary of learning objectives and topics.
> 2. **Concepts**: Core theoretical materials (slides, video lessons).
> 3. **Polyglot Tutorials**: In-depth walkthroughs with tabbed code blocks allowing you to toggle between **R** and **Python** solutions.
> 4. **AI Auditing**: Specific sections dedicated to verifying and "stress-testing" AI-generated code for the chapter's topic.
> 5. **Knowledge Check**: Ungraded self-assessments to reinforce key concepts.
> 6. **Practice Exercises**: Hands-on tasks using real-world health data to build your portfolio.

How Our Content is Presented

All our resources are hosted on an easy-to-access GitHub page. The format? Engaging text, reproducible code, clear outputs, and crisp videos that distill complex topics.

This document is created using Quarto, allowing us to weave analysis, code, and interpretation into a single reproducible document.

Open Copyright License

How to Cite

The BibTeX format can be downloaded from here.

Course Operations

Course Operations

This chapter is the single maintainer-facing home for course planning, assistant guidance, and release-readiness checks. Use it before editing the book, preparing a week for release, or asking an instructional reviewer to review course materials.

Operating Principles

- Treat the syllabus as the source of truth for weeks, deliverables, and deadlines.
- Keep student-facing content beginner-accessible and cloud-first.
- Treat the fork-based repository workflow as the official student path unless Canvas explicitly announces an instructor-managed alternative.
- Prefer reproducible Quarto, R, Python, and GitHub Codespaces workflows over local-machine assumptions.
- Keep generated output separate from source content.
- Add content where the normalized structure says it belongs: weeks, assignments, milestones, references, examples, or assets.

Editing Rules

- Update `_quarto.yml` whenever a page should appear in the book.
- Keep paths relative and test them after moving files.
- Use `assets/` for shared images and data that support teaching pages.
- Use `examples/` for runnable code and sample datasets.

- Do not bury assignment requirements only inside the syllabus; add a student-facing assignment or milestone brief.

Reproducibility Expectations

- Every activity should state expected inputs, outputs, and submission location.
- Every code activity should run in a fresh GitHub Codespace.
- Every dataset activity should record source, license, access date, and limitations.
- Every AI-assisted activity should include an audit note: what AI helped with, what changed, and how the result was verified.

Review Cadence

- Before term: verify links, screenshots, Codespace setup, dependencies, and all assignment briefs.
- Weekly: update troubleshooting notes based on student issues.
- Before each milestone: test the milestone instructions from a fresh repository.
- After term: archive changes, refresh screenshots, and update the completion matrix.

Done Checklists

Course Page

- The page appears in `_quarto.yml` if it is student-facing.
- The title matches the week or reference purpose.
- Learning goals, activity steps, and expected outputs are clear.
- Links and image paths render from the Quarto project root.
- Open work is clearly scoped and assigned.

Assignment

- The assignment has a student-facing brief.
- Required files and submission location are explicit.
- The work can be completed in GitHub Codespaces.
- Packages, data, and outputs are documented.
- The grading checklist matches the syllabus assessment logic.
- A1-A3 use Complete/Incomplete language; A4-A10 use the best-5-of-7 scored assignment structure.
- Every assignment brief states the Canvas repository-link submission route and AI-use expectation.
- Assignment 1 and Week 2 should teach the default sequence: open template repository, fork, launch Codespaces from the fork, commit and sync, then submit the fork link on Canvas.

Milestone

- The milestone states the exact artifact students submit.
- The milestone maps to the final portfolio.
- The rubric includes reproducibility, auditability, data stewardship, and KT value.
- Peer review and instructional review instructions are actionable.

Reproducibility

- A fresh Codespace can render or run the project.
- Paths are relative.
- Dependencies are declared with `renv`, `requirements.txt`, or a clearly documented equivalent.
- No required data exists only on one person's local computer.
- Randomness uses a seed when results depend on it.
- The README explains how to reproduce the output.

AI Audit

- The AI-use note says what tool was used and for what task.
- Generated code was checked against real column names and expected outputs.
- Generated summaries are grounded in actual results.
- Security, privacy, and bias risks are considered.
- The final text does not cite sources invented by AI.

NHANES Case Study

The NHANES Health Equity Dashboard case study is the course spine. Students revisit the same app, data, and workflow from different angles as their skills mature. It should not become a second term project.

Implemented Structure

- `examples/nhanes-equity/app/app.R`
- `examples/nhanes-equity/data/nhanes_equity_v6.rds`
- `examples/nhanes-equity/data/nhanes_equity_v6.csv`
- `examples/nhanes-equity/scripts/build_nhanes_equity.R`
- `examples/nhanes-equity/snippets/case-study-data-only.qmd`
- `examples/nhanes-equity/snippets/case-study-data-analysis.qmd`
- `examples/nhanes-equity/snippets/case-study-dashboard.qmd`
- `examples/nhanes-equity/README.md`

The cached data live in `examples/nhanes-equity/data/` so the app, script, README, and weekly snippets all use one consistent location.

Editor Decisions

- The NHANES sequence is data-first through Week 7 and app-forward starting in Week 8.

- The official setup path is fork-first. GitHub Classroom is not the default in this version of the course book; if used in a future offering, it should be labeled as an instructor-managed alternative in Canvas and in any local handout.
- Week 8's required pathway is local and Codespaces-friendly: run the Shiny exemplar, make one audience-facing text or documentation improvement, and record a test result.
- Static dashboard and deployment paths are optional demonstrations unless a later assignment explicitly asks for them.
- Weeks 6-8 now have complete exemplar packs; instructional reviewers use the provided keys and briefs for facilitation rather than drafting new NHANES materials.

Teaching Progression

- Weeks 1-2: provenance, ethics, folder structure, relative paths, and rendering; no app run instructions.
- Weeks 3-5: data loading, missingness checks, AI audit comments, documentation-only Git changes, and R/Python parity checks using the CSV snapshot.
- Weeks 6-7: visualization critique, flawed-plot audit, EDA, Table 1, and planted-error audit.
- Week 8: first app-forward week, with the Shiny dashboard exemplar used for a small KT adaptation.
- Weeks 9-10: slides, reports, source notes, limitations, and audience-specific communication.
- Week 11: fresh-session reproducibility testing and optional rebuild from CDC if internet access is available.

RA Watchpoints

- Students may confuse the cached case-study dataset with raw NHANES microdata.
- The app's Download CSV button exports the currently summarized table output, not raw NHANES microdata.
- The CDC rebuild path requires internet access and the `nhanesA` package.

- Week pages should include the shared snippet rather than duplicating instructions.
- Weeks 1-2 should not instruct students to run the app.
- Week 8 is the first week where app running is central.
- Student changes should stay small unless the instructor explicitly assigns a dashboard feature.
- Relative paths are part of the lesson; discourage absolute machine-specific paths.
- Weeks 6-8 should be treated as complete NHANES exemplars; updates should maintain the existing worked examples, activities, answer keys, and assignment briefs.

Case Study Done

- The book renders from the repo root.
- Weeks 1-11 each include a short case-study touchpoint.
- The case-study folder contains runnable app code, cached data, rebuild script, shared snippet, and README.
- Weeks 6-7 include worked examples, student-facing activities, and instructor-only keys.
- Students can complete the default classroom path without internet.
- CDC rebuilds are documented as optional, network-dependent reproducibility work.

Syllabus

Acknowledgement

UBC's Point Grey Campus is located on the traditional, ancestral, and unceded territory of the x mə k əyəm (Musqueam) people. The land it is situated on has always been a place of learning for the Musqueam people, who for millennia have passed on their culture, history, and traditions from one generation to the next on this site.

Course Information

Course Title:	Health Data Science: AI and Knowledge Translation
Credit Value:	3
Format:	Integrated Seminar & Hands-on Activities (1 session/week, 3 hours)
Time:	Thu 11:00 AM - 2:00 PM (Sept-Dec 2026)
Room:	Room posted in Canvas (in-person)

Course at a Glance

Core workspace	GitHub Codespaces (GitHub, Quarto, & VS Code in the browser)
Core language	R, Quarto, and reproducible project workflows
Secondary exposure	Short Python examples for polyglot awareness; optional extension pathway
Main project product	Reproducible Quarto report + required dashboard-style KT product + auditable repository
Primary assessment logic	Foundational gates, best-5 weekly assignments, staged project milestones, presentation, final portfolio

Prerequisites

None required. This course adopts a "Zero to AI Co-Pilot" pedagogy. We welcome students from all disciplines. You do not need prior experience with R, Python, statistics, or programming. We teach foundational concepts and tools from the ground up.

What "no prerequisites" means in practice

This is a beginner-accessible course, not a no-work course. You will learn a substantial but carefully scaffolded toolkit at a steady pace: R, GitHub Codespaces, Quarto, Git, reproducible workflows, AI-assisted coding, basic Python awareness, and dashboard-style knowledge translation. Because coding is a cumulative skill, the weekly assignments are carefully scaffolded to serve as your core practice. If you are new to programming, completing these assignments is how you will steadily build your foundational skills during the first month. Comfort with web browsers, basic file management, and willingness to troubleshoot are assumed; everything else is taught and supported through templates, in-class studio time, peer debugging, and TA support.

Primary Language: R, with Python Awareness

This course uses **R** as the primary programming language for all core graded work. R is widely used in epidemiology, biostatistics, public health, and reproducible research, and the tidyverse ecosystem (`dplyr`, `ggplot2`, `plotly`, Quarto, and dashboard-style reporting tools) provides a coherent pipeline from data import to publication.

The course remains **polyglot-aware** without becoming a full second programming course. Students will read and run short **Python/pandas** examples in GitHub Codespaces, compare Python and R solutions with AI assistance, and learn when Python may be the more appropriate tool. A Python-flavoured option may be offered for one activity or project extension, but students can complete all required outcomes in R.

Students will create a small dashboard-style knowledge translation product as part of the portfolio. The required core will focus on approaches that work reliably in GitHub Codespaces and Quarto-based publishing. Shiny, Shinylive, and AI-generated static HTML/JavaScript dashboards may be demonstrated as optional or advanced deployment pathways rather than required technologies for every student.

Computing Environment: Fork-Based Workflow

All supported coding takes place in **GitHub Codespaces**, a cloud computer accessible from any modern web browser. No local installation is required. Students **fork** a course template repository into their own GitHub account, then launch Codespaces from their fork.

Tools required for graded work will be selected and tested for use in Codespaces. Optional demonstrations may show additional deployment pathways, but students will not be penalized if an advanced demo technology is unsuitable for their device or project.

Contacts

Role:	Course Instructor
Name:	Dr. M. Ehsan Karim
Email:	ehsan.karim@ubc.ca
Office:	Appointments by request
Hours:	Weekly instructor and TA support times will be posted on Canvas

Typical response time: within 48 hours on weekdays.

Other Instructional Staff: TBA. To communicate with the computing TA, attend weekly classes and specified drop-in hours (as announced in Canvas).

Instructor Biographical Statement

Dr. M. Ehsan Karim is an Associate Professor in Health Data Science at the UBC School of Population and Public Health (SPPH), a Scientist at the Centre for Advancing Health Outcomes, an associate member of the Department of Statistics (UBC), and a previous Michael Smith Foundation for Health Research (MSFHR) Scholar. He obtained his PhD in Statistics from the University of British Columbia and completed his postgraduate training in the Department of Epidemiology, Biostatistics, and Occupational Health at McGill University. His current research focuses on causal inference, real-world observational data analyses, and applications of machine learning approaches in epidemiologic studies.

Course Description

This interdisciplinary course bridges traditional epidemiology, modern data science, responsible AI use, and knowledge translation. Using a "Zero to AI Co-Pilot" pedagogy, students learn how to work with health-related data in a cloud-based environment, use generative AI as a coding assistant, and critically audit AI-generated code and outputs. The course is beginner-accessible and assumes no prior coding or statistics background, while still requiring steady hands-on practice.

The curriculum emphasizes reproducible research and the "last mile" of knowledge translation: transforming real-world health data into clear, auditable, and audience-appropriate products. By the end of the course, students will have built a portfolio that includes documented code, a reproducible Quarto report, clear visualizations, a dashboard-style knowledge translation product, and a reflection on AI use and auditability.

Course Structure

This course integrates theoretical foundations with the practical realities of modern health data science. Through short seminars, guided setup time, hands-on studio work, and structured peer support, you will move from basic data literacy to reproducible, portfolio-ready communication products: reports, dashboards, visual stories, and project repositories that can be audited by others.

Course Pillars

1. **The Full Data Lifecycle:** From ethical data access and cleaning to secure cloud computing and final publication.
2. **Reproducible Science:** Git-based workflows, environment management (`renv`), and literate programming in Quarto - ensuring analyses are transparent, shareable, and robust.
3. **AI-Augmented Workflows:** Ethically leveraging LLMs as coding co-pilots to debug, refactor, and translate code, while auditing AI-generated outputs for coding errors, hallucinated claims, security risks, health bias, and reproducibility failures.
4. **Knowledge Translation (KT):** The "last mile" of data science - transforming results into dashboard-style products, visual stories, plain-language summaries, and policy-ready reports. Shiny and static web approaches are introduced as optional or advanced pathways rather than assumed for every project.
5. **Real-World Health Data:** Hands-on activities using messy, real-world datasets (e.g., NHANES, Canadian PUMFs) rather than toy examples.

Session Structure (3-Hour Block)

Each weekly session generally follows the following segments:

Segment	Duration	Description
Seminar and demonstration	45 min	Concepts, examples, and live coding by the instructor

Segment	Duration	Description
Guided setup	20 min	Everyone completes the first step of the activity together; TAs confirm students are unblocked
Break / technical reset	10 min	Short break and time to resolve login, rendering, or package issues
Hands-on studio	90 min	Independent or group work on the weekly activity; TA circulation and peer debugging
Wrap-up and next steps	15 min	Expected output demo, common errors, submission expectations, and preview of next week

Peer Support: Debugging Partners

To scale support with high enrollment, each student is assigned a **debugging partner**. Before raising a hand for TA help, try your partner first. This builds collaboration skills and reduces wait times for everyone.

Asynchronous Onboarding (Week 0)

Due before the first class session (Week 1).

A short Canvas module to complete before the term begins:

1. Create a **GitHub account** at github.com using your UBC email.
2. Choose a **professional username** (this will appear on your public portfolio).

3. Watch a **5-minute orientation video** (posted on Canvas) introducing the course workflow.
4. Complete the **pre-course survey** (anonymous; helps the teaching team calibrate to the cohort's baseline experience).

This takes approximately 15-20 minutes and ensures Week 1 can focus entirely on health data concepts rather than account setup.

Schedule of Topics

Week	Seminar Topics	Activities	Deadlines
	Part 1: Foundations & Workflows		
0 (Async)	-	Create GitHub account; watch orientation video; pre-course survey.	Complete before first class
1 (Sept 10)	Health Data, KT & Ethics: research lifecycle overview; KT 'last mile'; privacy/security; Indigenous Data Sovereignty (OCAP, CARE); AI hallucinations & bias (AI fluency does not equal correctness).	Explore open data portals; complete Data Intake Card; discuss data formats, provenance, stewardship, and responsible AI use; course workflow overview (Canvas + GitHub).	(No graded submission)
2 (Sept 17)	Modern Workflows: IDE vs notebook vs script; literate programming; reproducible project structure; Codespaces as the shared lab bench; Quarto basics.	Fork course template repo into personal GitHub account; launch Codespace; tour VS Code; edit + render Quarto notebook; stage, commit, sync; stop Codespace.	Assignment 1 (Mon 4 PM)
3 (Sept 24)	R with AI (Code runs does not mean code is correct): data import/export; core constructs (objects, functions, control flow); the tidyverse pipeline; debugging with LLMs; writing custom functions; intro to renv for dependency management.	Run R in Codespace; load dataset; inspect & transform with dplyr; write a custom function; initialize renv; document in Quarto; commit incrementally.	Assignment 2 (Mon 4 PM)

(continued)

Week	Seminar Topics	Activities	Deadlines
4 (Oct 1)	Git & Collaboration: commits as analysis provenance; branching/merging; Pull Requests & peer review mindset; repo hygiene (.gitignore, relative paths).	Git basics via GUI; meaningful commits; .gitignore & relative paths; create branch, open Pull Request; one member forks group project template, adds teammates as collaborators.	Assignment 3 (Mon 4 PM); Milestone 0: form project group
5 (Oct 8)	Polyglot Awareness & R Deepening: R as the core course language; when Python is useful; reading short Python/pandas examples; translation traps; AI-assisted parity checks; R mastery exercises on dplyr and functions.	Read and run a short Python/pandas notebook in Codespaces; compare with an R/tidyverse solution; use AI to translate a small task; verify parity with checks; complete R deepening exercises.	Assignment 4 (Mon 4 PM)
Part 2: Analysis, Visualization & Dashboards			
6 (Oct 15)	Data Visualization: visual reasoning in AI-assisted workflows; steering AI-generated plots (ggplot2, plotly); evaluating clarity, bias, aggregation, and misrepresentation.	Generate & refine visualizations with AI (ggplot2/plotly); evaluate AI-generated plots for clarity, bias, aggregation choices, and misleading design; embed visuals in Quarto with captions.	Assignment 5 (Mon 4 PM); Milestone 1: project proposal
7 (Oct 22)	EDA, Table 1 & AI Auditing: descriptive analysis workflow; what summary statistics claim; auditing AI-generated summaries/plots for planted coding, statistical, and stewardship errors; revisiting data provenance and Indigenous Data Sovereignty.	Perform EDA on project data; create Table 1; complete an AI audit exercise with planted errors; add or revise a data provenance and stewardship statement; checkpoint commits.	Assignment 6 (Mon 4 PM); Milestone 2: Preliminary Analysis
8 (Oct 29)	Dashboard Prototypes for KT: Quarto dashboard-style products and Shiny concepts; building a minimal dashboard from EDA outputs; deployment options introduced as demonstrations only (e.g., Shinylive/GitHub Pages).	Build a minimal dashboard-style KT product from EDA outputs; test in Codespaces; view optional Shiny/Shinylive deployment demo; identify which dashboard pathway is feasible for the term project.	Assignment 7 (Mon 4 PM)

(continued)

Week	Seminar Topics	Activities	Deadlines
9 (Nov 5)	Communication of Scientific Findings: audience-specific KT; plain language summaries; Quarto revealjs slides; citations & references; peer review practices.	Create Quarto revealjs slides (guided intro to slide format); integrate figures; structured peer feedback on clarity & KT.	Assignment 8 (Mon 4 PM)
	Part 3: Communication, Portfolios & Presentations		
10 (Nov 12)	Writing & Publishing Reports: scientific formatting in Quarto; dynamic reporting; embedding interactive elements; enabling GitHub Pages; publishing your portfolio; citation/grounding audit. Note: UBC Mid-Term Break is Nov 9-11; class meets as usual on Thu Nov 12.	Draft report in Quarto; add citations (Zotero/BibTeX); embed visuals; enable GitHub Pages and publish portfolio site (or private equivalent).	Assignment 9 and Milestone 3: Project Update due Mon Nov 16, 4 PM
11 (Nov 19)	Portfolio Surgery & Optional Advanced Deployment: reproducibility stress-test in a fresh Codespace; fix broken paths, missing dependencies, unclear READMEs; optional demo of AI-generated static HTML/JS dashboards and optional Python-flavoured project pathway.	Swap repositories with a partner for reproducibility stress-test; run in a fresh Codespace; fix failures; optional advanced demo comparing Shiny, Quarto dashboard, and static HTML approaches.	Assignment 10 and Milestone 4: Peer Review due Mon Nov 23, 4 PM
12 (Nov 26)	Term Project Presentations (Part 1): live Q&A and peer review.	Milestone 5 presentations; live Q&A; peer review feedback.	Presentation slides due before class
13 (Dec 3)	Term Project Presentations (Part 2): live Q&A and peer review.	Milestone 5 presentations; live Q&A; peer review feedback.	Presentation slides due before class
-	-	-	Final Portfolio due: Dec 11 (Fri 4 PM)

Learning Outcomes

By the end of this course, students will be able to:

1. Explain key ethical, privacy, security, Indigenous Data Sovereignty, and knowledge translation issues in health

data science, including responsible use of generative AI tools.
2. Use GitHub Codespaces, Quarto, Git, and GitHub to organize, document, version, and reproduce a small health data analysis.
3. Import, clean, summarize, and visualize health-related data using R and the tidyverse.
4. Read and compare short Python/pandas examples with AI assistance, identify common translation traps between R and Python, and explain when Python may be an appropriate tool.
5. Use generative AI as a coding assistant while critically auditing AI-generated code and outputs for coding errors, hallucinated claims, bias, security risks, and reproducibility problems.
6. Conduct a bounded exploratory data analysis and communicate descriptive findings using clear tables, figures, narrative interpretation, and appropriate limitations.
7. Create a dashboard-style knowledge translation product and a reproducible Quarto report that translate health data findings for a defined audience.
8. Manage reproducible software environments (`renv`) and project documentation so that analyses can be rerun in a fresh Codespace.
9. Provide constructive peer feedback on the clarity, reproducibility, auditability, and knowledge translation value of another project.

Learning Activities

Weekly Assignments (Weeks 2-11)

Each week, students complete short applied assignments that reinforce the tools and concepts from class. These assignments are scaffolded and practice-oriented, but they also provide an important measure of steady engagement and skill development. Unless otherwise noted, assignments are due the Monday after class at 4 PM, giving students time to revisit the activity, troubleshoot technical issues, and use office hours or discussion boards.

Grading policy:

- **Assignments 1-3 (Weeks 2-4) are required but worth 0% numerically.** These foundational assignments (Codespace/Quarto, R basics, Git) are graded on a Complete/Incomplete basis. Each must be completed to pass the course. Incomplete submissions may be resubmitted once within one week. A student who completes A1-A3 and otherwise earns at least 50% on the graded components will pass.
- **Assignments 4-10 (Weeks 5-11): Best 5 of 7 count.** Only your best 5 scores count toward your final grade. This provides flexibility for difficult weeks while ensuring consistent engagement after the foundation is established. Assignment 6 includes a dedicated AI-audit task involving planted errors in AI-generated code, summaries, and/or visualizations.

Reproducibility requirement

All assignments must render/run in a GitHub Codespace without manual intervention. Assignments that violate this rule will be returned for a ”Reproducibility Fix” and may incur a grade deduction.

Term Project / Data Science Portfolio

A capstone project synthesized across the full term, broken into milestones:

Milestone	Due	Description
M0: Group Formation	Week 4 (Mon)	Form project group of 3-4 students; one member forks template and adds teammates as collaborators

Milestone	Due	Description
M1: Proposal	Week 6 (Mon)	Research question, dataset selection, feasibility check, and one-paragraph data provenance/stewardship statement
M2: Preliminary Analysis	Week 7 (Mon)	Initial data cleaning, exploratory visualizations, Table 1 or equivalent descriptive summary, runnable repository
M3: Project Update	Week 10 (Mon Nov 16)	Draft dashboard-style KT product + report draft submitted for peer review; polish is expected later in the Final Portfolio
M4: Peer Review	Week 11 (Mon Nov 23)	Constructive, specific, actionable feedback on a peer's M3, including reproducibility and auditability checks
M5: Presentation	Weeks 12-13	In-class scientific seminar; knowledge translation focus
Final Portfolio	Dec 11 (Fri)	Complete report, required dashboard-style KT product, and fully auditable code repository

Analytic Scope of the Term Project

The project is graded on the data lifecycle, reproducibility, visualization clarity, AI auditability, and knowledge translation - not on the sophistication of statistical inference. Descriptive summaries, well-chosen visualizations, clear documentation of data decisions, and clearly stated limitations are expected. Formal hypothesis tests, regression models, predictive models, or causal claims are out of scope unless agreed with the instructor in the proposal.

The required knowledge translation product is a small dashboard-style product suitable for the chosen audience. This may be implemented through Quarto dashboard-style reporting, R-generated interactive elements, or another instructor-approved pathway that runs reliably in GitHub Codespaces. Advanced deployment options such as Shiny hosting, Shinylive, or AI-generated HTML/JavaScript may be demonstrated, but are not required for every student.

Reproducibility Contract

The Reproducibility Checklist

A significant portion of the final grade depends on reproducibility. The following checklist serves as both a learning guide and the grading rubric for the Final Portfolio. Refer to it every week:

1. **Fresh Codespace test:** The project launches and runs start-to-finish in a newly created GitHub Codespace without manual intervention.
2. **Relative paths only:** No absolute paths anywhere in the codebase.
3. **Dependencies declared:** All R packages recorded via `renv` (introduced in Week 3, reinforced throughout).
4. **No manual steps:** The pipeline from raw data to final output is fully scripted.

5. **README quality:** `README.md` explains the research question, data provenance, how to reproduce the analysis, and known limitations.
6. **.gitignore configured:** Rendered outputs, OS junk, and sensitive files are excluded from version control.
7. **Commit history:** Incremental, meaningful commits throughout the term.
8. **Documentation:** Code is commented; Quarto documents interleave code with interpretive narrative.
9. **Dashboard-style KT product works:** The project includes a small dashboard-style knowledge translation product that renders or runs in the supported course workflow. If the portfolio is public, it should be viewable through GitHub Pages or another instructor-approved route; if the project is private, the teaching team must be able to render or preview it in a fresh Codespace.

This contract is introduced in Week 1 and reinforced in every subsequent week.

Assessments of Learning

Scope & Expectations

- **Focus:** This project prioritizes descriptive health data analysis, visualization, dashboard-style knowledge translation, reproducible reporting, and AI auditability.
- **Audience:** SPPH 381H is an undergraduate course. No separate graduate-level requirements are assumed unless a graduate section is formally approved. Advanced students may pursue optional extensions with instructor approval, but full credit is achievable through the core R-based pathway.
- **Analytic boundaries:** Students are expected to clearly describe their data, workflow, results, and limitations. Formal modelling, prediction, causal inference, or hypothesis testing is optional only with instructor approval and must be described cautiously.

Component	Weight	Notes
Foundational Assignments (Weeks 2-4)	**Required (0%; Complete/ Incomplete)**	Must complete all 3 to pass; does not contribute to numeric grade
Weekly Assignments (Best 5 of 7, Weeks 5-11)	**35%**	Best 5 counted; each counted assignment worth 7%
Milestone 1: Project Proposal	**5%**	Dataset, question, feasibility, provenance/stewardship statement
Milestone 2: Preliminary Analysis	**5%**	Cleaning, EDA, descriptive summary, runnable repo
Milestone 3: Project Update	**5%**	Draft dashboard-style KT product + report draft for peer review
Milestone 4: Peer-Review Contribution	**5%**	Specific, actionable feedback on a peer's work; includes reproducibility check
Milestone 5: Presentation	**10%**	In-class scientific seminar (Weeks 12-13)
Final Portfolio	**35%**	Report, required dashboard-style KT product, auditable repository; graded against Reproducibility Contract
Total	**100%**	

- **Python option:** A small Python/pandas component may be used for an optional extension if it runs in Codespaces and is documented clearly; the required project can be completed entirely in R.
- **Auditability:** Graded against the Reproducibility Contract (above). The project must run in a fresh Codespace.

Late Submissions

- **Foundational Assignments (1-3):** May be resubmitted once within one week of the original deadline.

- **Weekly Assignments (4-10):** Due Mondays at 4 PM unless otherwise noted. Late submissions are not accepted. Missed assignments count as one of your dropped grades (Best 5 of 7 policy).
- **Final Portfolio:** Late submissions penalized 10% per day. Submissions more than 5 days late not accepted without prior arrangement.
- **Academic Concession:** Requests handled in accordance with UBC Policy V-135. Contact the instructor as soon as the need arises.

Course Resources

Online Textbook (OER)

The course textbook is a free, open-access **Quarto Book** published on GitHub Pages. Each chapter corresponds to one week of the course. The textbook serves as both pre-reading and reference material. It is version-controlled, searchable, and updated each term.
Link will be posted on Canvas before the first class.

Survival Guide & FAQ

A living document maintained on Canvas (and in the course GitHub repository) addressing the most common technical issues: Codespace launch failures, Git merge conflicts, rendering errors, AI tool access, package installation, rendering problems, and Codespaces usage management. Updated weekly by the teaching team based on issues encountered in class.

Required Hardware (BYOD Policy)

This course adopts a **Cloud-First** approach. Because we use GitHub Codespaces, you do not need a high-performance computer. Any device capable of running a modern web browser (including Chromebooks, older laptops, or tablets with a keyboard) is sufficient. GitHub Codespaces is the required and supported computing environment; other platforms (e.g., Colab, Kaggle) are not supported for graded work unless explicitly approved.

Bring your device to every class. Come fully charged, as classroom outlets may be limited.

Need a device? Students can borrow from the UBC Library.

Codespaces: Hours and Best Practices

GitHub Codespaces usage is measured in **core-hours**, not clock hours. At the time this outline was prepared, GitHub Free personal accounts included a monthly quota of Codespaces use; on a typical 2-core Codespace this is approximately **60 hours of active runtime per month**. GitHub may change quotas, so the Canvas Survival Guide will link to the current GitHub documentation and show students how to check their remaining usage.

This is sufficient for the course if managed well:

- **Always stop your Codespace** when you finish working (github.com/codespaces -> three dots -> Stop). Leaving a tab open wastes credits.
- **Reopen existing Codespaces** rather than creating new ones each session.
- **Use the default course machine type** unless instructed otherwise; larger machines consume core-hours faster.
- **If you run out of hours**, contact the teaching team for guidance before adding billing information. Plan your work accordingly.
- **If Codespaces is slow or unavailable** (rare), the Survival Guide includes contingency steps. We will communicate via Canvas if a platform-wide outage affects class.

Open Science & Portfolio Policy

A key goal of this course is to ensure you leave with more than just a grade. By the end of the term, you will have a public-facing **Data Science Portfolio** hosted on GitHub Pages featuring real-world health data analysis - a tangible asset for your CV or LinkedIn.

- **Default:** We encourage all students to publish final projects as public repositories.
- **Your privacy:** You may submit as a private repository (shared only with the teaching team) without any penalty. Communicate with the instructor by Week 4 if you opt for this.

University Policies

UBC provides resources to support student learning and to maintain healthy lifestyles but recognizes that sometimes crises arise and so there are additional resources to access including those for survivors of sexual violence. UBC values respect for the person and ideas of all members of the academic community. Harassment and discrimination are not tolerated nor is suppression of academic freedom. UBC provides appropriate accommodation for students with disabilities and for religious observances. UBC values academic honesty and students are expected to acknowledge the ideas generated by others and to uphold the highest academic standards in all of their actions.

Details of the policies and how to access support are available on the UBC Senate website.

Other Course Policies

Plagiarism

Students are expected to review the Student Discipline section of the UBC Calendar and know what constitutes plagiarism and academic misconduct. Such activities are subject to penalty.

Generative AI Policy (Permissive but Regulated)

Generative AI tools (e.g., GitHub Copilot) are **permitted and encouraged** in this course. However, you must adhere to the AI-Augmented Workflow guidelines:

- **Attribution:** Acknowledge the use of AI tools in your submissions (e.g., "Code block X was generated by Copilot and debugged by me").
- **Accountability:** You are fully responsible for any errors, biases, or security flaws in the code you submit. "The AI made a mistake" is not a valid excuse.
- **Verification:** Always audit AI output against official documentation, the course notes, and your own data.
- **Audit trail:** For major assignments and the final portfolio, include a brief AI-use note describing what the tool helped with, what you changed, and how you verified the result.

Learning Analytics

This course uses Canvas, which captures data about student activity. The instructor plans to use analytics data to: view overall class progress, track students' progress for personalized feedback, review content access statistics, and assess participation.

Copyright

All materials of this course (handouts, slides, assessments, readings, etc.) are the intellectual property of the Course Instructor or licensed for use in this course by the copyright owner. Redistribution without permission constitutes a breach of copyright and may lead to academic discipline.

Recording of class sessions by students is not permitted. Selected recordings by the instructor/TAs will be released on Canvas for viewing outside class. Contact the instructor immediately with any objections.

Syllabus Inventory Completion Matrix

Last updated after milestone/presentation/final-portfolio completion pass: 2026-05-14.

Summary

- Week/module pages ready: 15 of 15 tracked entries (Week 0, Weeks 1-13, Final Portfolio).
- Weekly assignments ready: A1-A10.
- Project milestones ready: M0-M5 plus Final Portfolio.
- Navigation state: student-facing assignments, milestones, and final portfolio brief are surfaced in `_quarto.yml`; instructor-only keys remain hidden.
- Workflow decision: fork-based repository setup is the official student workflow. GitHub Classroom is not the default route for this book.

Remaining Human-Only Release Items

These items require live course infrastructure rather than more drafting:

- Canvas orientation video link.
- Canvas pre-course survey link.
- Canvas course template repository link.
- Final presentation order, room, and live schedule.

Week And Module Matrix

Entry	Topic	Primary source	Status	Evidence and notes
Week 0	Asynchronous onboarding	`weeks/week00-onboarding/index.qmd`	Ready	Includes Canvas-posted orientation video/survey instructions, GitHub account guidance, fork workflow, support path, and Week 1 preparation.
Week 1	Health Data, KT & Ethics	`weeks/week01-health-data-ethics/index.qmd`; `activity-data-intake-card.qmd`	Ready	Data Intake Card and NHANES data spine frame provenance, ethics, stewardship, privacy, and KT.

Entry	Topic	Primary source	Status	Evidence and notes
Week 2	Modern Workflows	`weeks/week02-modern-workflows/index.qmd`; workflow subpages; `assignments/assignment01-workflows/README.md`	Ready	Student path is template repo, Fork, Codespaces from fork, commit/sync, submit fork link on Canvas.
Week 3	R with AI	`weeks/week03-r-with-ai/index.qmd`; `assignments/assignment02-r-with-ai/README.md`	Ready	Adds minimal `renv::status()`, `renv::snapshot()`, and `renv::restore()` awareness aligned with the dependency note.
Week 4	Git & Collaboration	`weeks/week04-git-collaboration/index.qmd`; `assignments/assignment03-git/README.md`; `milestones/m0-group-formation/README.md`	Ready	Group repo setup now uses a forked group template and collaborator access; M0 is fully specified.

Entry	Topic	Primary source	Status	Evidence and notes
Week 5	Polyglot Awareness & R Deepening	`weeks/week05-polyglot-r-deepening/index.qmd`; `assignments/assignment04-polyglot/README.md`	Ready	R-first parity workflow, Python awareness, dependency notes, and AI audit expectations are present.
Week 6	Data Visualization	`weeks/week06-visualization/index.qmd`; worked example; activity; A5	Ready	Worked ggplot example, flawed-plot audit, instructor key, M1 link, and final A5 brief are present.
Week 7	EDA, Table 1 & AI Auditing	`weeks/week07-eda-ai-audit/index.qmd`; worked example; activity; A6	Ready	Table 1 example, planted-error starter/key, M2 link, and final A6 brief are present.

Entry	Topic	Primary source	Status	Evidence and notes
Week 8	Dashboard Prototypes for KT	`weeks/week08-dashboard-kt/index.qmd`; `starter-dashboard.qmd`; A7	Ready	Fast-forward week; Shiny exemplar, starter dashboard, testing, privacy-safe guidance, and A7 brief are present.
Week 9	Communication of Scientific Findings	`weeks/week09-audience-communication/index.qmd`; `revealjs-peer-feedback.qmd`; `starter-slides.qmd`; A8	Ready	Runnable revealjs starter, three-slide structure, plain-language activity, peer feedback form, and accessibility/citation checklist are present.

Entry	Topic	Primary source	Status	Evidence and notes
Week 10	Writing & Publishing Reports	`weeks/week10-reporting-publishing/index.qmd`; `report-template.qmd`; A9; M3	Ready	Report template has YAML, citation workflow, generated figure/table, publishing/private preview checklist, and grounding audit.
Week 11	Portfolio Surgery	`weeks/week11-portfolio-surgery/index.qmd`; `reproducibility-test.qmd`; A10; M4; Final Portfolio	Ready	Step-by-step runbook, sample failure log, final portfolio checklist link, and optional Python pathway note are present.

Entry	Topic	Primary source	Status	Evidence and notes
Week 12	Presentations Part 1	`weeks/week12-presentations/index.qmd`; M5	Ready	Timing, rules, slide submission, rubric, peer feedback form, slide checklist, and portfolio reminder are present.
Week 13	Presentations Part 2	`weeks/week13-presentations/index.qmd`; M5; Final Portfolio	Ready	Presentation logistics, closing reflection, final portfolio reminders, and final revision priorities are present.

Entry	Topic	Primary source	Status	Evidence and notes
Final	Final Portfolio	`milestones/final-portfolio/README.md`; `final-portfolio-checklist.md`; `repository-sharing-guidance.md`	Ready	Brief, checklist, public/private repository guidance, deliverables, Canvas route, AI-use expectations, and rubric are present.

Assignment Matrix

Assignment	Week	Source	Status	Notes
A1	2	`assignments/assignment01-workflows/README.md`	Ready	Fork-based workflow, render, commit/sync, and fork link submission.

Assignment	Week	Source	Status	Notes
A2	3	`assignments/assignment02-r-with-ai/README.md`	Ready	R import/inspection, dplyr, function, AI audit, dependency note, and `renv::status()` awareness.
A3	4	`assignments/assignment03-git/README.md`	Ready	Git, branches, pull request evidence, documentation edit, and M0 readiness.
A4	5	`assignments/assignment04-polyglot/README.md`	Ready	R/Python parity, helper files, dependency note, and AI-use note.
A5	6	`assignments/assignment05-visualization/README.md`	Ready	Flawed plot audit, corrected plot, caption, and M1 link.

Assignment	Week	Source	Status	Notes
A6	7	`assignments/assignment06-data-ai-audit/README.md`	Ready	Table-1, planted-error audit, stewardship note, and M2 link.
A7	8	`assignments/assignment07-dashboard/README.md`	Ready	Dashboard-style KT product, testing, audience note, limitation, and privacy check.
A8	9	`assignments/assignment08-communication/README.md`	Ready	Slide deck/outline, plain-language summary, peer feedback, source note, and AI-use note.

Assignment	Week	Source	Status	Notes
A9	10	`assignments/assignment09-reporting/README.md`	Ready	Quarto report, citation workflow, publishing checklist, grounding audit, and M3 link.
A10	11	`assignments/assignment10-portfolio-surgery/README.md`	Ready	Fresh-environment test, failure log, peer review, README repair, and final risk list.

Milestone Matrix

Milestone	Source	Status	Notes
M0	`milestones/m0-group-formation/README.md`	Ready	Group formation, shared fork, access evidence, communication plan, Codespace check, and AI-use note.

Milestone	Source	Status	Notes
M1	`milestones/m1-proposal/README.md`	Ready	Proposal, Data Intake Card, initial visual plan, reproducibility plan, and AI-use note.
M2	`milestones/m2-preliminary-analysis/README.md`	Ready	Preliminary analysis, rendered output, Table 1, missingness, methods note, and AI audit.
M3	`milestones/m3-project-update/README.md`	Ready	Report draft, dashboard preview, reproducibility note, feedback request, and AI-use note.
M4	`milestones/m4-peer-review/README.md`	Ready	Peer review, reproducibility check, auditability check, KT feedback, and reviewer reflection.

Milestone	Source	Status	Notes
M5	`milestones/m5-presentation/README.md`	Ready	Slides, timing, rubric, source/limitations note, feedback record, and AI-use note.
Final Portfolio	`milestones/final-portfolio/README.md`	Ready	Final report, dashboard-style KT product, checklist, repository access, sharing guidance, and AI-use audit.

Navigation Notes

- `_quarto.yml` lists `assignments/index.qmd` and A1-A10 README pages directly.
- `_quarto.yml` lists `milestones/index.qmd`, M0-M5, and Final Portfolio.
- Instructor-only answer keys under week `instructor/` folders are intentionally not listed.
- Starter templates may be linked from assignment/week pages without being separate navigation chapters.

Syllabus-To-Content Coverage Audit

Last updated after milestone/presentation/final-portfolio completion pass: 2026-05-14.

Executive Summary

- Complete topic groups: 15 of 15 tracked entries.
- Weekly assignment briefs complete: A1-A10.
- Project milestones complete: M0-M5 plus Final Portfolio.
- Student-facing navigation: assignments, milestones, and final portfolio are surfaced.
- Remaining human-only release items: Canvas orientation video link, Canvas pre-course survey link, Canvas course template repository link, and final presentation order/room.

Highest-Priority Release Checks

1. Post the orientation video in Canvas and link it from the Week 0 Canvas module.
2. Post the pre-course survey in Canvas and verify access from a student account.
3. Post the course template repository link in Canvas for the fork workflow.
4. Post final presentation order, room, and live schedule in Canvas.
5. Re-render the full book after any live Canvas wording changes.

Coverage Matrix

Week 0

- **Topic name:** Asynchronous Onboarding
- **Status:** Complete
- **Evidence:** `weeks/week00-onboarding/index.qmd` / `Required Tasks`: "Watch the orientation video posted in Canvas"; `Course Repository Entry Point`: "Open the course template repository linked from Canvas"; `Support Path`: Canvas support channel with error details.
- **What is missing:** Live Canvas URLs are external to the repository.
- **Recommended next additions:** Add the actual Canvas video, survey, and template-repository links inside Canvas.

Week 1

- **Topic name:** Health Data, KT & Ethics
- **Status:** Complete
- **Evidence:** `weeks/week01-health-data-ethics/index.qmd` / `Learning Objectives`; `weeks/week01-health-data-ethics/activity-data-intake-card.qmd` / Data Intake Card activity and NHANES example.
- **What is missing:** No content drafting gap.
- **Recommended next additions:** Refresh portal links before term.

Week 2

- **Topic name:** Modern Workflows
- **Status:** Complete
- **Evidence:** `weeks/week02-modern-workflows/index.qmd` / `Getting Started: GitHub Account and Fork`: "Click Fork"; `Assignment 1` brief: "Submit your forked GitHub repository link on Canvas."
- **What is missing:** No content drafting gap.

- **Recommended next additions:** Keep screenshots aligned with GitHub's current interface.

Week 3

- **Topic name:** R with AI
- **Status:** Complete
- **Evidence:** `weeks/week03-r-with-ai/index.qmd` / `Minimal renv Checkpoint`: `renv::status()`, `renv::snapshot()`, `renv::restore()`; `assignments/assignment02-r-with-ai/README.md` / `Reproducibility Requirements`.
- **What is missing:** No content drafting gap.
- **Recommended next additions:** Recheck package environment before release.

Week 4

- **Topic name:** Git & Collaboration
- **Status:** Complete
- **Evidence:** `weeks/week04-git-collaboration/index.qmd` / `Setting Up Your Group Repository`: group fork and collaborator workflow; `milestones/m0-group-formation/README.md`: access evidence and communication plan.
- **What is missing:** No content drafting gap.
- **Recommended next additions:** Verify group template repository permissions before M0 opens.

Week 5

- **Topic name:** Polyglot Awareness & R Deepening
- **Status:** Complete
- **Evidence:** `weeks/week05-polyglot-r-deepening/index.qmd` / R-first parity workflow; `assignments/assignment04-polyglot/README.md` / parity table, notebook hygiene, dependency note, and AI-use note.
- **What is missing:** No content drafting gap.
- **Recommended next additions:** Test Python examples in Codespaces before release.

Week 6

- **Topic name:** Data Visualization
- **Status:** Complete
- **Evidence:** `weeks/week06-visualization/worked-example-ggplot.qmd`: runnable ggplot example; `activity-ai-visual-audit.qmd`: audit deliverables; `instructor/answer-key-visual-audit.md`: planted issues and corrections; A5 brief.
- **What is missing:** No content drafting gap.
- **Recommended next additions:** Re-render the worked example if the NHANES CSV changes.

Week 7

- **Topic name:** EDA, Table 1 & AI Auditing
- **Status:** Complete
- **Evidence:** `weeks/week07-eda-ai-audit/worked-example-table1.qmd`: cohort definition, missingness, Table 1-style summary; `instructor/planted-error-key.md`: planted error corrections; A6 brief.
- **What is missing:** No content drafting gap.
- **Recommended next additions:** Keep the unweighted/descriptive limitation visible.

Week 8

- **Topic name:** Dashboard Prototypes for KT
- **Status:** Complete
- **Evidence:** `weeks/week08-dashboard-kt/index.qmd`: first app-forward week; `starter-dashboard.qmd`: audience question, control, visualization, interpretation, caution; A7 brief.
- **What is missing:** No content drafting gap.
- **Recommended next additions:** Test the Shiny exemplar in Codespaces before Week 8.

Week 9

- **Topic name:** Communication of Scientific Findings

- **Status:** Complete
- **Evidence:** `weeks/week09-science-communication/index.qmd` / `Plain-Language Translation`; `starter-slides.qmd`: revealjs YAML and three-slide KT story; `revealjs-peer-feedback.qmd`: peer feedback and accessibility checklist; A8 brief.
- **What is missing:** No content drafting gap.
- **Recommended next additions:** Verify slide rendering after any revealjs theme changes.

Week 10

- **Topic name:** Writing & Publishing Reports
- **Status:** Complete
- **Evidence:** `weeks/week10-reporting-publishing/report-template.qmd`: YAML, citation to `@cdc-nhanes`, one table, one figure, limitations, publishing checklist, grounding audit; A9 brief; M3 brief.
- **What is missing:** No content drafting gap.
- **Recommended next additions:** Recheck `references.bib` if source citations change.

Week 11

- **Topic name:** Portfolio Surgery & Optional Advanced Deployment
- **Status:** Complete
- **Evidence:** `weeks/week11-portfolio-surgery/reproducibility-test.qmd`: step-by-step runbook, sample failure log, blank failure log, optional Python pathway, final portfolio checklist link; A10 and M4 briefs.
- **What is missing:** No content drafting gap.
- **Recommended next additions:** Run one fresh Codespace smoke test before portfolio week.

Week 12

- **Topic name:** Term Project Presentations, Part 1
- **Status:** Complete

- **Evidence:** `weeks/week12-presentations/index.qmd`: timing rules, slide submission paths, rubric, peer feedback form, slide checklist, portfolio reminder; M5 brief.
- **What is missing:** Live presentation order and room are external to the repository.
- **Recommended next additions:** Post live presentation order in Canvas.

Week 13

- **Topic name:** Term Project Presentations, Part 2
- **Status:** Complete
- **Evidence:** `weeks/week13-presentations/index.qmd`: timing rules, slide submission, closing reflection, final portfolio reminder, revision priorities; M5 and Final Portfolio links.
- **What is missing:** Live presentation order and room are external to the repository.
- **Recommended next additions:** Post live presentation order in Canvas.

Final Portfolio

- **Topic name:** Final Portfolio
- **Status:** Complete
- **Evidence:** `milestones/final-portfolio/README.md`: required artifacts, Canvas route, public/private repository guidance, reproducibility checklist, AI-use expectations, grading checklist; `final-portfolio-checklist.md`; `repository-sharing-guidance.md`.
- **What is missing:** Live Canvas submission link is external to the repository.
- **Recommended next additions:** Verify course staff access for private repositories before grading.

Deliverables Alignment

Deliverable	Status	Evidence
A1	Complete	`assignments/assignment01-workflows/README.md` uses fork repository workflow and Canvas fork-link submission.
A2	Complete	`assignments/assignment02-r-with-ai/README.md` includes R workflow, AI audit, dependency note, and `renv::status()` awareness.
A3	Complete	`assignments/assignment03-git/README.md` covers commits, diffs, branches, pull request evidence, and M0 readiness.
A4	Complete	`assignments/assignment04-polyglot/README.md` covers R/Python parity and dependency hygiene.
A5	Complete	`assignments/assignment05-visualization/README.md` covers ggplot, flawed plot audit, caption, and AI-use note.
A6	Complete	`assignments/assignment06-eda-ai-audit/README.md` covers EDA, Table 1, planted-error audit, and stewardship note.
A7	Complete	`assignments/assignment07-dashboard/README.md` covers dashboard-style KT product and testing.

Deliverable	Status	Evidence
A8	Complete	`assignments/assignment08-communication/README.md` covers slide deck/outline, plain-language summary, peer feedback, and source note.
A9	Complete	`assignments/assignment09-reporting/README.md` covers Quarto report, citations, publishing/private preview, and grounding audit.
A10	Complete	`assignments/assignment10-portfolio-surgery/README.md` covers fresh environment testing, failure log, peer review, and final risk list.
M0	Complete	`milestones/m0-group-formation/README.md` includes group formation, access, communication plan, Codespace check, and AI-use note.
M1	Complete	`milestones/m1-proposal/README.md` includes proposal, Data Intake Card, visual plan, reproducibility plan, and AI-use note.

Deliverable	Status	Evidence
M2	Complete	`milestones/m2-preliminary-analysis/README.md` includes preliminary analysis, rendered output, Table 1, missingness, methods, and audit note.
M3	Complete	`milestones/m3-project-update/README.md` includes report draft, dashboard preview, feedback request, reproducibility note, and AI-use note.
M4	Complete	`milestones/m4-peer-review/README.md` includes peer review, reproducibility check, auditability check, KT feedback, and reflection.
M5	Complete	`milestones/m5-presentation/README.md` includes slides, timing, rubric, source/limitations note, feedback, and AI-use note.
Final Portfolio	Complete	`milestones/final-portfolio/README.md`, `final-portfolio-checklist.md`, and `repository-sharing-guidance.md`.

Consistency Checks

- Fork workflow mismatch is resolved: Week 0, Week 2, A1, syllabus-facing text, and Course Operations all use fork-first language.
- Student-facing assignments A1-A10 are listed in `_quarto.yml`.
- Student-facing milestones M0-M5 and Final Portfolio are listed in `_quarto.yml`.
- Instructor-only files in Week 6 and Week 7 remain outside `_quarto.yml`.
- Starter files are linked from week or assignment pages rather than promoted as main chapters unless they are core student-facing pages.

Appendix A: Files Scanned

- `syllabus/SPPH-381H-Course-Outline-v4.qmd`
- `syllabus/syllabus-inventory-completion-matrix.md`
- `weeks/week00-onboarding/index.qmd`
- `weeks/week01-health-data-ethics/**`
- `weeks/week02-modern-workflows/**`
- `weeks/week03-r-with-ai/**`
- `weeks/week04-git-collaboration/**`
- `weeks/week05-polyglot-r-deepening/**`
- `weeks/week06-visualization/**`
- `weeks/week07-eda-ai-audit/**`
- `weeks/week08-dashboard-kt/**`
- `weeks/week09-science-communication/**`
- `weeks/week10-reporting-publishing/**`
- `weeks/week11-portfolio-surgery/**`
- `weeks/week12-presentations/index.qmd`
- `weeks/week13-presentations/index.qmd`
- `assignments/assignment01-workflows/README.md` through `assignments/assignment10-portfolio-surgery/README.md`
- `milestones/**`
- `_quarto.yml`
- `course-operations/index.qmd`

Appendix B: Keyword Patterns Used

- Workflow: `fork`, `Codespaces`, `Quarto`, `render`, `commit`, `sync`, `relative paths`
- R and reproducibility: `R`, `dplyr`, `read_csv`, `renv`, `status`, `snapshot`, `restore`
- Ethics and data: `provenance`, `stewardship`, `privacy`, `security`, `OCAP`, `CARE`, `responsible AI`
- Visualization and EDA: `ggplot2`, `plotly`, `visual audit`, `misleading`, `Table 1`, `missingness`, `planted-error`
- KT products: `dashboard`, `Shiny`, `plain language`, `revealjs`, `report`, `citations`, `grounding audit`
- Portfolio: `fresh Codespace`, `failure log`, `peer review`, `presentation`, `final portfolio`, `AI-use note`

Week 0: Asynchronous Onboarding

Week 0: Asynchronous Onboarding

Purpose

Week 0 gets the account setup and orientation work out of the way before the first class. You do not need to install local software for this course. The default workflow uses a personal fork of the course template repository, GitHub Codespaces, Quarto, and Canvas.

Required Tasks

Complete these steps before Week 1:

1. Log in to Canvas and open the Week 0 module.
2. Watch the orientation video posted in Canvas.
3. Complete the pre-course survey posted in Canvas.
4. Create or confirm access to a GitHub account.
5. Submit the Week 0 Canvas onboarding check with your GitHub username.

GitHub Account

Use a professional GitHub username that you are comfortable sharing in a course setting. If you already have a GitHub account, you may use it. You do not need to create a second account unless your current username or profile is not appropriate for coursework.

Before submitting the onboarding check:

- confirm you can log in to GitHub
- turn on two-factor authentication if GitHub prompts you
- make sure you know which email address is attached to the account
- record your GitHub username exactly as it appears in the profile URL

Existing-Account Help

If you already have a GitHub account but cannot access it:

- try GitHub account recovery before creating a new account
- use the email address most likely attached to the account
- check whether your browser is logged into a different personal account
- ask for help through the Canvas course support channel if you are blocked

If you create a new account, use that account consistently for course repositories.

Course Repository Entry Point

The current course workflow uses a fork-based setup:

1. Open the course template repository linked from Canvas.
2. Click **Fork**.
3. Create a personal fork under the GitHub account you submitted in the onboarding check.
4. Launch Codespaces from your fork.
5. Commit and sync changes to your fork.
6. Submit your fork repository link on Canvas.

If a future offering uses an instructor-managed repository system, Canvas will say so explicitly. For this course book, the fork-based workflow is the default student route.

Support Path

Use the Canvas course support channel for setup problems. Include:

- your GitHub username
- the step where you got stuck
- the exact error message or screenshot
- whether you are using GitHub, Codespaces, or Canvas

Do not post passwords, tokens, or private account recovery details.

What To Bring To Week 1

Bring:

- a working Canvas login
- a working GitHub login
- your GitHub username
- a laptop or access plan for in-class work
- one question about health data, public data, AI, or reproducibility

Definition Of Done

Week 0 is done when you have watched the Canvas orientation video, completed the Canvas pre-course survey, submitted the onboarding check, and can log in to GitHub with the account you will use for course work.

Week 1: Health Data, KT & Ethics

Health Data & Ethics

Overview

Week 1 sets the ethical and communication foundation for the whole course. Health data science does not end when code runs or a p-value appears; it ends when evidence is translated responsibly for a real audience. This week introduces open health data, data provenance, stewardship, privacy, responsible AI use, Indigenous Data Sovereignty awareness, and Knowledge Translation (KT) as the "last mile" of analysis.

This is an awareness-heavy week. There is no coding, no dashboard running, and no app setup. The NHANES case study appears only as a data/provenance artifact so you can practice evaluating a dataset before using it.

Learning Objectives

By the end of this week, you should be able to:

- identify credible open health data portals;
- distinguish microdata from aggregated indicators;
- complete a Data Intake Card for a public health dataset;
- record basic provenance, licensing, access, and citation information;
- name privacy, security, and stewardship risks that can still apply to public data;
- explain why AI summaries must be checked against source documentation;
- recognize when Indigenous Data Sovereignty, OCAP, or CARE principles may be relevant;
- frame a dataset as the start of a KT product, not just as a file to analyze.

Connection

This first week asks, "Should we use this data, and how should we describe it?" Next week asks, "How do we organize the files so another person can reproduce our work?" Later weeks return to the same NHANES case study through analysis, visualization, EDA, and dashboard design.

Case Study Data Spine

The **NHANES Health Equity data spine** is the recurring dataset thread for this course. In the early weeks, use it only as a documented public-health data artifact: provenance, ethics, file organization, and reproducible paths.

- Cached RDS: `examples/nhanes-equity/data/nhanes_equity_v6.rds`
- CSV snapshot: `examples/nhanes-equity/data/nhanes_equity_v6.csv`
- Case-study README: `examples/nhanes-equity/README.md`

The cached files are provided so early work can happen without internet access.

```r
library(readr)
library(dplyr)

candidate_paths <- c(
  "examples/nhanes-equity/data/nhanes_equity_v6.csv",
  "../../examples/nhanes-equity/data/nhanes_equity_v6.csv"
)

nhanes_path <- candidate_paths[file.exists(candidate_paths)][1]
if (is.na(nhanes_path)) {
  stop("Could not find examples/nhanes-equity/data/nhanes_equity_v6.csv")
}

nhanes_spine <- read_csv(nhanes_path, show_col_types = FALSE)

dim(nhanes_spine)
#> [1] 105626     18
```

```r
glimpse(nhanes_spine)
#> Rows: 105,626
#> Columns: 18
#> $ Cycle          <chr> "1999-2000", "1999-2000", "1999-2000", "1999-2000", "19~
#> $ BMI            <dbl> 14.90, 24.90, 17.63, NA, 29.10, 22.56, 29.39, 15.51, 18~
#> $ Weight         <dbl> 12.5, 75.4, 32.9, 13.3, 92.5, 59.2, 78.0, 40.7, 45.5, 1~
#> $ Height         <dbl> 91.6, 174.0, 136.6, NA, 178.3, 162.0, 162.9, 162.0, 156~
#> $ Waist          <dbl> 45.7, 98.0, 64.7, NA, 99.9, 81.6, 90.7, 64.1, 64.6, 108~
#> $ WeightMEC      <dbl> 10982.899, 28325.385, 46192.257, 10251.260, 99445.066, ~
#> $ Strata         <dbl> 5, 1, 7, 2, 8, 2, 4, 6, 9, 7, 1, 6, 13, 12, 11, 11, 5, ~
#> $ PSU            <dbl> 1, 3, 2, 1, 2, 2, 2, 1, 2, 1, 2, 2, 2, 1, 2, 1, 1, 1, 2~
#> $ Gender         <chr> "Female", "Male", "Female", "Male", "Male", "Female", "~
#> $ Race           <chr> "Non-Hispanic Black", "Non-Hispanic White", "Non-Hispan~
#> $ Education      <chr> NA, "College Grad", NA, NA, "College Grad", NA, "HS or ~
#> $ Marital        <chr> NA, NA, NA, NA, "Married/Partner", "Never Married", "Ma~
#> $ Age            <dbl> 2, 77, 10, 1, 49, 19, 59, 13, 11, 43, 15, 37, 70, 81, 3~
#> $ PIR            <dbl> 0.86, 5.00, 1.47, 0.57, 5.00, 1.21, NA, 0.53, NA, NA, 1~
#> $ EducationClean <chr> NA, "College Grad", NA, NA, "College Grad", NA, "HS or ~
#> $ MaritalClean   <chr> NA, NA, NA, NA, "Married/Partner", "Never Married", "Ma~
#> $ IncomeGroup    <chr> "Low Income (<1.3)", "High Income (>3.5)", "Middle Inco~
#> $ WHtR           <dbl> 0.4989083, 0.5632184, 0.4736457, NA, 0.5602916, 0.50370~
```

For Week 1, treat the NHANES case study as a provenance and stewardship example only. Do not run the app. Do not rebuild the data. Do not upload row-level data to an AI tool.

In-Class Activity

Use Activity: Data Intake Card.

1. Choose one approved public health dataset or use the shared NHANES case-study example.
2. Find the dataset publisher, documentation, access page, and citation guidance.
3. Complete the Data Intake Card.
4. Write one KT framing sentence: who could use this dataset, and what decision or conversation could it support?

5. Name one ethics, privacy, security, or stewardship issue that should be checked before publishing results.

Student output: a completed Data Intake Card and one KT framing sentence.

Open Data Portals

Open data can accelerate public health learning when the source, meaning, and limits of the data are clear. Start with reputable portals that provide documentation, terms of use, and stable links.

Recommended starter portals:

- CDC/NCHS public-use survey data, such as NHANES or NHIS;
- WHO Global Health Observatory for aggregated international indicators;
- BC Data Catalogue for provincial open data;
- Government of Canada Open Government Portal for Canadian datasets.

When you land on a dataset page, look for:

- publisher or steward;
- population, geography, and time coverage;
- data level, such as microdata or aggregated indicators;
- documentation, codebook, or methodology notes;
- license or terms of use;
- citation recommendation;
- last updated date or access date.

Provenance

Provenance means the traceable story of where data came from and how it reached your project. A dataset is not just a filename. It has a steward, collection context, transformations, limitations, and documentation.

For every dataset, record:

- original source page;
- local file path if a copy is stored in the repository;
- date accessed;
- file format;
- variables you plan to use;
- known limitations or cautions;
- citation or attribution language.

The Data Intake Card is the course habit that makes provenance visible.

Stewardship

Stewardship means using data in a way that respects the people, communities, and institutions represented by the data. Public data still requires care.

Ask:

- Could a group be stigmatized by a careless summary?
- Are small cells or rare combinations being exposed?
- Does the dataset include populations for whom special governance principles may apply?
- Are limitations visible enough for a non-technical audience?
- Does the planned KT product support understanding rather than overclaiming?

Privacy And Security

In this course, Week 1 uses public, de-identified, or aggregated data only. Still, avoid treating "public" as the same as "risk-free."

Course rules:

- Do not upload row-level health data to AI tools.
- Do not attempt re-identification.

- Do not publish small-cell claims.
- Do not copy data into public repositories unless the license and course instructions allow it.
- Do not use absolute local paths that reveal a personal machine or user name.

Responsible AI

AI tools may help you draft plain-language summaries or identify questions to ask about documentation. They are not source documentation.

Acceptable Week 1 AI use:

- asking for a plain-language explanation of a term from a codebook;
- drafting a first pass of a dataset description;
- generating a checklist of questions to verify.

Required checks:

- cite the official source, not the AI output;
- verify variable meanings in documentation;
- include an AI-use note if AI helped draft your card;
- revise AI language that overstates certainty or hides limitations.

Indigenous Data Sovereignty

Indigenous Data Sovereignty refers to the rights and interests of Indigenous Peoples in data about their peoples, lands, resources, and knowledges. Two frameworks students should recognize are:

- OCAP: ownership, control, access, and possession, specific to First Nations contexts in Canada;
- CARE: collective benefit, authority to control, responsibility, and ethics.

Week 1 expectation: if a dataset includes Indigenous identifiers, communities, lands, or governance-relevant content, flag that on the Data Intake Card and review the source guidance before using or publishing analysis.

Formats And Metadata

Common formats:

- CSV: rectangular rows and columns;
- JSON: nested data, common for APIs;
- XLSX: spreadsheet files that may contain formatting and hidden assumptions;
- RDS: R-specific saved objects;
- GeoJSON or shapefiles: spatial data for maps.

Metadata explains what the file means. It may include variable definitions, units, missing-value codes, collection methods, survey design variables, and weighting instructions. A dataset without metadata is risky for analysis because column names alone rarely tell the full story.

KT Framing

KT asks who needs the information, what decision or conversation it supports, and how the evidence should be communicated. A Week 1 KT framing sentence can be simple:

> This dataset could help [audience] understand [health issue] so they can [decision, planning task, or conversation], with the limitation that [key caution].

Example:

> The NHANES classroom dataset could help students understand how BMI summaries differ across income groups while practicing clear limitations about descriptive, unweighted survey summaries.

What Students Leave With

By the end of Week 1, you should have:

- one completed Data Intake Card;
- one KT framing sentence;
- one ethics or stewardship caution;
- a clear understanding that the NHANES case study starts as a data artifact, not an app to run;
- readiness to place files into a reproducible project structure in Week 2.

Activity: Data Intake Card

Goal

Evaluate whether a health dataset is usable, ethical, documented, and appropriate for reproducible analysis. This activity is about judgment and documentation. No coding is required.

Approved Portals

Choose one dataset or indicator page from one of these portals:

- CDC/NCHS public-use survey data, such as NHANES or NHIS;
- WHO Global Health Observatory;
- BC Data Catalogue;
- Government of Canada Open Government Portal.

The NHANES Health Equity data spine may be used as the shared classroom example.

Student Template

Copy this template into your notes.

```
dataset_name: ""
publisher_org: ""
permalink_url: ""
date_accessed: "YYYY-MM-DD"
license_terms: ""
```

```yaml
geography: ""
time_coverage: ""
data_level: ""   # microdata / aggregated indicator / unknown
file_format: ""  # CSV / RDS / XLSX / JSON / other
metadata_or_codebook: ""
variables_of_interest:
  - name: ""
    definition: ""
    unit: ""
    caveat: ""
known_limitations:
  - ""
privacy_security_notes: ""
stewardship_notes: ""
indigenous_data_relevance: "no / yes / unknown"
kt_framing_sentence: ""
how_to_cite: ""
ai_use_note: ""
```

Completed Example

```yaml
dataset_name: "NHANES Health Equity classroom dataset"
publisher_org: "CDC / National Center for Health Statistics"
permalink_url: "https://wwwn.cdc.gov/nchs/nhanes/"
date_accessed: "YYYY-MM-DD"
license_terms: "Public-use NHANES files; cite CDC/NCHS documentation"
geography: "United States"
time_coverage: "Multiple NHANES cycles represented in the prepared class file"
data_level: "Microdata-derived classroom dataset"
file_format: "CSV and RDS cached in examples/nhanes-equity/data/"
metadata_or_codebook: "CDC/NCHS NHANES documentation and case-study README"
variables_of_interest:
  - name: "BMI"
    definition: "Body mass index from NHANES body measures data"
    unit: "kg/m^2"
    caveat: "Use descriptively unless survey design and weights are handled correctly"
known_limitations:
  - "Prepared class dataset is for classroom analysis and should not be treated as a causal ana
```

```
  - "Population inference requires attention to NHANES survey design and weights"
privacy_security_notes: "Use public-use data responsibly, avoid small-cell claims, and do not u
stewardship_notes: "State limitations plainly and avoid stigmatizing group comparisons"
indigenous_data_relevance: "unknown in the prepared class example; check source documentation i
kt_framing_sentence: "This dataset can help students practice describing health equity patterns
how_to_cite: "Cite CDC/NCHS NHANES documentation and the course case-study README"
ai_use_note: "AI may help draft plain-language wording, but source documentation must be checke
```

Completion Checklist

To be marked complete, your card must include:

- stable source link or access date;
- publisher or steward;
- license or terms of use;
- geography and time coverage;
- data level and file format;
- documentation or codebook location;
- at least one documented variable definition;
- one limitation or stewardship risk;
- one privacy or security note;
- Indigenous Data Sovereignty relevance marked as no, yes, or unknown;
- KT framing sentence;
- AI-use note.

Scoring Guide

Rating	Evidence
Complete	Source, publisher, terms, metadata, variable definition, limitation, stewardship note, KT framing, and AI-use note are present and grounded in documentation.

Rating	Evidence
Needs work	Source documentation is missing, license terms are unclear, variable definitions are guessed, risks are absent, or AI-written text is not checked against official documentation.

Submission

Submit the completed card in Canvas under the Week 1 module. Starting in Week 2, keep a copy in your course repository as:

```
data-intake-card.md
```

Reproducibility Check

Before submitting, check:

- the link opens or the access date is recorded;
- the publisher and license are named;
- variable definitions come from documentation, not guesses;
- file format and data level are named;
- the AI-use note says what AI helped with and how the text was checked.

Week 2: Modern Workflows

Modern Workflows

Overview

Summary: This week you set up the computational environment you will use for the rest of the course. By the end, the technology should fade into the background so you can focus on health data science, not installation errors. We adopt a **"Zero to AI Co-Pilot"** approach: you do not need to install R, Python, or Git on your personal laptop. Everything runs in a cloud-native environment called GitHub Codespaces.

Learning Objectives:

- Understand the "public computer vs cloud drive" analogy for cloud computing.
- Create a GitHub account, fork the course template repository, and work from your personal fork.
- Launch a Codespace and identify the three key areas of the VS Code interface.
- Distinguish between an IDE, a notebook, and a script, and explain why this course uses Quarto notebooks.
- Explain the concept of literate programming and why it matters for reproducible health research.
- Edit and render a Quarto (`.qmd`) document: YAML header, Markdown text, and code chunks.
- Maintain a reproducible project structure using the `data/` and `output/` folder convention.
- Stage, commit, and sync changes using the VS Code Source Control panel.
- Stop your Codespace to conserve your free monthly hours.

Connection

Week 1 focused on whether a dataset is trustworthy enough to use. This week focuses on where that dataset lives in a project and how relative paths make the same work portable across machines. Next week you will start inspecting the data with R.

Case Study Data Spine

The **NHANES Health Equity data spine** is the recurring dataset thread for this course. In the early weeks, use it only as a documented public-health data artifact: provenance, ethics, file organization, and reproducible paths.

- Cached RDS: `examples/nhanes-equity/data/nhanes_equity_v6.rds`
- CSV snapshot: `examples/nhanes-equity/data/nhanes_equity_v6.csv`
- Case-study README: `examples/nhanes-equity/README.md`

The cached files are provided so early work can happen without internet access.

```r
library(readr)
library(dplyr)

candidate_paths <- c(
  "examples/nhanes-equity/data/nhanes_equity_v6.csv",
  "../../examples/nhanes-equity/data/nhanes_equity_v6.csv"
)

nhanes_path <- candidate_paths[file.exists(candidate_paths)][1]
if (is.na(nhanes_path)) {
  stop("Could not find examples/nhanes-equity/data/nhanes_equity_v6.csv")
}

nhanes_spine <- read_csv(nhanes_path, show_col_types = FALSE)

dim(nhanes_spine)
#> [1] 105626     18
```

```r
glimpse(nhanes_spine)
#> Rows: 105,626
#> Columns: 18
#> $ Cycle          <chr> "1999-2000", "1999-2000", "1999-2000", "1999-2000", "19~
#> $ BMI            <dbl> 14.90, 24.90, 17.63, NA, 29.10, 22.56, 29.39, 15.51, 18~
#> $ Weight         <dbl> 12.5, 75.4, 32.9, 13.3, 92.5, 59.2, 78.0, 40.7, 45.5, 1~
#> $ Height         <dbl> 91.6, 174.0, 136.6, NA, 178.3, 162.0, 162.9, 162.0, 156~
#> $ Waist          <dbl> 45.7, 98.0, 64.7, NA, 99.9, 81.6, 90.7, 64.1, 64.6, 108~
#> $ WeightMEC      <dbl> 10982.899, 28325.385, 46192.257, 10251.260, 99445.066, ~
#> $ Strata         <dbl> 5, 1, 7, 2, 8, 2, 4, 6, 9, 7, 1, 6, 13, 12, 11, 11, 5, ~
#> $ PSU            <dbl> 1, 3, 2, 1, 2, 2, 2, 1, 2, 1, 2, 2, 2, 1, 2, 1, 1, 1, 2~
#> $ Gender         <chr> "Female", "Male", "Female", "Male", "Male", "Female", "~
#> $ Race           <chr> "Non-Hispanic Black", "Non-Hispanic White", "Non-Hispan~
#> $ Education      <chr> NA, "College Grad", NA, NA, "College Grad", NA, "HS or ~
#> $ Marital        <chr> NA, NA, NA, NA, "Married/Partner", "Never Married", "Ma~
#> $ Age            <dbl> 2, 77, 10, 1, 49, 19, 59, 13, 11, 43, 15, 37, 70, 81, 3~
#> $ PIR            <dbl> 0.86, 5.00, 1.47, 0.57, 5.00, 1.21, NA, 0.53, NA, NA, 1~
#> $ EducationClean <chr> NA, "College Grad", NA, NA, "College Grad", NA, "HS or ~
#> $ MaritalClean   <chr> NA, NA, NA, NA, "Married/Partner", "Never Married", "Ma~
#> $ IncomeGroup    <chr> "Low Income (<1.3)", "High Income (>3.5)", "Middle Inco~
#> $ WHtR           <dbl> 0.4989083, 0.5632184, 0.4736457, NA, 0.5602916, 0.50370~
```

In-Class Activity

- Locate the case-study data, README, and snippet folders in the Explorer.
- Explain why the data example uses relative paths instead of machine-specific paths.
- Check that the weekly page includes the shared snippet rather than duplicating instructions.
- Render the book from the repository root and check that this page updates in the rendered output.

Student output: a rendered Quarto page and one commit showing a small, reproducible edit. Assignment link: Assignment 1.

`workflows1.qmd` The Analogy: Public Computer and Cloud Drive

To understand how we work in this course, forget about "servers" and "clients." Imagine you are working in a university computer lab.

The Repository (Your Cloud Drive)

Think of **GitHub** as your personal **Google Drive or Dropbox**.

- It is where your files live permanently.
- If your laptop crashes, the files are still here.
- **Key concept:** This is your "Master Copy."

The Codespace (The Public Computer)

Think of a **Codespace** as a **public computer in the library**.

- It is a temporary machine you borrow to do your work.
- It comes pre-installed with all the software you need (R, Python, Quarto), so you don't have to install anything on your own laptop.
- **Crucial difference:** Just like a public library computer, it can be wiped clean. If you close the browser without sending your work back to your Cloud Drive, your changes may be lost.

The Workflow

1. **Log in:** Launch your Codespace (open the public computer).
2. **Work:** Edit files using the computer's software.
3. **Save to the cloud:** Commit and push your work back to your Cloud Drive (GitHub).
4. **Log off:** Stop the Codespace to save your free hours.

This cycle — launch, work, commit, stop — is the rhythm of every week in this course.

`workflows2.qmd` Getting Started: GitHub Account and Fork

Step 1: Create a GitHub Account

If you do not already have one:

1. Go to github.com and click **Sign up**.
2. Use your UBC email address (this helps with student benefits).
3. Choose a professional username — this will appear on your public portfolio later in the course.

> **Tip**
>
> **Portfolio thinking:** Your GitHub profile will become part of your professional presence. Pick a username you would be comfortable putting on a CV (e.g., `jchen-epi` rather than `xXcoolcoder99Xx`).

Step 2: Fork the Course Template

1. Open the course template repository linked from Canvas.
2. Click **Fork** in the upper-right corner of the GitHub page.
3. Create the fork under your own GitHub account.
4. Keep the default repository name unless Canvas gives a specific naming rule.
5. After the fork is created, make sure the page URL includes your GitHub username.

Step 3: Verify Your Repository

Navigate to your forked repository. You should see:

- A file list including `practice_report.qmd`, `README.md`, and a `data/` folder.
- A formatted README below the file list explaining the project.

If you see these in your fork, you are ready to launch your Codespace.

`workflows3.qmd` Launching Your Codespace

1. On your repository page, click the green **<> Code** button.
2. Select the **Codespaces** tab.
3. Click **Create codespace on main**.

The first launch takes 2–3 minutes while the environment builds. Subsequent launches are faster.

> **Note**
>
> **Reopening an existing Codespace:** After you stop or close your Codespace, do **not** create a new one each time. Go to github.com/codespaces, find your existing Codespace, and click its name to restart it. Creating multiple Codespaces wastes your hours and causes confusion.

`workflows4.qmd` Tour of VS Code: The "Big Three" Zones

The VS Code interface can look intimidating — it is designed for professional software engineers. You only need to know three areas. Safely ignore 90% of the buttons.

1. The Explorer (Left Sidebar)

Your **file cabinet**. It shows folders and files in your project. You will use this to open `.qmd` files, navigate the `data/` folder, and create new files.

2. The Editor (Center)

Your **work area**. This is where you read and write code and text. You can have multiple files open in tabs.

3. The Terminal (Bottom Panel)

The **engine room** where R and Python actually run. You will not type commands here often — most of your work happens in the Editor — but this is where output and error messages appear.

> Tip
>
> **Lost a panel?** Use the top menu: **View → Explorer** or **View → Terminal** to bring them back. You can also press `` Ctrl+` `` (backtick) to toggle the Terminal.

`workflows5.qmd` Concepts: IDE, Notebook, Script

The syllabus mentions three ways to write code. Understanding the difference helps you choose the right tool throughout the course.

IDE (Integrated Development Environment)

VS Code is an **IDE** — a text editor with superpowers: file browsing, syntax highlighting, built-in terminal, extensions, and version control. It is the container that holds everything else.

Script

A plain text file containing only code (e.g., `.R` or `.py`). Runs top-to-bottom. Best for reusable utilities and automation. You will write your first script in Week 5.

Notebook

A document that **interleaves code and narrative**. You run code in chunks and see results inline. Examples include Jupyter notebooks (`.ipynb`, Week 5) and Quarto documents (`.qmd`, which you use starting today).

This course primarily uses Quarto notebooks because they combine the strengths of notebooks (interactivity, narrative) with the ability to produce publication-quality reports, slides, and websites — all from a single file.

`workflows6.qmd` Concepts: Literate Programming

The Big Idea

Traditional workflows separate code from writing: you run an analysis in one program, copy results into a Word document, and hope nothing changes. **Literate programming** weaves code, results, and interpretation into a single document. When the data changes, you re-render and the entire report updates automatically.

This is not just convenient — it is a **scientific obligation**. In health research, reviewers and regulators need to see exactly how you went from raw data to conclusions. A literate document is a transparent, auditable record of your reasoning.

How Quarto Implements This

A `.qmd` file has three ingredients:

1. **YAML header** (metadata at the top, between `---` lines)
2. **Markdown text** (human-readable prose)
3. **Code chunks** (executable R or Python code in fenced blocks)

When you **Render**, Quarto runs every code chunk, captures the output (tables, figures, numbers), and weaves everything into a finished HTML, PDF, or Word document.

You will use literate programming in every subsequent week of this course: your assignments, your EDA (Week 7), your reports (Week 10), and your final portfolio are all Quarto documents.

`workflows7.qmd` Project Structure: Files and Folders

A reproducible project needs a consistent folder structure. This course uses the following convention:

```
my-project/
  data/           ← Raw data. Never edit files here directly.
  output/         ← Cleaned data, figures, and tables generated by your code.
  practice_report.qmd   ← Your Quarto notebook.
  README.md       ← Explains what this project is.
  .gitignore      ← Tells Git which files to ignore (more in Week 4).
```

Rules

- **`data/` is read-only.** Import from it; never save into it.
- **`output/` is generated.** Your code writes cleaned data and figures here. In Week 3, you will export `output/clean_data.csv`.
- **Use relative paths.** Always refer to files relative to the project root (e.g., `"data/patients.csv"`), never absolute paths (e.g., `"/home/user/Documents/patients.csv"`). This ensures your code runs on any machine.

Activity: In the Explorer pane, click the **New Folder** icon and create the `output/` folder if it does not already exist.

`workflows8.qmd` Tutorial: Your First Quarto Document

Step 1: Open the Starter File

In the Explorer, click `practice_report.qmd`. It opens in the Editor.

Step 2: Understand the YAML Header

At the very top, between `---` lines, you will see something like:

```
---
title: "My First Report"
author: "Your Name"
format: html
---
```

Change `author:` to your actual name. The YAML header controls metadata and output format.

Step 3: Read the Markdown

Below the YAML, you will see plain text with formatting:

- `# Heading` creates a section title.
- `**bold**` makes text **bold**.
- `- item` creates a bullet point.

This is **Markdown** — a lightweight way to format text without a word processor.

Step 4: Read the Code Chunks

Look for grey blocks that start with ```` ```{r} ```` and end with ```` ``` ````. These are **code chunks**. Everything inside them is R code that will execute when you render.

````
```{r}
# This is a code chunk
2 + 2
```
````

Step 5: Edit a Code Chunk

Find a code chunk in the starter file. Make a small change — for example, change a number or add a comment. This is your first edit.

Step 6: Render

Click the **Render** button (blue arrow icon at the top of the editor). A preview window appears showing your formatted report with code output embedded.

> Tip
>
> **What just happened:** Quarto ran every code chunk, captured the results, combined them with your Markdown text, and produced an HTML document. This is literate programming in action.

`workflows9.qmd` Tutorial: Stage, Commit, and Sync

Your work currently exists only on the public computer (Codespace). To save it permanently to your Cloud Drive (GitHub), you need to **commit**.

Step 1: Open Source Control

Click the **Source Control** icon on the left sidebar (it looks like a branching graph). You will see a list of files that have changed since your last commit.

Step 2: Stage Your Changes

Hover over a changed file name. Click the + icon that appears. This **stages** the file — it tells Git "I want to include this change in my next snapshot."

> **Note**
>
> **Staging vs committing:** Staging selects *which* changes to include. Committing takes the snapshot. Think of staging as putting items on the conveyor belt; committing is pressing the "scan" button at checkout.

Step 3: Write a Commit Message

In the text box at the top of the Source Control panel, type a short, descriptive message:

> Updated author name and edited first code chunk in practice report

A good commit message answers: *what did I change and why?*

Step 4: Commit

Click the **Commit** button (checkmark). Your changes are now saved as a snapshot in your local history.

Step 5: Sync with GitHub

Click **Sync Changes** (in the Source Control panel or the status bar at the bottom). This pushes your commit to GitHub — your Cloud Drive.

Verify: Go to your repository on github.com and check that your changes appear. You should see your commit message next to the updated file.

> ⚠ Warning
>
> **If you skip the Sync step**, your commit exists only on the Codespace. If the Codespace is deleted, the commit goes with it. Always Sync.

`workflows10.qmd` Stopping Your Codespace (Saving Credits)

GitHub Codespaces gives you a monthly allowance of free hours (approximately 60 core-hours). **If you leave the browser tab open, the meter keeps running**, even if you are not actively working.

The Right Way to Stop

1. Go to github.com/codespaces.
2. Find your active Codespace.
3. Click the **three dots (...)** → **Stop codespace**.

What Happens If You Just Close the Tab?

The Codespace runs for another 30 minutes before it times out automatically. That is 30 minutes of wasted credits every time.

> ⚠ Warning
>
> **Policy:** It is your responsibility to manage your compute hours. If you run out before the month resets, you will need to wait or pay out of pocket. Get in the habit of stopping your Codespace at the end of every session.

`workflows11.qmd` Troubleshooting

Sometimes things break. Here is a graduated approach — try the simplest fix first.

Level 1: Reload the Window

Press `F1` (or `Ctrl+Shift+P` / `Cmd+Shift+P`) to open the **Command Palette**. Type `Reload Window` and select it. This restarts VS Code without rebuilding the entire environment.

Level 2: Restart the R or Python Session

If R or Python is frozen, open the Command Palette → type `R: Restart R Session` (or the equivalent for Python). This clears the session without touching your files.

Level 3: Rebuild the Container (Nuclear Option)

If nothing else works:

1. Open the **Command Palette**.
2. Type `Rebuild Container`.
3. Select **Codespaces: Rebuild Container**.

This shuts down and rebuilds the entire environment from scratch. It solves nearly all technical glitches but takes a few minutes.

> ⚠ Warning
>
> **Before rebuilding:** Make sure you have committed and synced your work. Uncommitted changes may survive a rebuild, but there is no guarantee.

`workflows12.qmd` Knowledge Check

1. In the analogy, what is the "Cloud Drive" and what is the "Public Computer"?
2. Why don't you need to install R or Python on your laptop for this course?
3. Name the "Big Three" zones of the VS Code interface and what each is for.
4. What is the difference between an IDE, a notebook, and a script?
5. What is literate programming, and why does it matter for health research?
6. What are the three ingredients of a `.qmd` file?
7. What is the difference between **staging** and **committing**?
8. Why must you click **Sync Changes** after committing?
9. How do you properly stop a Codespace to save your free hours?

`workflows13.qmd` Assignment 1: Your First Quarto Report

Due Friday 4 pm. Submit via Canvas.

Task

Edit and render the starter `practice_report.qmd`, demonstrating that you can navigate your Codespace, edit Quarto, and commit your work.

Step-by-Step Workflow

1. **Launch** your Codespace from your forked repository.
2. **Edit the YAML:** Change `author:` to your name.

3. **Add a Markdown section:** Below the existing content, add a new heading (`## My Notes`) and write 2–3 sentences about what you learned this week. Use at least one Markdown feature (bold, italics, or a bullet list).
4. **Edit a code chunk:** Modify an existing code chunk or add a new one. For example:

````
```{r}
# My first R calculation
2 + 2
```
````

5. **Render** the document. Check that the output looks correct.
6. **Commit and Sync:**
   - Open Source Control.
   - Stage your changed files (+).
   - Write a commit message: e.g., `"Complete Week 2 assignment: edit YAML, add notes, render report"`.
   - Click **Commit**, then **Sync Changes**.
7. **Stop** your Codespace.
8. **Verify** on github.com that your commit appears in the repository.
9. **Submit** your forked repository link on Canvas.

`workflows14.qmd` Quick Reference

Action	How
Launch Codespace	Repository → green <> **Code** → Codespaces → **Create codespace on main**
Reopen Codespace	github.com/codespaces → click Codespace name

Action	How
Stop Codespace	github.com/codespaces → **...** → **Stop codespace**
Open Explorer	**View** → **Explorer** or click file icon (left sidebar)
Open Terminal	**View** → **Terminal** or `Ctrl+``
Render Quarto	Click **Render** button (blue arrow) at top of editor
Stage a file	Source Control → hover file → click +
Commit	Type message → click
Sync to GitHub	Click **Sync Changes**
Reload window	Command Palette (`F1`) → `Reload Window`
Rebuild container	Command Palette (`F1`) → `Codespaces: Rebuild Container`

The Analogy: Public Computer vs Cloud Drive

To understand the workflow, imagine working in a university computer lab.

Repository (Cloud Drive)

Important Distinction: While GitHub is your "Cloud Drive," it does **NOT** work like Google Drive.

- **Google Drive:** Auto-saves every keystroke.
- **GitHub:** Works like **Email**. You can write a draft (save in Codespace), but the cloud never sees it until you hit "Send" (Commit & Push).
- Stores your files permanently

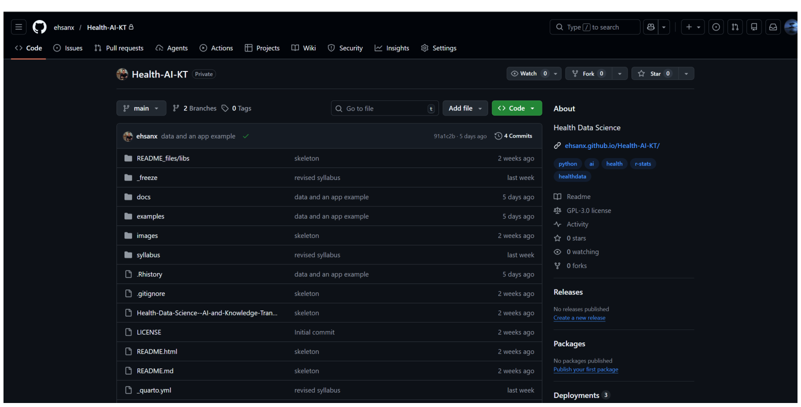


Figure 1: The code will be stored and accessible on GitHub as long as the company exists. Given that it's owned by a large corporation like Microsoft, it will likely last for a long time.

- Safe if your computer crashes
- This is your **master copy**

Codespace (Public Computer)

Codespaces is like a library computer:

- **Temporary machine** It is a disposable environment. If it breaks, we just delete it and get a new one.

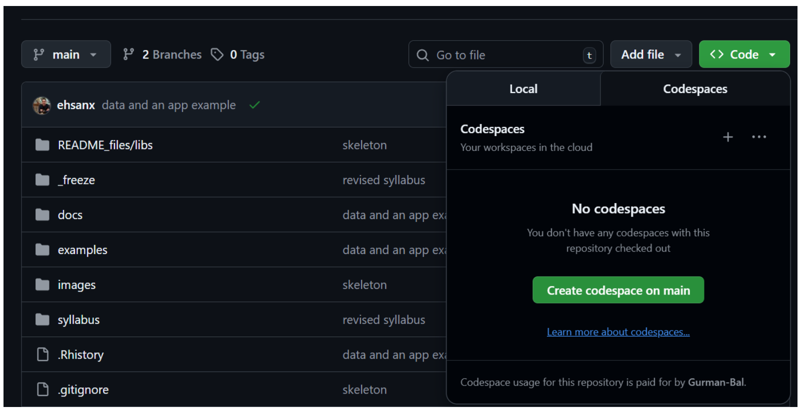


Figure 2: Can be accessed by clicking the `<> Code` button, navigating to the Codespaces tab, and then starting up a Codespace which contains all the file in the Repo.

- **Pre-installed software** It comes with R, Python, and Quarto installed, so you don't have to install anything on your own laptop.
- **The "Two-Save" Rule (Crucial)** Because this is a temporary computer, there are two steps to saving:

  1. **Ctrl+S (Save):** This only saves to the temporary machine. If the machine times out (closes), this is lost.
  2. **Commit & Push:** This sends your work back to the Repository (Cloud Drive). This is the only way to be safe.

Warning: If you close the tab without “Committing” (Sending), your work is erased.

Check for understanding

Answer:

- Where are your permanent files stored?
- What happens if you press `Ctrl+S` but never commit?

Tour of the Repository

Step-by-step

1. Open your forked course repository on GitHub

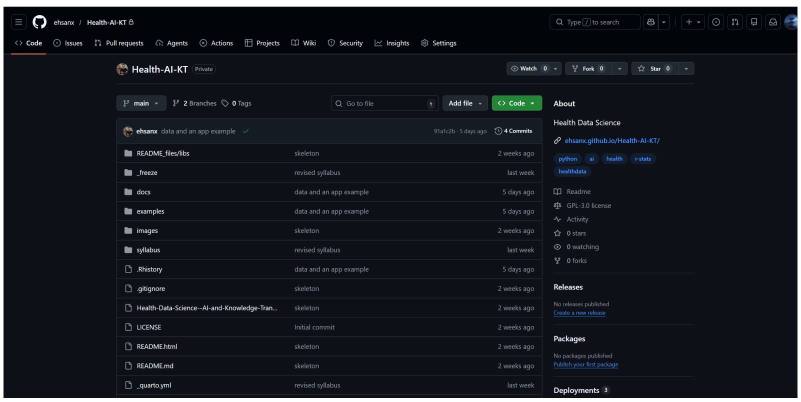
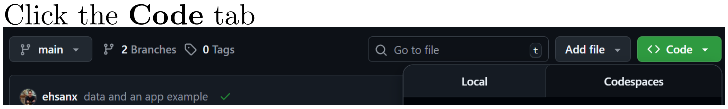


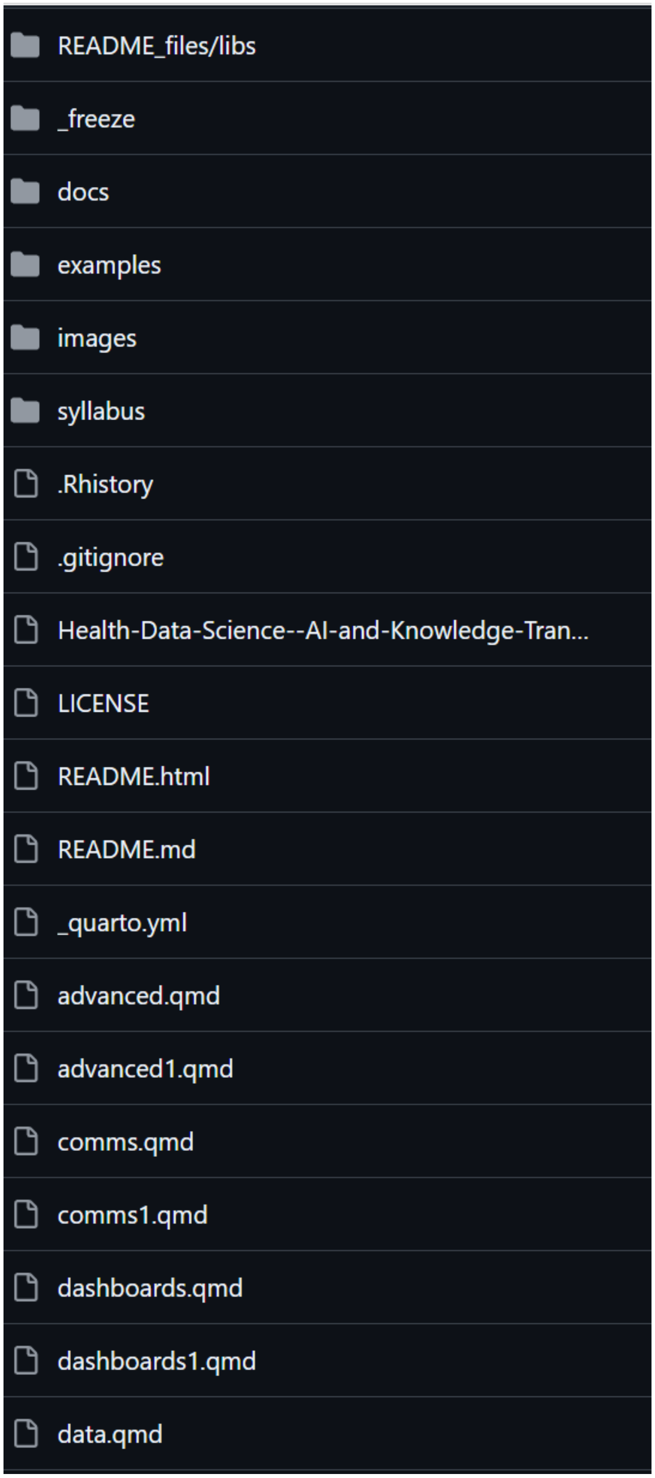
Figure 1: Main github page

2. Click the **Code** tab



   - **Why?** GitHub has many tabs (Issues, Actions, Settings). The **Code** tab is your "Reset" button. If you ever get lost in a menu, click this to go back to your files.

3. Look at the file list


README_files/libs
_freeze
docs
examples
images
syllabus
.Rhistory
.gitignore
Health-Data-Science--AI-and-Knowledge-Tran...
LICENSE
README.html
README.md
_quarto.yml
advanced.qmd
advanced1.qmd
comms.qmd
comms1.qmd
dashboards.qmd
dashboards1.qmd
data.qmd

    - This shows the **current state** of the project in your fork.
    - **Crucial:** If a file is listed here, it is safe. If it is *not* here, it only exists on your temporary "Desk" (Codespace) and is at risk of being lost.

4. Scroll to read the README

    - The `README.md` file is displayed automatically at the bottom. Think of it as the "Cover Sheet" or "Instruction Manual" for your project.

Task

Find this file:

```
practice_report.qmd
```

Verification: If you can see `practice_report.qmd` in this list, it means the file is safely stored in the cloud (The Vault).

If you can see it in your fork, it is safely stored in the cloud.

Opening Your First Codespace

This process is like walking into the library and turning on a computer. Start from your forked repository, then create a fresh, temporary environment in the cloud.

Steps

1. Click the green **Code** button

    - This button is usually for downloading files, but we are using it to launch a computer.

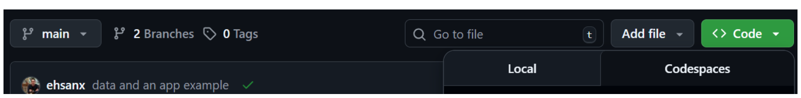


Figure 1: Code button

2. Click the **Codespaces** tab

    - **Crucial Choice:** You will see two tabs: "Local" and "Codespaces."
    - **Local:** Means downloading to your own messy laptop (Hard mode).
    - **Codespaces:** Means using the pre-installed cloud computer (Easy mode). Always choose this.

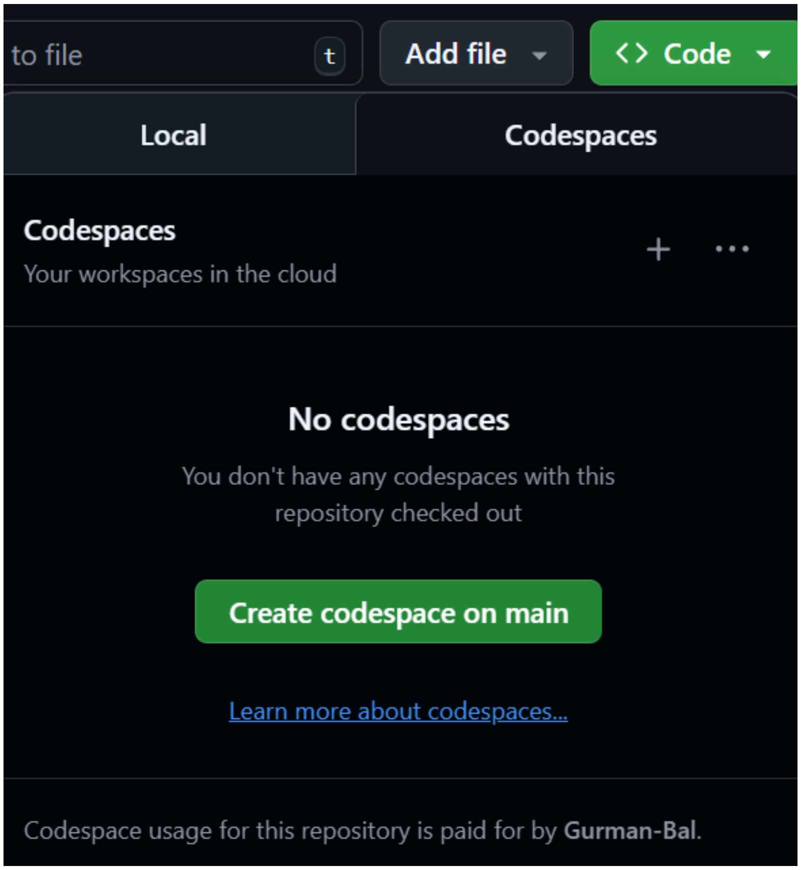


Figure 2: Codespaces tab

3. Click **Create Codespace on main**

    - This tells GitHub to build a new computer for you from scratch.

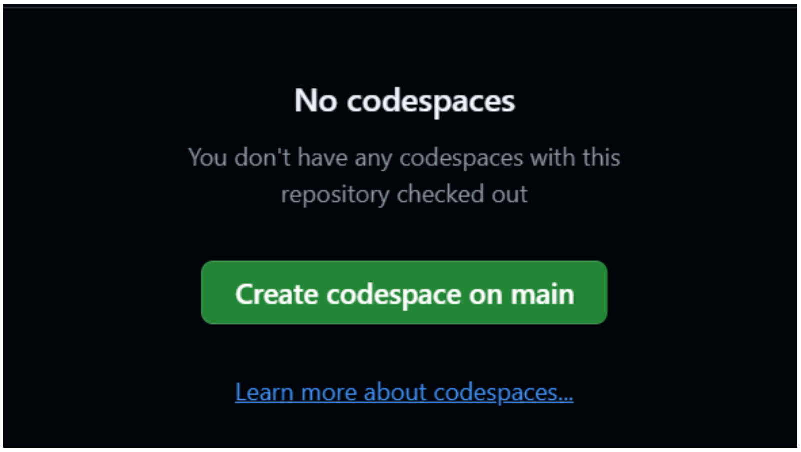


Figure 3: Create Codespace

The "Boot Up" Phase

Wait 2–3 minutes.

- **What is happening?** GitHub is silently installing R, Python, Quarto, and all the course libraries for you.
- **The Analogy:** Think of this as the "ventilation system" starting up in a lab. It takes a moment to get the environment ready.

Success: You should see **VS Code** (a dark-themed code editor) appear in your browser window.

Reopening a Codespace

The Rule: Do not create a new Codespace each time you log in.

- **Why?** If you click the green "Create" button again, GitHub builds a *second* computer.
- **The Risk:** If you left uncommitted work (Drafts) on "Computer A" and you start "Computer B," your work will appear lost. It isn't lost: it's just on the other computer.

To resume exactly where you left off:

Steps

1. Go to the **Codespace Dashboard**: https://github.com/codespaces

    - Think of this as the "Front Desk" where all your active computers are listed.

2. Find your Codespace

    - Look for the box labeled with your course repository name.
    - **Note:** GitHub gives each Codespace a funny random name (e.g., *literate-computing-machine* or *glorious-couscous*). This is the name of your rented desk.

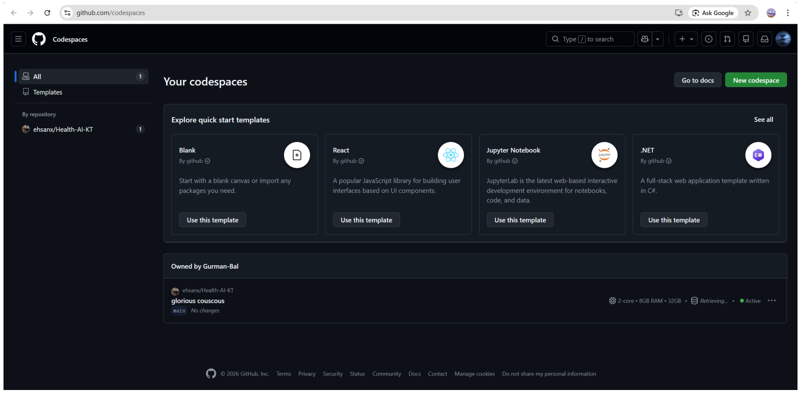


Figure 1: GitHub Codespaces

3. Click the **Name** of the Codespace

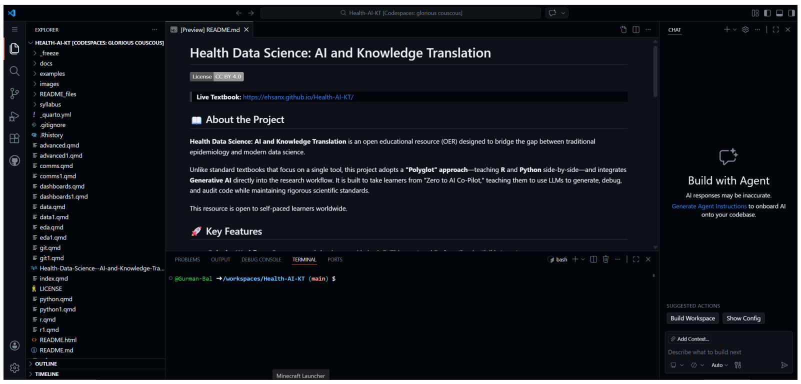
    - Do **not** click the “New codespace” button in the corner. Click the text link of the funny name.

Figure 2: Your Codespace

4. Wait for it to resume

    - **Benefit:** Resuming an existing Codespace is much faster than creating a fresh one because the software is already installed.

VS Code Orientation



When you open a Codespace, you are using a version of **VS Code** in your browser. It has the same interface as the desktop version used by professional software engineers at Google and Microsoft.

Don't Panic: The interface looks complex because it has hundreds of features. **You only need to know three areas.** You can safely ignore 90% of the buttons you see.

Explorer (left)

The "File Cabinet"

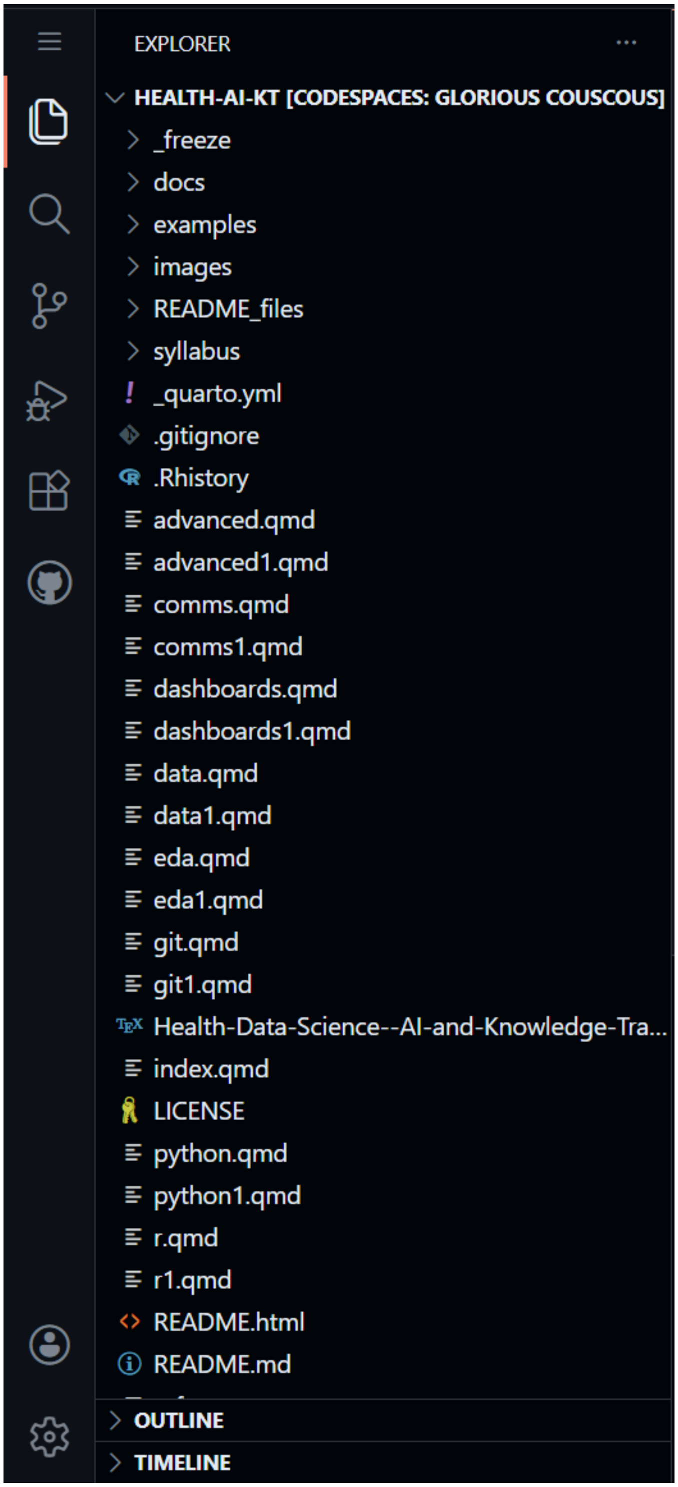


Figure 1: Shows files and folders.

- This shows your folders (directories) and files.
- Think of this as your navigation pane. Double-click a file here to open it in the Editor.

Editor (center)

The "Notebook"

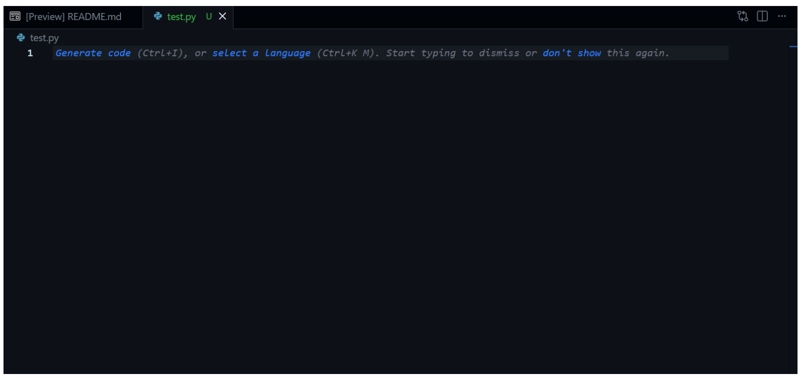


Figure 2: Where you type.

- This is where you do your actual work: typing code and writing text.
- In this course, you will mostly be editing `.qmd` (Quarto) files here.

Terminal (bottom)

The "Engine Room"

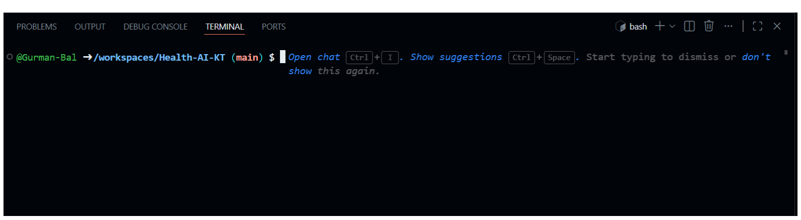


Figure 3: Terminal window

- This is where R and Python actually run.
- You will see text scrolling here when you "Render" a document. It looks like "The Matrix," but it is usually just the computer telling you what it is working on.

If the terminal disappears (or you accidentally close it):

View → Terminal

Mini Task

Open and close a few files in the Explorer to get comfortable with the layout.

Files vs Folders

We use a consistent structure to ensure reproducibility. In Data Science, being "organized" isn't just about being neat—it is a requirement for your code to work on someone else's machine.

The Golden Rule of Health Data

We treat data like a **Crime Scene**—never touch the evidence.

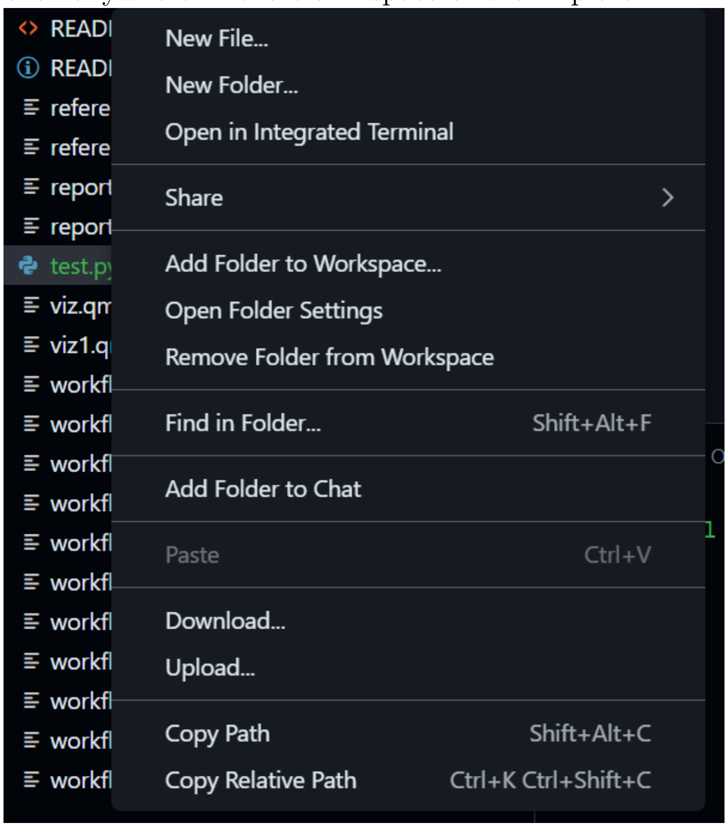
- **Start:** `data/` (The Crime Scene). **READ ONLY.**
  - *Rule:* Never open a dataset in Excel and "fix" a typo. That is tampering with evidence.
  - *Correct Way:* Load the "dirty" data into R/Python and fix the typo using code. This leaves a trail of exactly what you changed.
- **End:** `output/` (The Evidence). All graphs and cleaned tables go here.
- **The Detective:** Your code (`.qmd`) moves information from Start to End.

Files vs. Folders (The Mental Model)

- **Files (e.g., `report.qmd`):** These are **sheets of paper**. You write on them.
- **Folders (e.g., `data/`):** These are **labeled boxes**. You store things in them.

Create a folder

In Explorer:

1. Right-click anywhere in the blank space of the Explorer 

pane.
2. Select **New Folder**
3. Name it:

```
images
```

Pro Tip: Always use **lowercase** and **no spaces** for folder names (e.g., `my_data`, not `My Data`). Computers struggle with spaces in file paths.

Typical structure

Your project should always look like this:

```
data/      raw data
output/    generated results
images/    screenshots
.qmd       reports
```

Remember: The `data` folder is for *input*. The `output` folder is for *results*. They should never mix.

The Magic Moment: Render Quarto

Why do we "Render"?

In the old days, you would run code, copy a graph, save it as a JPG, and paste it into Word. If the data changed, you had to redo every step manually.

The Better Way: Think of your code (`.qmd` file) as a **Recipe** and the final report (HTML/PDF) as the **Meal**. * **Editing:** You don't change the meal; you change the recipe. * **Rendering:** This is the "cooking" process. When you click **Render**, the computer follows your instructions from scratch to create a fresh meal.

This is the secret to reproducibility: if you give your recipe to a friend, they can cook the exact same meal.

Step 1 — Open the file

1. Go to the **Explorer** pane (Left side).
2. Locate and click on the file:

```
practice_report.qmd
```

This will open the "Source Code" in the center Editor window. You will see a mix of normal text and code chunks.

Step 2 — Edit name (The YAML Header)

Look at the very top of the file. You will see a block of text sandwiched between three dashes (`---`). This is called the **YAML Header**. It controls the document's metadata.

Find the line that says `author:`

```
author: "Your Name"
```

Task: Delete "Your Name" and type your actual name inside the quotes. Do not remove the quotes!

Step 3 — Render

1. Look for the **Render** button at the top of the Editor pane (it often looks like a Blue Arrow pointing right, or simply says "Render").
2. **Click it.**
3. **Observe:**
    - **The Terminal (Bottom):** You will see text scrolling quickly. This is R/Python executing your code chunks.
    - **The Preview (Right):** A new window should appear showing a beautiful, formatted report with your name on it.

Congratulations! You just performed "Knowledge Translation"—turning raw code into a human-readable document.

If it fails (Troubleshooting)

Rendering involves many moving parts, so it sometimes gets stuck.

- **The “Wait” Rule:** If nothing happens for 20 seconds, the background process might be initializing. Give it a moment.
- **The “Retry” Rule:** Sometimes the connection hiccups. Click **Render** one more time.
- **The “Red Text” Rule:** Look at the **Terminal** tab at the bottom. If you see red text, it is an error message. Read it—it usually tells you exactly what went wrong (e.g., “File not found” or “Syntax error”).

Commit Your Work

Your work is currently only on the "Rented Desk" (Codespace). It is a **Draft**. To save it permanently to the "Vault" (GitHub), you must **Commit** and **Push** (Send).

The Analogy: Think of this like shipping a package. You have packed the box (saved the file), but now you need to put a label on it and hand it to the courier.

Steps

1. Click the **Source Control** icon
    - It looks like a **blue bubble** or a **connect-the-dots graph** on the far left sidebar.
    - This opens the "Shipping Department." You will see a list of every file you changed.

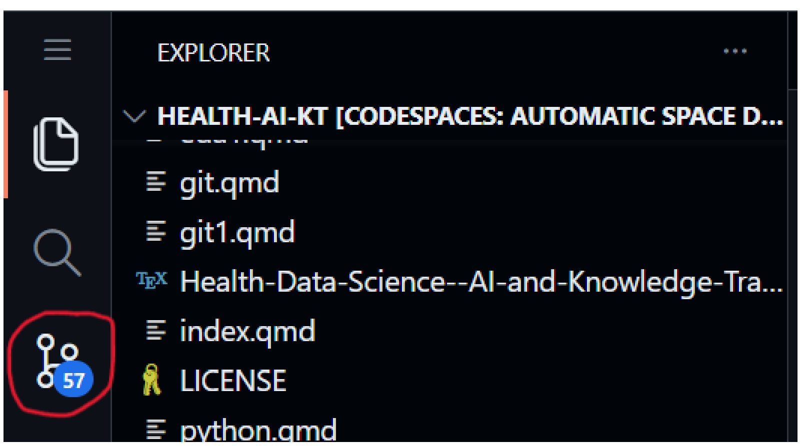


Figure 1: Source control icon circled in red

2. Type a **Commit Message**

   - **The Label:** You cannot ship a package without a label.
   - **What to write:** Be descriptive but brief. Imagine you are leaving a note for your future self about *what* you did.
   - *Example:* "Updated author name in practice report".
   - *Bad Example:* "stuff" or "fix".

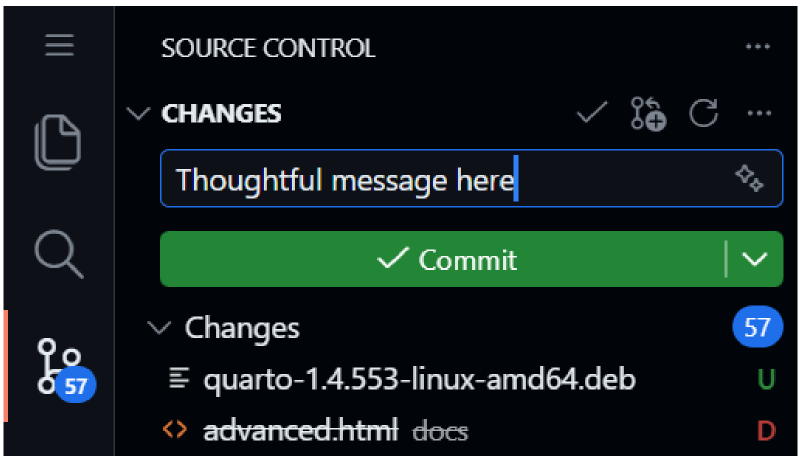


Figure 2: This is required

3. Click **Commit and Sync** (or Commit and Push)

   - This is the **"Send"** button.
   - **Action:** It wraps up your changes, stamps them with the time and your message, and teleports them to the Cloud Vault.
   - *Note:* You might be asked to click "Always" or "Yes" to approve the sync.

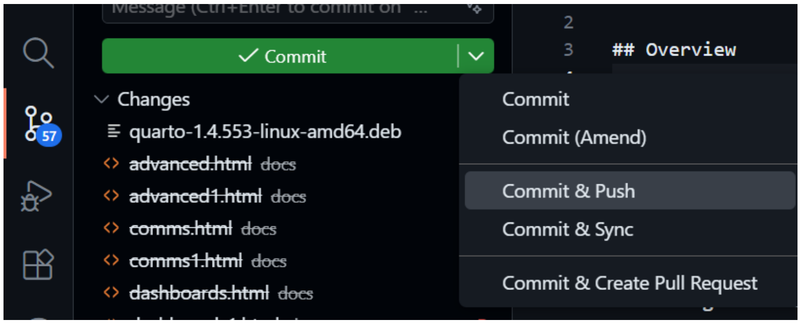


Figure 3: This will push your changes to the remote branch

Verify (Trust but Verify)

Did it actually work?

1. Go back to your **GitHub Repository** tab (The Vault).
2. **Refresh** the page.
3. Look at the file list. You should see your "Commit Message" displayed next to the file you just edited.

Troubleshooting: Rebuild Container

Sometimes the cloud computer gets "stuck." Maybe the terminal froze, or R crashed.

The Fix: We don't fix broken computers; we get new ones. "Rebuilding the Container" is the cloud equivalent of **"Have you tried turning it off and on again?"**—but better, because it gives you a fresh machine while keeping your files safe.

When to use this?

If things break:

- **Terminal frozen:** You type, but nothing appears.
- **Render fails:** You get weird errors that make no sense.
- **Environment issues:** R or Python can't find a package that was there yesterday.

Steps

1. **Open the Command Palette**
    - Press `F1` (or `Ctrl+Shift+P` on Windows / `Cmd+Shift+P` on Mac).
    - A search bar will appear at the very top of the screen. This is the "God Mode" search bar for VS Code.
2. **Search**
    - Type: `Rebuild Container`

3. **Execute**

    - Click on the option that says: **Codespaces: Rebuild Container**

4. **Wait (~2-5 minutes)**

    - The screen will reload.
    - **Don't Panic:** You might see a black screen with scrolling text logs. This is normal: it is reinstalling the software.
    - When your files reappear in the Explorer, you are ready to go.

Did I lose my work?

- **Saved files:** No. Anything you pressed `Ctrl+S` on is safe.
- **Unsaved typing:** Yes.
- **R Memory:** Yes. You will need to re-run your code chunks from the top.

Stop Your Codespace

Codespaces is not free forever. You have a monthly allowance (approx. 60 hours/month for free accounts).

The Analogy: Think of a Codespace like a **Taxi**.

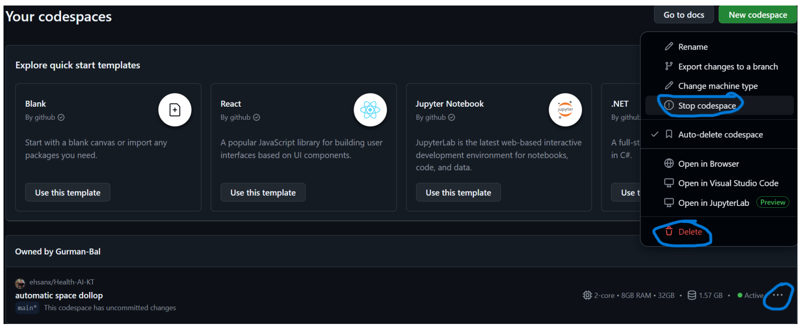
- **Working:** When you are coding, the meter is running.
- **The Trap:** If you just close your browser tab, it is like getting out of the taxi but asking the driver to "wait here." **The meter keeps running.**
- **The Fix:** You must explicitly "end the ride" (Stop the Codespace).

Correct shutdown

1. Go to the **Codespace Dashboard**: https://github.com/codespaces
   - This is your command center.
2. Find your active Codespace
   - Look for the green text that says "Active".
3. Click ... (The "Meatball" Menu)
   - It is located on the far right side of the Codespace box.
4. Click **Stop Codespace**
   - **Stop:** This "pauses" the computer. It saves your installed packages and RAM state so you can resume quickly next time. **(Recommended)**
   - **Delete:** This destroys the computer. You will have to wait 2-3 minutes for the "Boot Up" phase next time you start.

Critical Warning: Closing the browser tab is **NOT** enough.

- GitHub keeps the machine running for **30 minutes** after you close the tab (in case your internet flickered).
- **The Cost:** If you just close the tab every day, you will waste ~15 hours of your monthly quota on empty time.



Final Checklist

☐ I understand repository vs Codespace

☐ I created or located my forked course repository

☐ I rendered a `.qmd` file

☐ I committed changes

☐ I know how to rebuild container

☐ I know how to stop Codespace

Reflection

Write 2–3 sentences:

- What was easiest?
- What was confusing?
- What questions do you still have?

Week 3: R with AI

Week 3: R with AI

Overview

Summary: In Week 2 you learned how to use your Codespace, edit a Quarto document, and make your first commit. Now you learn how to speak R—not by memorizing syntax, but by directing an AI co-pilot and *auditing* what it writes. Your job is to be the pilot: you decide what the analysis should do, the AI drafts the code, and you verify every line before it flies.

Learning Objectives:

- Understand that R is a calculator that needs specific, case-sensitive instructions.
- Load packages with `library()` and explain why this is needed each session.
- Run a minimal `renv` checkpoint so package changes are visible.
- Use core constructs: variables (assignment), functions (with arguments), and the pipe (`%>%` / `|>`).
- Perform basic data transformations using `dplyr` verbs: `filter()`, `select()`, `mutate()`, `summarise()`, `group_by()`.
- Inspect a dataset on first contact: structure, dimensions, missingness.
- Identify control flow (`if/else`, loops) in AI-generated code and audit whether it is correct.
- Write and adapt a simple custom function.
- Apply an AI safety checklist to catch hallucinations, wrong column names, and unsafe operations.
- Document your analysis in Quarto by weaving code chunks with Markdown narrative.
- Commit incrementally as you work (reinforcing Week 2 Git basics).

Connection

Week 2 made the project structure reproducible. This week you use that structure to load, inspect, summarize, and audit R code with AI support. Next week you will put those edits under version control.

Case Study Data Analysis

The **NHANES Health Equity data spine** now supports code-reading, EDA, visualization, and reproducibility checks. Use the cached CSV/RDS for routine class work; a CDC retrieval script exists for advanced users.

- Cached RDS: `examples/nhanes-equity/data/nhanes_equity_v6.rds`
- CSV snapshot: `examples/nhanes-equity/data/nhanes_equity_v6.csv`
- Case-study README: `examples/nhanes-equity/README.md`

The default classroom path is to use the cached data so the analysis work is reproducible without internet access.

```r
library(readr)
library(dplyr)

candidate_paths <- c(
  "examples/nhanes-equity/data/nhanes_equity_v6.csv",
  "../../examples/nhanes-equity/data/nhanes_equity_v6.csv"
)

nhanes_path <- candidate_paths[file.exists(candidate_paths)][1]
if (is.na(nhanes_path)) {
  stop("Could not find examples/nhanes-equity/data/nhanes_equity_v6.csv")
}

nhanes_analysis <- read_csv(nhanes_path, show_col_types = FALSE)

nhanes_analysis |>
  summarise(
    rows = n(),
```

```
    missing_bmi = sum(is.na(BMI)),
    missing_income = sum(is.na(IncomeGroup))
  )
#> # A tibble: 1 x 3
#>     rows missing_bmi missing_income
#>    <int>       <int>          <int>
#> 1 105626        9356           9415

nhanes_analysis |>
  filter(!is.na(BMI), !is.na(IncomeGroup), Age >= 20, Age <= 80) |>
  group_by(IncomeGroup) |>
  summarise(
    n = n(),
    mean_bmi = round(mean(BMI), 1),
    .groups = "drop"
  )
#> # A tibble: 3 x 3
#>   IncomeGroup            n mean_bmi
#>   <chr>              <int>    <dbl>
#> 1 High Income (>3.5) 16330     28.6
#> 2 Low Income (<1.3)  15293     29.4
#> 3 Middle Income      19228     29.3
```

In-Class Activity

- Load the CSV snapshot and inspect column names, row count, and missingness.
- Summarize one numeric variable and one grouping variable with clear `NA` handling.
- Ask an AI assistant to comment on your code, then audit whether the comments match the actual operations.
- Note one place where an AI assistant could hallucinate a variable name or overstate what the descriptive summary means.

Student output: a short Quarto note with one data inspection, one summary, and one AI-audit comment. Assignment link: Assignment 2.

`r1.qmd` Packages: Opening the Toolbox

Install vs Load

- `install.packages("tidyverse")` — **buying** the toolbox. Done once per environment (already done in your course Codespace).
- `library(tidyverse)` — **opening** the toolbox. Required every time you restart R or reopen your Codespace.

```r
library(tidyverse)
```

> **Note**
>
> If you see an error like `there is no package called 'tidyverse'`, the package is not installed in your Codespace. Use the Canvas support channel or office hours and include the full error message.

Minimal renv Checkpoint

`renv` helps record which R packages a project uses. You do not need to master `renv` this week, but you should recognize the three checkpoint commands that support reproducibility.

```r
# Check whether the project environment has changed.
renv::status()

# Record package changes when you intentionally add or update packages.
renv::snapshot()

# Restore the recorded package environment in a fresh setup.
renv::restore()
```

For Assignment 2, run `renv::status()` and write a short dependency note. Only run `renv::snapshot()` if you intentionally added a package.

`r2.qmd` Core Constructs: Boxes, Toasters, and the Assembly Line

Variables: The Labeled Box

R has no memory unless you store results in a named box using `<-`.

```r
patient_age <- 55
clinic_name <- "Vancouver General"
```

> **Warning**
>
> **Case sensitivity:** `Age`, `age`, and `AGE` are three different boxes. This is one of the most common bugs in AI-generated code. Always check that column names match your data exactly.

Functions: The Toaster

A function takes **input** and returns **output**. Arguments (inside the parentheses) control its behaviour.

```r
mean(c(72, 85, 90), na.rm = TRUE)
```

- `c(72, 85, 90)` is the input (a vector of numbers).
- `na.rm = TRUE` tells R to ignore missing values. **Always look for this** when auditing AI code that works with health data — missingness is the norm, not the exception.

The Pipe: The Assembly Line

The pipe passes the result of one step into the next. Think of it as **"and then..."**

```
data %>%
  filter(age > 65) %>%
  summarise(mean_bmi = mean(bmi, na.rm = TRUE))
```

Read this as: *"Take the data, and then keep only patients over 65, and then calculate the average BMI."*

> Tip
>
> You will see two pipe symbols in the wild: `%>%` (from the `magrittr` / `tidyverse` packages) and `|>` (built into R 4.1+). They do the same thing for our purposes. If AI generates the other one, don't panic — both work in your Codespace.

`r3.qmd` Data Import and First Contact

Loading Data

Use relative paths so your code runs on any machine (including your grader's Codespace).

```
my_data <- read_csv("data/patients.csv")
```

How to get the path right: Right-click the file in the VS Code Explorer → **Copy Relative Path** → paste it into your code.

AI Prompt:

> Read the CSV file at "data/patients.csv" and save it as `my_data`. Use the `readr` package.

First Contact: Inspect Before You Analyze

Never start analysis without looking at your data. Run these four checks:

```
# How big is it?
dim(my_data)          # rows x columns

# What are the column names and types?
glimpse(my_data)

# Quick summary statistics
summary(my_data)

# How much is missing?
colSums(is.na(my_data))
```

Open the **Environment** pane (top-right in RStudio or via the R extension) to see your variables and click a data frame to view it in a grid.

> ⚠ Warning
>
> **Health data reality:** Real datasets like NHANES have substantial missingness. Knowing *where* and *how much* is missing is not optional — it shapes every downstream decision.

`r4.qmd` Data Transformation: The Five `dplyr` Verbs

These five verbs are the core of data wrangling in R. You will use them in nearly every assignment and in your term project.

`filter()` — Keep rows that match a condition

```r
# Patients aged 65 and older
seniors <- my_data %>% filter(age >= 65)
```

`select()` — Keep (or drop) columns

```r
# Keep only ID, age, and BMI
slim <- my_data %>% select(id, age, bmi)
```

`mutate()` — Create or modify a column

```r
# Add a BMI category column
my_data <- my_data %>%
  mutate(bmi_category = case_when(
    bmi < 18.5 ~ "Underweight",
    bmi < 25   ~ "Normal",
    bmi < 30   ~ "Overweight",
    TRUE       ~ "Obese"
  ))
```

`summarise()` — Collapse rows into a summary

```r
my_data %>% summarise(mean_age = mean(age, na.rm = TRUE))
```

`group_by()` + `summarise()` — Summaries by group

```r
my_data %>%
  group_by(sex) %>%
  summarise(
    n = n(),
    mean_bmi = mean(bmi, na.rm = TRUE)
  )
```

AI Prompt:

> Using `dplyr`, filter my dataset to patients over 65, group by sex, and calculate the mean BMI for each group. Handle missing values.

Tip

Audit the AI output: After running the AI-generated code, check that (a) the column names match your actual data, (b) `na.rm = TRUE` is present where needed, and (c) the row counts make sense (e.g., the filtered data should be smaller than the original).

Git checkpoint: You have now loaded, inspected, and transformed data. Stage your `.qmd`, write a commit message describing what you did, and commit. Don't wait until the end.

`r5.qmd` Control Flow: Auditing AI Decisions

You will not write many `if/else` blocks or loops by hand. But AI frequently generates them, so you need to **read** them.

If/Else: The Fork in the Road

```
if (mean_age > 50) {
  print("Older cohort")
} else {
  print("Younger cohort")
}
```

Audit question: Is the threshold (50) appropriate for my research question, or did the AI pick an arbitrary number?

Loops: The Treadmill

```r
for (col in c("age", "bmi", "sbp")) {
  print(summary(my_data[[col]]))
}
```

Audit questions: Does the loop have a clear end condition? Could this be replaced with a simpler vectorized operation (e.g., `summarise(across(...))`)?

> **Note**
>
> In R, loops are rarely the best tool. If AI gives you a loop, ask: *"Can this be done without a loop using dplyr or purrr?"* The answer is usually yes.

`r6.qmd` Writing a Simple Function

The syllabus asks you to write at least one custom function. A function packages a repeated task so you don't copy-paste code.

Example: A Reusable Summary

```r
summarise_column <- function(data, column_name) {
  data %>%
    summarise(
      n_total   = n(),
      n_missing = sum(is.na(.data[[column_name]])),
      mean_val  = mean(.data[[column_name]], na.rm = TRUE),
      sd_val    = sd(.data[[column_name]], na.rm = TRUE)
    )
}

```

```r
# Use it
summarise_column(my_data, "bmi")
summarise_column(my_data, "age")
```

Your Task

1. Ask your AI co-pilot to write a function that takes a data frame and a column name and returns summary statistics.
2. **Audit the code:** Does it handle `NA`? Does it work on a numeric column? What happens if you pass a categorical column?
3. **Adapt it:** Change one thing (e.g., add a median, or add a `group_by` argument). This is the difference between accepting code blindly and understanding it.

AI Prompt:

> Write an R function called `summarise_column` that takes a data frame and a column name (as a string) and returns the count, number of missing values, mean, and standard deviation. Then show me how to call it on the "bmi" column of `my_data`.

`r7.qmd` Debugging: Reading the Red Text

Errors are normal. AI makes mistakes, you make typos, and data surprises you.

The "Fix It" Loop

1. **Read** the error message. Look for keywords: `not found`, `unexpected`, `non-numeric`.
2. **Copy** the full error text.
3. **Paste** it into your AI chat and ask:

> Here is an R error from my Codespace: [paste]. Explain what went wrong in one sentence and give corrected code.

Common Errors and What They Mean

Error message	Likely cause
`object 'X' not found`	Typo in variable name, or you forgot to run an earlier chunk
`could not find function "X"`	Forgot `library(tidyverse)` or misspelled the function
`non-numeric argument to binary operator`	Trying to do math on a text/factor column
`unexpected symbol`	Missing comma, parenthesis, or pipe

> ⚠ Warning
>
> **AI debugging guardrail:** When AI suggests a fix, re-read the corrected code before running it. Does the fix address the *actual* error, or did AI rewrite the whole chunk and change your logic?

`r8.qmd` AI Safety: The Audit Checklist

Every time you accept AI-generated code, run through this checklist:

1. **Column names:** Are they spelled exactly as they appear in `glimpse(my_data)`?
2. **Data types:** Is the code doing math only on numeric columns?
3. **Missingness:** Is `na.rm = TRUE` present where needed?

4. **Privacy:** Does the code avoid printing individual patient rows that could be identifiable?
5. **Logic:** Does the code do what *you* intended, or what the AI *assumed* you meant?

The Paper Trail Rule

Every time you use AI to write a code chunk, add a `#` comment above it explaining what you asked for and what you verified:

```
# AI prompt: "Calculate mean BMI by sex, handling NAs"
# Verified: column names match, na.rm = TRUE present, output has 2 rows (M/F)
my_data %>%
  group_by(sex) %>%
  summarise(mean_bmi = mean(bmi, na.rm = TRUE))
```

This comment trail is your audit log. It demonstrates accountability (required by the course AI policy) and helps you debug later.

`r9.qmd` Exporting Clean Data

After cleaning and transforming, save your work so future weeks can pick up where you left off.

```
# Save a clean version for Week 4 and beyond
write_csv(my_data, "output/clean_data.csv")
```

> Tip
>
> **Reproducibility habit:** Your Quarto document should contain *every step* from raw data to clean export. Anyone (including your grader) should be able to render it from scratch and get the same `clean_data.csv`.

Git checkpoint: Your analysis is complete. Stage, commit with a clear message (e.g., `"Complete Week 3: load, inspect, transform, and export patient data"`), and Sync.

`r10.qmd` Literate Analysis in Quarto

Your `.qmd` file is not just code — it is a **document**. Weave narrative around your code chunks to explain what you are doing and why.

Pattern

````
## Data Inspection

We begin by examining the structure of the dataset to understand
variable types and missingness before any transformations.

```{r, eval=FALSE}
glimpse(my_data)
colSums(is.na(my_data))
```


The dataset contains `nrow(my_data)` observations across
`ncol(my_data)` variables. Notably, BMI is missing for
approximately 12% of records, which we handle with `na.rm = TRUE`
in downstream summaries.

This interleaving of code and prose is **literate programming** --- the methodology that underp

---

## `r11.qmd` Knowledge Check

````

```
1. What is the difference between `install.packages()` and `library()`?
2. Why is `na.rm = TRUE` important when working with health data?
3. What does `my_data %>% filter(age > 65) %>% summarise(n = n())` do? Read it aloud using "and
4. Name the five core `dplyr` verbs and what each does in one sentence.
5. You see the error `object 'BMI' not found` but your column is called `bmi`. What went wrong
6. What is the difference between `%>%` and `|>`? Does it matter for this course?
7. Why should you write `#` comments above AI-generated code?
8. What four checks should you run immediately after loading a new dataset?

---

## `r12.qmd` Assignment: The "Mad Libs" Report

**Task:** Build a short Quarto report that loads, inspects, transforms, and summarizes a healt

### Step-by-Step Workflow

1. Open `week3_assignment.qmd` in your Codespace.
2. **Load packages and data:** Use AI to write the `library()` and `read_csv()` calls. Verify
3. **Inspect missingness:** Run `colSums(is.na(...))`. Add a Markdown paragraph interpreting wh
4. **Transform:** Use `dplyr` verbs to:
   - Filter to a meaningful subgroup (e.g., age   18).
   - Create at least one new column with `mutate()`.
   - Produce a grouped summary with `group_by()` + `summarise()`.
5. **Write a function:** Create (or adapt from AI) a reusable summary function. Call it on at l
6. **Export:** Save your cleaned data to `output/clean_data.csv`.
7. **Document:** Every code chunk should have:
   - A `#` comment stating what you asked the AI and what you verified.
   - A Markdown paragraph below interpreting the result.
8. **Render** the document to verify it runs cleanly from top to bottom.
9. **Git:** Stage, write a descriptive commit message, Commit, and Sync.

### Deliverable

A rendered `.qmd` that a reader can follow as a narrative: what data you started with, what you

---

## `r13.qmd` Quick Reference

```

````
| Concept | R code | Notes |
|---|---|---|
| Load packages | `library(tidyverse)` | Every session |
| Read CSV | `read_csv("data/file.csv")` | Use relative paths |
| Assignment | `x <- 5` | The `<-` arrow stores a value |
| Inspect structure | `glimpse(df)` | Column names, types, preview |
| Dimensions | `dim(df)` | Rows × columns |
| Summary stats | `summary(df)` | Min, median, mean, max, NAs |
| Check missingness | `colSums(is.na(df))` | Count NAs per column |
| Filter rows | `filter(df, age > 65)` | Keep matching rows |
| Select columns | `select(df, id, age, bmi)` | Keep named columns |
| New column | `mutate(df, bmi_cat = ...)` | Create or modify |
| Summarize | `summarise(df, m = mean(x, na.rm = TRUE))` | Collapse to one row |
| Group + summarize | `group_by(df, sex) %>% summarise(...)` | Per-group summaries |
| Export | `write_csv(df, "output/clean.csv")` | Save for later |
| Pipe | `%>%` or `|>` | "And then..." |


`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6IndlZWtzXFx3ZWVrMDMtci13aXRoLWFpIn0= -->`{=html

```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6IndlZWtzXFx3ZWVrMDMtci13aXRoLWFpIiwiYm9va0l0ZW1U
```

# The Setup (The Cockpit)


Before we start flying, we need to check our instruments in **VS Code**. If your environment is

## Verify Your AI Extensions
1. Click the **Extensions** icon on the far-left sidebar (it looks like four squares).
2. In the search bar, ensure both **GitHub Copilot Chat** and **R** are installed and active.

![](../../assets/images/week3/r1.png)

## Set the AI Model to "0x"
1. Open the Copilot Chat sidebar (Shortcut: **Ctrl+Alt+I** or **Cmd+Option+I** on Mac).
2. Look for the model selector dropdown at the bottom of the chat window.
3. **Crucial Check:** Choose a model with a **"0x" multiplier** (like GPT-4o 0x). If you do no
   * *Troubleshooting:* If the Chat asks you to "Sign In" or "Subscribe," your Student Pack is
````

```

## Interface Review: Where does R actually live?
To learn R, we are going to start in the "sandbox." Here is how your screen is laid out:

* **The Console (Bottom):** Your **Engine Room**. *Start here!* This is where you can type R co
* **The Environment (Top Right):** Your **Work Bench**. Whenever you save a piece of data (like
* **The Explorer (Left Sidebar):** Your **File Cabinet**. This is where you find your `data/` f
* **The Editor (Top Center):** Your **Laboratory Notebook**. Typing in the Console is great for

## Interface Review: Working with .qmd and .Rmd files
When you open a `.qmd` (Quarto) or `.Rmd` (RMarkdown) file, you must know where to look. Here i

* **The Explorer (Left Sidebar):** Your **File Cabinet**. This is where you find your reports a
* **The Editor (Top Center):** Your **Laboratory Notebook**. This is where you write your text
    * *Note:* R code must be placed inside grey **Code Chunks** (starting with ` ```{r} ` and e
* **The Console (Bottom):** Your **Engine Room**. When you click the green "Play" button in the
    * *Pro Tip:* You can type quick math directly into the console (like `100 / 4`), but it wil
* **The Environment (Top Right):** Your **Work Bench**. Whenever you load data or save a calcul


`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6IndlZWtzXFx3ZWVrMDMtci13aXRoLWFpIn0= -->`{=html

```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6IndlZWtzXFx3ZWVrMDMtci13aXRoLWFpIiwiYm9va0l0ZW1U
```

Packages (The Toolbox)

In R, tools come in "packages." Think of `install.packages` as buying a physical toolbox, and

-   **To open your tools:** Type this in your code and run it:

    ```
    library(tidyverse)
    ```

> **Note:** You must run `library(tidyverse)` every single time you restart your Codespace.


```

`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6Im1pbGVzdG9uZXMifQ== -->`{=html}

```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6Im1pbGVzdG9uZXMiLCJib29rSXRlbVR5cGUiOiJjaGFwdGV
```

Milestones
`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6IndlZWtzXFx3ZWVrMDMtci13aXRoLWFpIn0= -->`{=htm

```````{.quarto-title-block template='C:\Users\wilds\AppData\Local\Apps\Quarto\share\projects\
---
title: Milestones
```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6IndlZWtzXFx3ZWVrMDMtci13aXRoLWFpIiwiYm9va0l0ZW1U
```

---

# Basics: Boxes & Toasters

R is a system of storage (Boxes) and transformation (Toasters).

## Variables (The Box)

Use `<-` to save data into a name.

Code snippet

```
Save the number 55 into a variable called patient_age patient_age
```

## Functions (The Toaster)

Functions take input, perform an action, and return output. Look at the snippet below

Code snippet

```
calculate_rectangle_area <- function(length, width) {
  area <- length * width
  return(area)
}

# Example usage:
calculate_rectangle_area(10, 5) # Returns 50
```

`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6IndlZWtzXFx3ZWVrMDMtci13aXRoLWFpIn0= -->`{=htm

```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6IndlZWtzXFx3ZWVrMDMtci13aXRoLWFpIiwiYm9va0l0ZW1U
```

146

# Control Flow

## The Pipe %\>% (The Assembly Line)

The symbol `%>%` means **"and then..."**.
```

# Milestones

Milestones move the term project from team setup to final portfolio. They are short checkpoints, not separate side projects. Each one should make the next stage easier to complete and easier for another person to review.

## Submission Contract

For every milestone:

- commit and sync the required files before submitting
- submit the GitHub repository link on Canvas
- include the latest commit hash in the Canvas comment box
- use relative paths
- document any package or data access requirements
- disclose and audit AI use

## Milestone Map

| Milestone | Timing | Focus | Link |
|---|---|---|---|
| M0 | Week 4 | Group formation and repository access | M0: Group Formation |
| M1 | Week 6 | Project proposal and data intake | M1: Project Proposal |

| Milestone | Timing | Focus | Link |
|---|---|---|---|
| M2 | Week 7 | Preliminary analysis and Table 1-style summary | M2: Preliminary Analysis |
| M3 | Week 10 | Draft report and dashboard-style update | M3: Project Update |
| M4 | Week 11 | Peer review and reproducibility feedback | M4: Peer Review |
| M5 | Weeks 12-13 | Final presentation | M5: Final Presentation |
| Final Portfolio | Final deadline | Complete project record | Final Portfolio |

## Reproducibility Expectations

Milestone work should be reviewable in GitHub Codespaces or a documented local setup. Keep data access clear, use relative paths, and avoid manual edits to generated outputs unless the edit is explicitly documented.

## AI-Use Expectations

AI tools may help with drafting, code review, debugging, and clarity checks. Students remain responsible for correctness. Each milestone includes an AI-use note so reviewers can see what was AI-assisted and what was verified by the team.

## Definition Of Done

A milestone is ready when the required files are present, the repository is synced, the Canvas submission points to the correct commit, and another student could understand what changed without private explanation.

# Milestone Template

Use this template for project milestones M0-M5 and the Final Portfolio.

## Deliverable

- **Milestone:**
- **Due:**
- **Submit:**
- **Main artifact:**

## Required Evidence

- Repository link.
- Rendered report, dashboard, slides, or review document.
- Commit history showing meaningful progress.
- AI-use note.
- Reproducibility check result.

## Definition of Done

- The artifact opens or renders in the supported workflow.
- Data provenance and stewardship decisions are documented.
- The work avoids unsupported causal or predictive claims.
- A peer or TA can rerun the project from a fresh environment.

# M0: Group Formation

## Purpose

M0 makes sure every project team has a working collaboration setup before analytic work begins. By the end of this milestone, each group should know who is on the team, where the shared repository lives, how communication will happen, and how the team will handle reproducibility and AI use.

## Due Timing

Submit by the Week 4 deadline posted in Canvas. This milestone is complete/incomplete and is meant to unblock the project work that follows.

## Required Deliverables

Create a `submission/` folder in this milestone directory:

```
milestones/m0-group-formation/submission/
```

Add these files:

- `group-formation.md`
- `repo-access-evidence.md`
- `communication-plan.md`
- `codespace-check.md`
- `ai-use-note.md`

## What To Submit

Submit your GitHub repository link on Canvas after committing and syncing the files above. In the Canvas comment box, include the latest commit hash for the submission.

## File Guidance

`group-formation.md` should include:

- group name
- student names
- GitHub usernames
- one-sentence project area or dataset interest
- primary repository owner

`repo-access-evidence.md` should include:

- shared repository link
- confirmation that all group members can view the repo
- confirmation that all group members can open the repo in Codespaces or a local equivalent

`communication-plan.md` should include:

- primary communication channel
- expected response window
- weekly check-in plan
- how the group will document decisions

`codespace-check.md` should include:

- who tested the repository
- what command or render was tested
- whether the test passed
- any setup issue and the planned fix

`ai-use-note.md` should include:

- whether AI tools were used to draft the group plan
- what the group verified independently
- what the group will not outsource to AI

## Reproducibility Expectations

Use relative paths and keep project files inside the shared repository. If a group member needs a package, script, or external file to reproduce work, document it in the repository README before the next milestone.

## AI-Use Expectations

AI tools may help draft communication norms or check clarity. The team remains responsible for verifying repository access, Codespaces setup, project scope, and all submitted claims.

## Grading Checklist

This milestone is marked complete when:

- all group members are listed with GitHub usernames
- shared repository access is documented
- communication norms are specific enough to use
- Codespaces or local setup has been tested
- AI-use expectations are acknowledged
- files are committed, synced, and linked through Canvas

## Definition Of Done

M0 is done when every group member can reach the shared repository, knows the team communication plan, and can explain how the project will be kept reproducible from this point forward.

# M1: Project Proposal

## Purpose

M1 turns a broad project idea into a scoped, feasible, and ethically grounded analytic plan. The proposal should make the audience, data source, question, reproducibility plan, and first visualization direction clear enough for feedback.

## Due Timing

Submit by the Week 6 deadline posted in Canvas. This milestone prepares the group for Week 7 preliminary analysis and Week 8 dashboard prototyping.

## Required Deliverables

Create or update:

```
milestones/m1-proposal/submission/
```

Include these files:

- `proposal.md`
- `data-intake-card.md`
- `initial-visual-plan.md`
- `reproducibility-plan.md`
- `ai-use-note.md`

## What To Submit

Submit your GitHub repository link on Canvas after committing and syncing the files above. In the Canvas comment box, include the latest commit hash and the folder path for the milestone.

## Proposal Guidance

`proposal.md` should include:

- project title
- group members
- audience and KT purpose
- one primary question
- one secondary question
- expected data source
- planned analytic output
- known limitations

`data-intake-card.md` should summarize:

- data source and steward
- access method
- file format
- unit of analysis
- key variables
- privacy and stewardship considerations
- Indigenous Data Sovereignty or CARE/OCAP awareness when relevant

`initial-visual-plan.md` should include:

- one planned figure
- what the figure should help the audience decide or understand
- one risk of misinterpretation

`reproducibility-plan.md` should include:

- expected repository folders

- data access plan
- package/dependency notes
- how another student could rerun the work

## Reproducibility Expectations

Use relative paths. Do not commit restricted, private, or identifiable data. If public data are too large for GitHub, commit a small sample or documented access script and explain how the full file is obtained.

## AI-Use Expectations

AI may help brainstorm wording, search terms, or code structure. Students must verify data-source claims, privacy considerations, citations, and feasibility. Include an `ai-use-note.md` describing what AI helped with and what was checked manually.

## Grading Checklist

This milestone is marked complete when:

- the question is specific and feasible
- the audience and KT purpose are named
- data provenance and access are documented
- privacy, stewardship, and equity risks are acknowledged
- at least one visualization direction is described
- reproducibility plan is concrete
- AI use is disclosed and audited

## Definition Of Done

M1 is done when another group can read the proposal and understand what you will analyze, why it matters, what data you will use, and how the work will be rerun.

# M2: Preliminary Analysis

## Purpose

M2 checks that the project can move from proposal to evidence. The milestone should show a documented cohort or analytic sample, a missingness check, a descriptive summary, and a careful interpretation that avoids overclaiming.

## Due Timing

Submit by the Week 7 deadline posted in Canvas. This milestone prepares the group for Week 8 dashboard prototyping and Week 9 communication work.

## Required Deliverables

Create or update:

```
milestones/m2-preliminary-analysis/submission/
```

Include these files:

- `preliminary-analysis.qmd`
- `preliminary-analysis.html`
- `table1.csv`
- `missingness-summary.csv`
- `methods-note.md`
- `ai-audit-note.md`

## What To Submit

Submit your GitHub repository link on Canvas after committing and syncing the files above. In the Canvas comment box, include the latest commit hash and the folder path for the milestone.

## Analysis Requirements

Your preliminary analysis should include:

- a clear analytic sample or cohort definition
- a short explanation of inclusion and exclusion decisions
- a missingness check for key variables
- a Table 1-style descriptive summary
- at least one figure or table generated by code
- a methods note that states what the summary can and cannot claim

## Reproducibility Expectations

The `.qmd` file must render in GitHub Codespaces or a documented local environment. Use relative paths, document packages, and avoid manual edits to generated tables or figures.

## AI-Use Expectations

AI may help draft code, suggest summaries, or review language. Students must check row counts, filters, missingness handling, labels, and interpretation. The `ai-audit-note.md` must identify at least three checks performed by the group.

## Grading Checklist

This milestone is marked complete when:

- cohort definition is explicit
- missingness is reported
- Table 1-style output includes counts and descriptive statistics
- code-generated output is reproducible
- interpretation is descriptive and non-causal unless justified
- AI-generated help is disclosed and audited
- files are committed, synced, and submitted through Canvas

## Definition Of Done

M2 is done when a peer can rerun the analysis, see the same descriptive outputs, and understand the limits of the claims being made.

# M3: Project Update

## Purpose

M3 turns the preliminary analysis into a project draft that is ready for feedback. The update should show the current evidence, the intended KT product, and the main decisions that still need peer or instructor critique.

## Due Timing

Submit by the Week 10 deadline posted in Canvas. This milestone supports report drafting, publishing checks, and the Week 11 reproducibility review.

## Required Deliverables

Create or update:

```
milestones/m3-project-update/submission/
```

Include these files:

- `project-update.md`
- `report-draft.qmd`
- `report-draft.html`
- `dashboard-preview.md`
- `reproducibility-note.md`
- `feedback-request.md`
- `ai-use-note.md`

## What To Submit

Submit your GitHub repository link on Canvas after committing and syncing the files above. In the Canvas comment box, include the latest commit hash, the folder path, and the rendered report link if one is available.

## Update Requirements

`project-update.md` should include:

- current project title and audience
- refined research or KT question
- data source and analytic sample
- one finding that appears stable
- one finding or choice that needs review
- next steps before peer review

`dashboard-preview.md` should include:

- link or screenshot path for the dashboard-style product
- intended audience
- one interaction, filter, or decision-support feature
- one limitation or privacy caution

`feedback-request.md` should ask for targeted feedback on:

- correctness
- interpretation
- visual clarity
- reproducibility
- audience fit

## Reproducibility Expectations

The report draft must render in GitHub Codespaces or a documented local environment. Relative paths are required. Generated figures and tables should come from code, not manual copying.

## AI-Use Expectations

AI may help revise prose, identify unclear sections, or suggest checks. Students must verify citations, numeric results, interpretations, and any code suggested by AI. The `ai-use-note.md` should name the main AI-assisted tasks and the verification steps.

## Grading Checklist

This milestone is marked complete when:

- report draft renders
- current findings are supported by code-generated evidence
- dashboard-style product is visible or clearly previewed
- feedback request is specific
- reproducibility note explains how to rerun the work
- AI use is disclosed and audited

## Definition Of Done

M3 is done when another student can open the repo, render the draft report, review the dashboard preview, and provide useful feedback without guessing what the project is trying to do.

# M4: Peer Review

## Purpose

M4 gives each team structured feedback on correctness, reproducibility, interpretation, and audience fit before final submission. The review should be specific, respectful, and useful enough that the receiving group can act on it.

## Due Timing

Submit by the Week 11 deadline posted in Canvas. Peer review supports the final presentation and final portfolio revision.

## Required Deliverables

Create or update:

```
milestones/m4-peer-review/submission/
```

Include these files:

- `peer-review.md`
- `reproducibility-check.md`
- `auditability-check.md`
- `kt-feedback.md`
- `reviewer-reflection.md`
- `ai-use-note.md`

## What To Submit

Submit your GitHub repository link on Canvas after committing and syncing the files above. In the Canvas comment box, include the reviewed group name and the latest commit hash for your submission.

## Review Requirements

`peer-review.md` should include:

- reviewed project title
- two strengths
- three prioritized revision suggestions
- one question for the group
- one issue that would block final reproducibility if unfixed

`reproducibility-check.md` should document:

- whether the report rendered
- whether paths were relative
- whether data access was clear
- whether figures and tables were generated from code
- any error messages or setup blockers

`auditability-check.md` should document:

- whether data provenance is clear
- whether AI use is disclosed
- whether claims are supported by evidence
- whether limitations are stated

`kt-feedback.md` should focus on:

- audience fit
- plain-language clarity
- visual clarity
- actionability
- privacy-safe communication

## Reproducibility Expectations

Run the reviewed project in GitHub Codespaces when possible. If a full rerun is not possible, document exactly where the process stopped and what evidence you reviewed instead.

## AI-Use Expectations

AI may help organize feedback or identify unclear writing. Reviewers must independently inspect the files, outputs, and claims. Do not use AI to invent results or infer evidence that is not in the reviewed repository.

## Grading Checklist

This milestone is marked complete when:

- feedback is specific and actionable
- reproducibility check is documented
- auditability and KT feedback are included
- at least one blocking risk is named or the absence of blockers is justified
- AI use is disclosed
- files are committed, synced, and submitted through Canvas

## Definition Of Done

M4 is done when the reviewed group can use your comments to improve the final portfolio without needing a follow-up meeting to understand the feedback.

# M5: Final Presentation

## Purpose

M5 is the oral KT version of the final project. The presentation should explain the audience, question, data, one or two main findings, limitations, and the practical meaning of the work without overclaiming.

## Due Timing

Presentation order and date are posted in Canvas. Submit slides before your presentation block. The standard timing is 7 minutes of presentation, 3 minutes of questions, and 1 minute for transition.

## Required Deliverables

Create or update:

```
milestones/m5-presentation/submission/
```

Include these files:

- `slides.qmd`
- `slides.html` or `slides.pdf`
- `source-and-limitations-note.md`
- `presentation-checklist.md`
- `peer-feedback-received.md`
- `ai-use-note.md`

## What To Submit

Submit your GitHub repository link on Canvas after committing and syncing the files above. In the Canvas comment box, include the latest commit hash, the slide path, and any rendered slide link.

## Slide Requirements

Use 5 to 7 slides:

1. Title, team, audience, and KT purpose
2. Question and why it matters
3. Data source, analytic sample, and stewardship note
4. Main figure or table
5. Interpretation and limitation
6. Practical implication or next step
7. Backup or Q&A slide if useful

## Source And Limitations Note

`source-and-limitations-note.md` should include:

- data source and citation
- key inclusion or exclusion choices
- whether results are descriptive, predictive, or causal
- one privacy or stewardship caution
- one limitation that should be stated aloud

## Reproducibility Expectations

Slides should render in GitHub Codespaces or a documented local environment. Figures and tables should come from the project repository whenever possible. Screenshots may be used for dashboards, but the source page or app path must be documented.

## AI-Use Expectations

AI may help improve slide wording, draft speaker notes, or identify jargon. Students must verify all numbers, citations, screenshots, and claims. The `ai-use-note.md` should state what AI helped with and what was checked manually.

## Presentation Rubric

Use this checklist while preparing:

- audience and KT purpose are clear
- question is focused
- data source and limitations are transparent
- main visual is readable from the back of the room
- interpretation is accurate and non-causal unless justified
- privacy and stewardship risks are acknowledged
- timing fits the presentation window
- questions are answered with evidence and humility

## Definition Of Done

M5 is done when the slides are committed, rendered, submitted through Canvas, and ready to present without relying on local-only files.

# Final Portfolio

## Purpose

The final portfolio is the complete project record: report, code, data documentation, dashboard-style KT product, presentation materials, reproducibility evidence, and AI-use audit. It should let another person understand what was done, why it matters, and how to rerun or inspect the work.

## Due Timing

Submit by the final portfolio deadline posted in Canvas. The syllabus deadline is the course final submission deadline unless Canvas states a more specific time.

## Required Deliverables

Create or update:

```
milestones/final-portfolio/submission/
```

Include these files:

- `final-portfolio-checklist.md`
- `repository-access-note.md`
- `final-risk-list.md`
- `ai-use-note.md`

Your project repository should also include:

- final report source and rendered output

- dashboard-style KT product or static preview
- data provenance and stewardship documentation
- reproducibility instructions
- presentation slides or presentation summary
- clear README for the whole project

## What To Submit

Submit your GitHub repository link on Canvas after committing and syncing all final materials. In the Canvas comment box, include:

- latest commit hash
- final report path
- dashboard or KT product path
- final portfolio checklist path
- rendered public link if the repository is public and safe to publish

## Public And Private Repository Guidance

Use a public repository only when the data and outputs are allowed to be public. Do not publish restricted, private, identifiable, or culturally sensitive data. If the repository must stay private, keep it private, grant course staff access, and submit the private repository link on Canvas.

Public repositories should avoid raw restricted data and should use documented public data, simulated data, or aggregate outputs that are safe to share. Private repositories still require clear documentation and reproducibility instructions for course staff. See Repository Sharing Guidance before changing repository visibility.

## Reproducibility Checklist

Use the Final Portfolio Checklist to confirm:

- repository opens from a fresh clone or Codespace
- README explains the project, audience, and file structure
- all paths are relative
- required packages are documented
- report renders from source
- dashboard or KT product can be opened or rerun
- generated figures and tables can be traced to code
- data provenance and access rules are documented
- privacy and stewardship risks are addressed
- AI use is disclosed and audited
- final commit is synced before Canvas submission

## AI-Use Expectations

AI may help polish writing, check code, create revision plans, or test explanations. Students must verify all outputs, citations, code, numeric results, and interpretations. The final `ai-use-note.md` should describe major AI uses across the project and the checks used to protect correctness.

## Grading Checklist

The final portfolio is ready for assessment when:

- report is complete and evidence-based
- dashboard-style KT product matches the intended audience
- data provenance and limitations are transparent
- repository is organized and reproducible
- final outputs are committed and synced
- presentation materials align with the final project
- AI-use audit is complete
- repository access route is clear

## Definition Of Done

The final portfolio is done when course staff can open the submitted repository, locate the final artifacts, rerun or inspect the work, and understand the project without asking for missing files.

# Reference: Data Sources

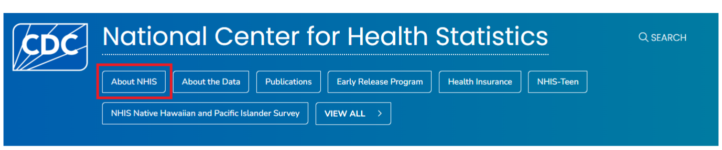
# NHIS

The National Health Interview Survey (NHIS) is a cross-sectional household interview survey conducted by the US Centers for Disease Control (CDC). The survey collects information on the health status, health care access, and health behaviors of the US population.

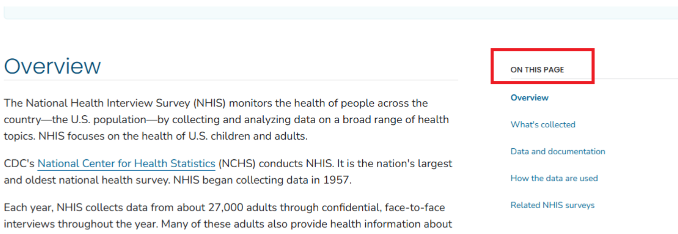

CDC website for NHIS

## About NHIS

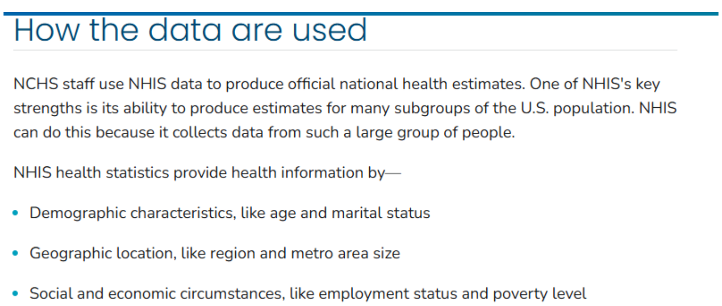
You can find more information about the NHIS survey by clicking on About NHIS:




On the `About NHIS` page, you will find information about what the survey collects, data and documentation, the instructions on how to use the data, and information on related NHIS surveys.

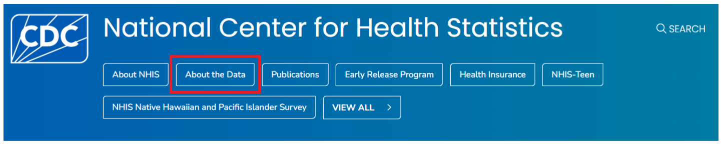



You can click on each of the links or scroll down to find information on each of these topics one by one. For example, if we click on the `How the data are used` link, we will find information as follows:



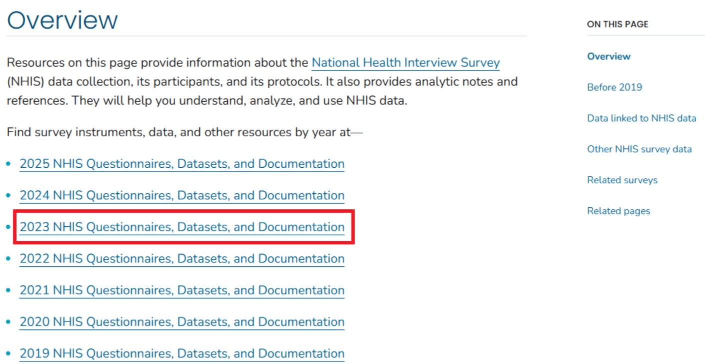

## About the Data

If you click on `About the Data` link, you will find information about the NHIS questionnaires, datasets, and documentation:



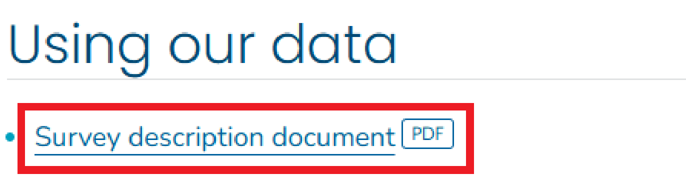

Under the `Overview` tab, you will find information about different NHIS cycles and the questionnaires, datasets, and documentation available in each cycle. As an example, let's click on the `2023 NHIS Questionnaires, Datasets, and Documentation` link to see the details for the 2023 NHIS:

NHIS Data, Questionnaires and Related Documentation

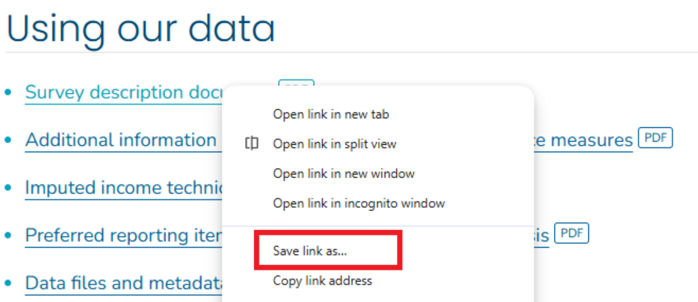



## NHIS Questionnaires, Datasets, and Documentation

To view the description of the 2023 NHIS, click on the `Survey description document` link:




This will open the PDF of the 2023 NHIS survey description. Alternatively, you can right-click on the link, click on `Save link as...` and save the PDF file on your computer for future reference.

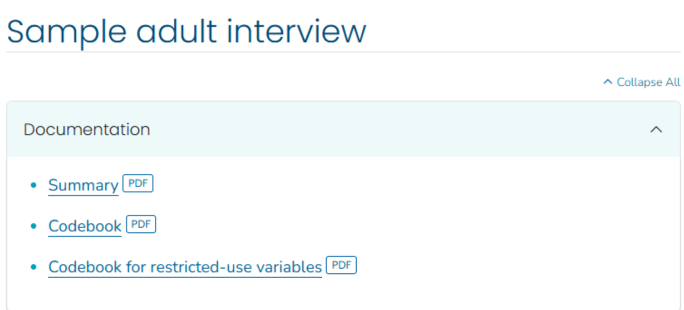



Similarly, you can open and save the questionnaires and related materials under the `Questionnaires and survey materials` tab.

In the recent NHIS, data are available in 5 categories: interview data for adults, interview data for children, data for parent-child pairs, imputed income, and paradata. You can click each link for documentation to see a summary of the data and the codebook.

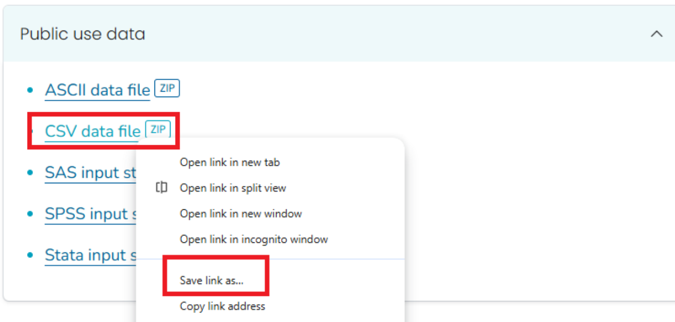
Data are available in 5 categories:

- Sample adult interview
- Sample child interview
- Parent-child pair data
- Imputed income
- Paradata



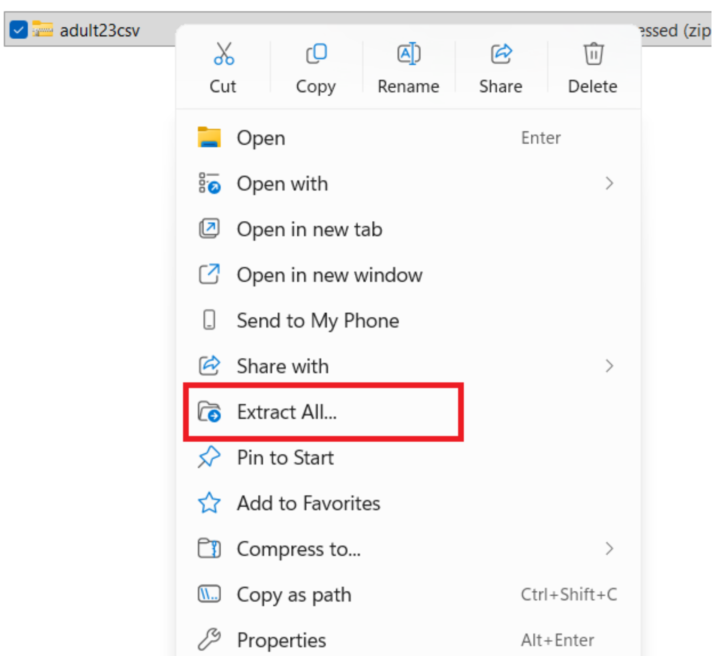

## NHIS Data

The data files are stored on the NHIS website in different formats. You can save the dataset in any statistical package that supports these file formats, e.g., ASCII, CSV, SAS. Let us save the `adult` dataset in the CSV format, which is in a zip file.





The next step is to unzip the file to extract the CSV. Depending on your computer, you can unzip the file with or without software. For example, if you are using Windows, you can right-click the zip file and select `Extract All...` to unzip it.



Once unzipped, you will find two files: the CSV file for the dataset and a readme file with some instructions. Below is the example for the `adult` dataset:

| | | |
|---|---|---|
| adult23 | Microsoft Excel Co... | 28,738 KB |
| readme | Text Document | 3 KB |

To open the datafile in Excel, - you can double click on the CSV file, or - right click on the CSV file and click on `Open`.

Later, you will learn how to open the data file in different statistical software. For Excel, it would take a few seconds/minutes because of the large size of the dataset. Once you open the data file, you will see the data in a tabular format with rows and columns.

| URBRRL | RATCAT_A | INCTCFLG_A | IMPINCFLG_A | LANGSPECR_A | LANGSOC_A | LANGDOC_A | LANGMED_A | LANGHM_A | PPSU | PSTRAT |
|---|---|---|---|---|---|---|---|---|---|---|
| 3 | 4 | 0 | 0 | | | | | | 2 | 103 |
| 4 | 8 | 0 | 0 | | | | | | 2 | 122 |
| 4 | 14 | 0 | 0 | | | | | | 2 | 122 |
| 4 | 10 | 0 | 0 | | | | | | 2 | 122 |
| 4 | 5 | 0 | 0 | | | | | | 2 | 122 |
| 4 | 14 | 0 | 0 | | | | | | 2 | 122 |
| 3 | 12 | 0 | 0 | | | | | | 2 | 103 |

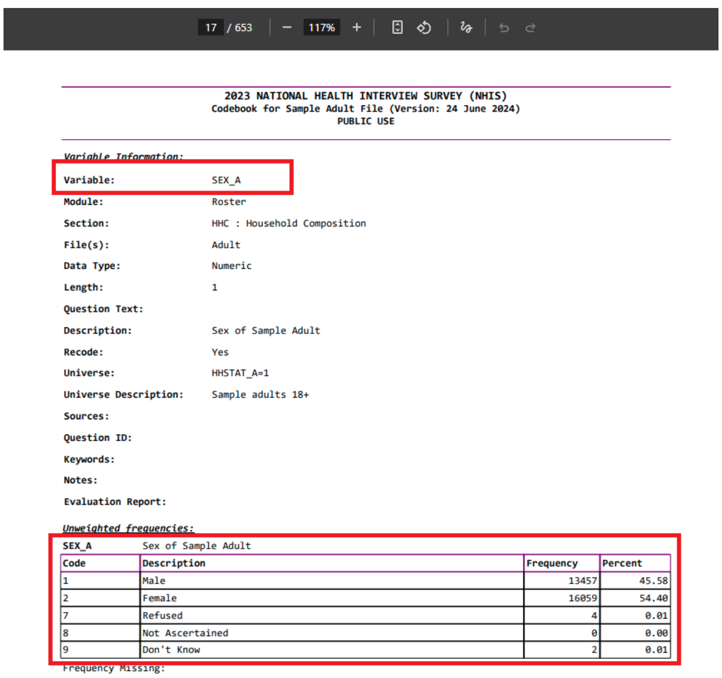
- Each row represents a person.
- Each column represents a variable.
- The first row contains the variable names. The remaining rows contain the data for each person.

## Use of the codebook

To understand the variables in the dataset, you can refer to the codebook. The codebook provides information about the variable names, labels, and values. For example, we want to understand the variable `SEX_A`:

| FQ | FR | FS | FT | FU | FV |
|---|---|---|---|---|---|
| SRVY_YR | SEX_A | AGEP_A | AGE65 | ASTATNEV | MENTHOL |
| 2023 | 1 | 67 | | 1 | |
| 2023 | 1 | 73 | | 1 | |
| 2023 | 1 | 48 | | 1 | |
| 2023 | 2 | 42 | | 1 | |
| 2023 | 2 | 50 | | 1 | |
| 2023 | 2 | 46 | | 1 | |
| 2023 | 2 | 36 | | 1 | |
| 2023 | 1 | 44 | | 1 | |

We can look for SEX_A in the codebook, which will provide us with the variable label and the values for that variable (e.g., 1 for Male, 2 for Female, and so on).



17 / 653 117%

2023 NATIONAL HEALTH INTERVIEW SURVEY (NHIS)
Codebook for Sample Adult File (Version: 24 June 2024)
PUBLIC USE

*Variable Information:*

| | |
|---|---|
| **Variable:** | SEX_A |
| **Module:** | Roster |
| **Section:** | HHC : Household Composition |
| **File(s):** | Adult |
| **Data Type:** | Numeric |
| **Length:** | 1 |
| **Question Text:** | |
| **Description:** | Sex of Sample Adult |
| **Recode:** | Yes |
| **Universe:** | HHSTAT_A=1 |
| **Universe Description:** | Sample adults 18+ |
| **Sources:** | |
| **Question ID:** | |
| **Keywords:** | |
| **Notes:** | |
| **Evaluation Report:** | |

*Unweighted frequencies:*

**SEX_A** Sex of Sample Adult

| Code | Description | Frequency | Percent |
|---|---|---|---|
| 1 | Male | 13457 | 45.58 |
| 2 | Female | 16059 | 54.40 |
| 7 | Refused | 4 | 0.01 |
| 8 | Not Ascertained | 0 | 0.00 |
| 9 | Don't Know | 2 | 0.01 |

Frequency Missing:

There are so many variables in the dataset, and it is not possible to understand all of them at once. You can refer to the codebook to understand the variables that you are interested in for your analysis. For example, the WTFA_A variable is the sample adult weight variable used to calculate weighted estimates for the US adult population. Similarly, the HYPEV_A variable is the hypertension ever variable, indicating whether a doctor or other health professional has ever told the person they have hypertension. Again, you can refer to the codebook to understand the variables that you are interested in.

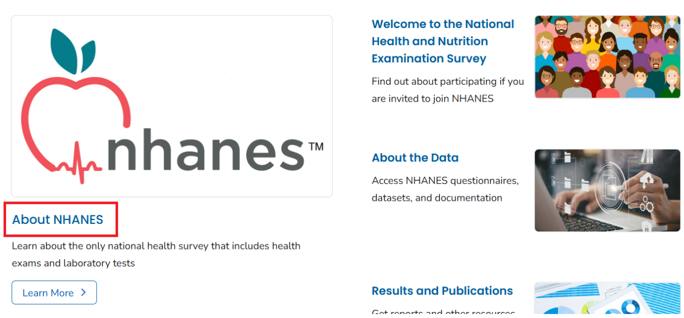
# NHANES

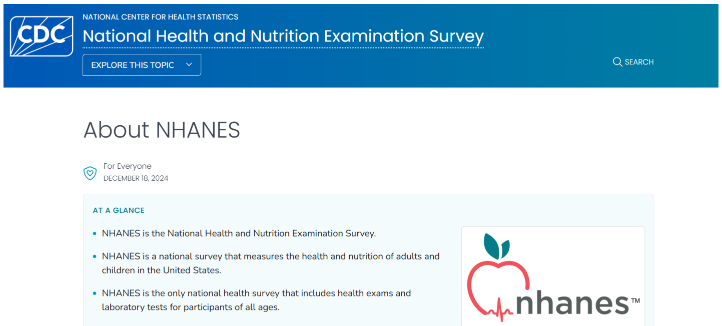
The National Health and Nutrition Examination Survey (NHANES) is a cross-sectional survey conducted by the US Centers for Disease Control (CDC). The survey collects information on the health and nutritional status of the US population. The survey includes interviews, physical examinations, and laboratory tests.


CDC website for NHANES

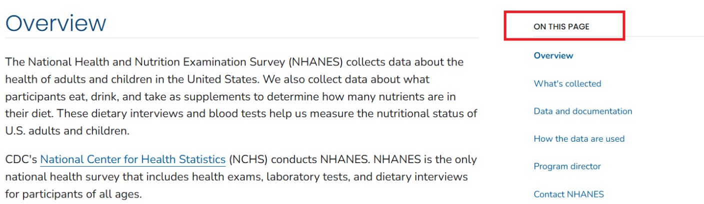
## About NHANES

You can find more information about the NHANES survey by clicking on About NHANES:

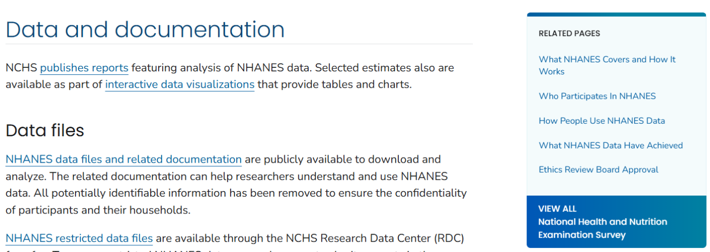





On the `About NHANES` page, you will find information about what the survey collects, data and documentation, the instructions on how to use the data, and information on related NHANES surveys.

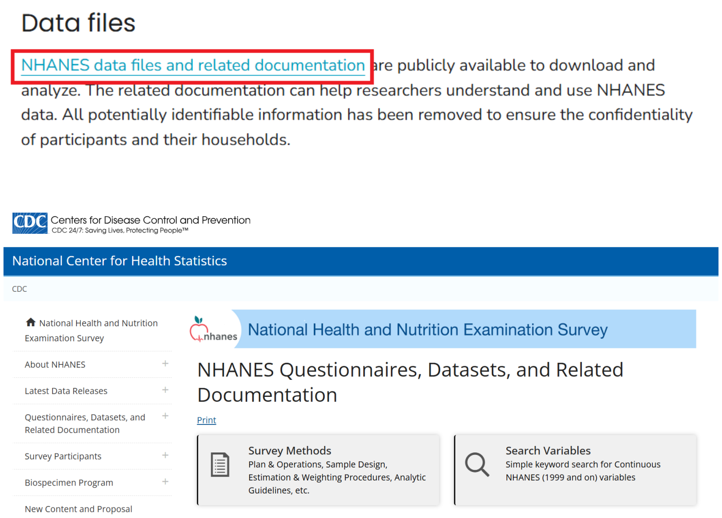



You can click on each of the links or scroll down to find information on each of these topics one by one. For example, if we click on the `Data and documentation` link, we will find information as follows:



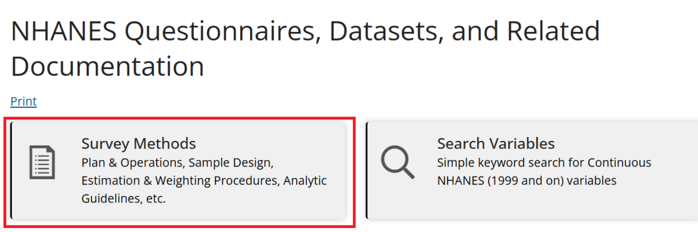

## Survey Methods and Analytic Guidelines

If you click on `NHANES data files and related documentation` link, you will find information about the NHANES questionnaires, datasets, and documentation:



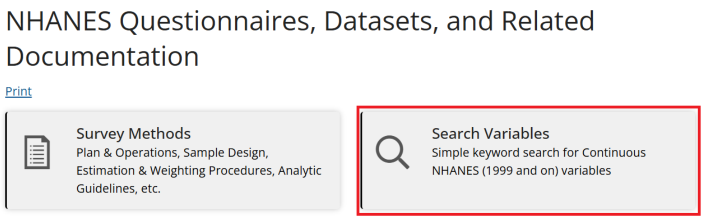

You can click `Survey Methods` to find information on survey methods and analytic guidelines. The `Survey Methods and Analytic Guidelines` page provides information about the plan and operation of the NHANES survey, the sampling design, methods for calculating weights and variance estimation, and guidelines for analyzing NHANES data, along with links to related resources.

NHANES data files and related documentation




## Search Variables

If you click on `Search Variables` link, you will find a search tool to search for variables in the NHANES datasets (1999 and on).



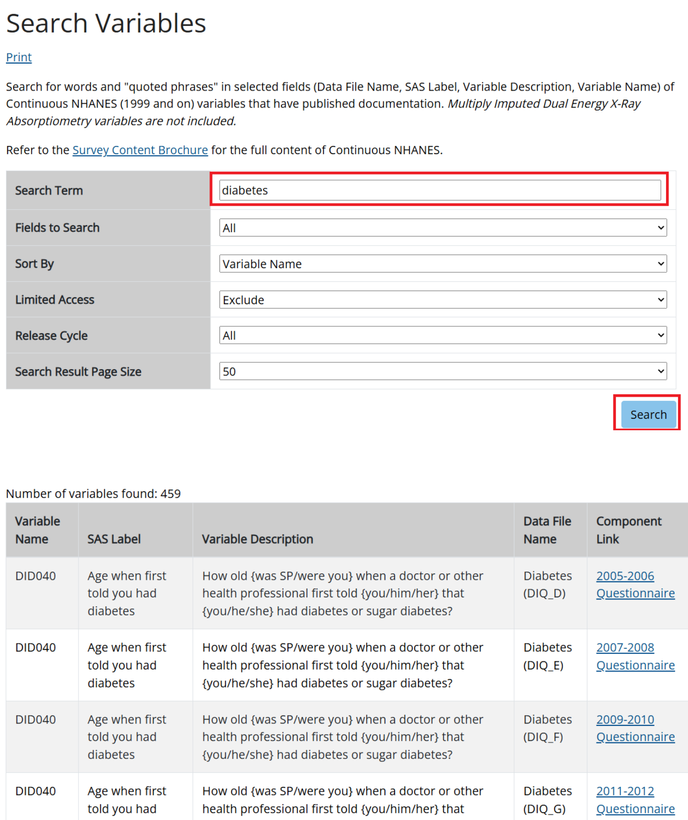
## NHANES Questionnaires, Datasets, and Related Documentation

Print

Survey Methods
Plan & Operations, Sample Design, Estimation & Weighting Procedures, Analytic Guidelines, etc.

Search Variables
Simple keyword search for Continuous NHANES (1999 and on) variables

You can search for variables by entering keywords in the search box. For example, if you enter `diabetes` in the search box, you will find a list of variables related to diabetes in the NHANES datasets.



## Search Variables

Print

Search for words and "quoted phrases" in selected fields (Data File Name, SAS Label, Variable Description, Variable Name) of Continuous NHANES (1999 and on) variables that have published documentation. *Multiply Imputed Dual Energy X-Ray Absorptiometry variables are not included.*

Refer to the Survey Content Brochure for the full content of Continuous NHANES.

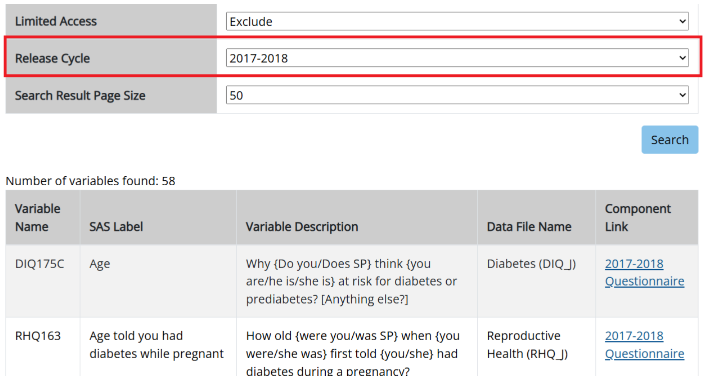
| | |
|---|---|
| Search Term | diabetes |
| Fields to Search | All |
| Sort By | Variable Name |
| Limited Access | Exclude |
| Release Cycle | All |
| Search Result Page Size | 50 |

Search

Number of variables found: 459

| Variable Name | SAS Label | Variable Description | Data File Name | Component Link |
|---|---|---|---|---|
| DID040 | Age when first told you had diabetes | How old {was SP/were you} when a doctor or other health professional first told {you/him/her} that {you/he/she} had diabetes or sugar diabetes? | Diabetes (DIQ_D) | 2005-2006 Questionnaire |
| DID040 | Age when first told you had diabetes | How old {was SP/were you} when a doctor or other health professional first told {you/him/her} that {you/he/she} had diabetes or sugar diabetes? | Diabetes (DIQ_E) | 2007-2008 Questionnaire |
| DID040 | Age when first told you had diabetes | How old {was SP/were you} when a doctor or other health professional first told {you/him/her} that {you/he/she} had diabetes or sugar diabetes? | Diabetes (DIQ_F) | 2009-2010 Questionnaire |
| DID040 | Age when first told you had | How old {was SP/were you} when a doctor or other health professional first told {you/him/her} that | Diabetes (DIQ_G) | 2011-2012 Questionnaire |

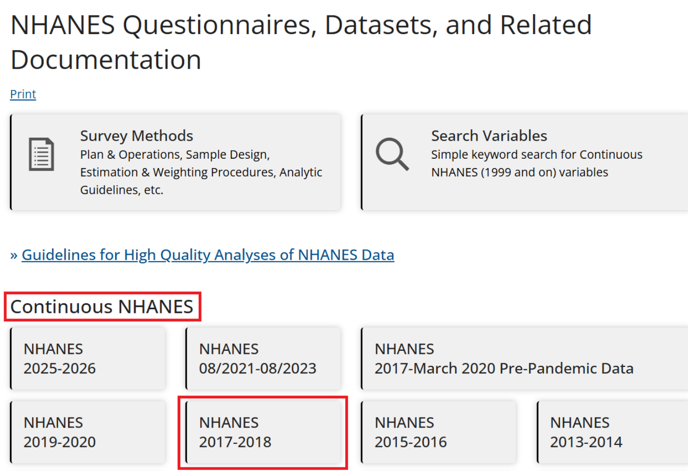
You can restrict your search to a specific NHANES cycle by selecting the cycle from the drop-down menu. For example, if you select the `2017-2018` cycle, you will find a list of variables related to diabetes in the 2017-2018 NHANES dataset.



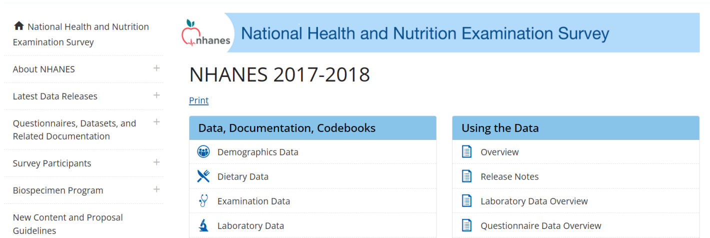

| Variable Name | SAS Label | Variable Description | Data File Name | Component Link |
|---|---|---|---|---|
| DIQ175C | Age | Why {Do you/Does SP} think {you are/he is/she is} at risk for diabetes or prediabetes? [Anything else?] | Diabetes (DIQ_J) | 2017-2018 Questionnaire |
| RHQ163 | Age told you had diabetes while pregnant | How old {were you/was SP} when {you were/she was} first told {you/she} had diabetes during a pregnancy? | Reproductive Health (RHQ_J) | 2017-2018 Questionnaire |

## NHANES Questionnaires, Datasets, and Documentation

Let's explore the questionnaires, datasets, and documentation for the 2017-2018 NHANES cycle. You need to click on `NHANES 2017-2018` under the `Continuous NHANES` tab:



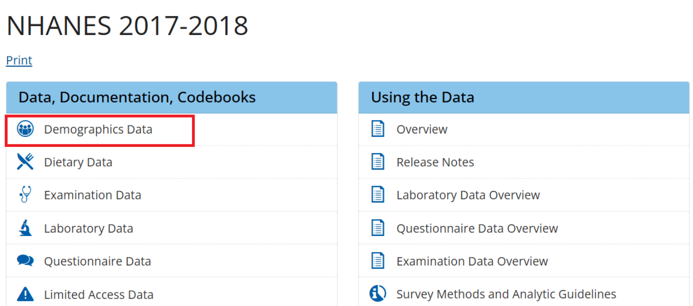

This will open the page for the 2017-2018 NHANES cycle, where you can find information on the survey description, questionnaires, datasets, and documentation.

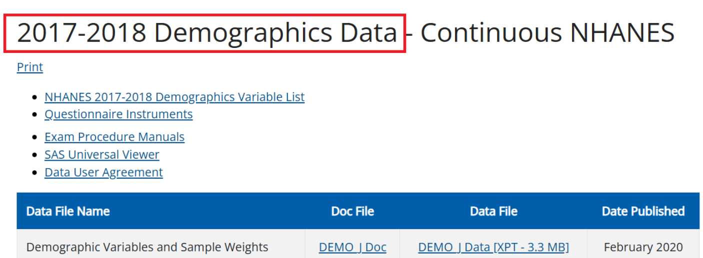



There are six categories of data available for the 2017-2018 NHANES cycle: Demographics, Dietary, Examination, Laboratory, Questionnaire, and Limited Access data. You can click each category to find the datasets and documentation available in that category. For example, if you click on `Demographics Data`, you will find a list of datasets and documentation related to demographics data in the 2017-2018 NHANES cycle.

Data are available in 6 categories:

- Demographics
- Dietary
- Examination
- Laboratory
- Questionnaire
- Limited Access




There is a `Doc File` available for each dataset that provides information about the dataset, including the variables it contains, their coding, and any special considerations for using it. It is important to read the `Doc` file before using the dataset to understand the data structure and how to use it properly. The `Data File` contains the dataset data in `.XPT` format, which is a SAS Transport file, not a CSV.

2017-2018 Demographics Data - Continuous NHANES

Print

- NHANES 2017-2018 Demographics Variable List
- Questionnaire Instruments
- Exam Procedure Manuals
- SAS Universal Viewer
- Data User Agreement

| Data File Name | Doc File | Data File | Date Published |
| --- | --- | --- | --- |
| Demographic Variables and Sample Weights | DEMO_J Doc | DEMO_J Data [XPT - 3.3 MB] | February 2020 |

## Data

You need to click on the DEMO_J Data [XPT - 3.3 MB] link to download the dataset. Note that you cannot easily open an .XPT file in Excel or Google Sheets. In the real world, data isn't always in a friendly format. For this week, let's download the NHANES_Demo_Converted.csv file from our Canvas page to look at the rows. Later, we will use an AI co-pilot to write R code that reads .XPT files automatically.

Let's open the NHANES_Demo_Converted.csv file in Excel and see the first few rows:

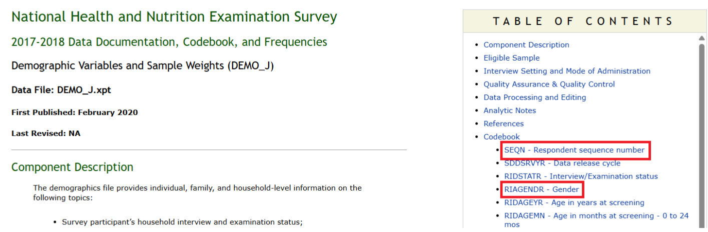
| | A | B | C | D | E | F | G |
| --- | --- | --- | --- | --- | --- | --- | --- |
| 1 | SEQN | SDDSRVYR | RIDSTATR | RIAGENDR | RIDAGEYR | RIDAGEMN | RIDRETH1 |
| 2 | 93703 | 10 | 2 | 2 | 2 | NA | 5 |
| 3 | 93704 | 10 | 2 | 1 | 2 | NA | 3 |
| 4 | 93705 | 10 | 2 | 2 | 66 | NA | 4 |
| 5 | 93706 | 10 | 2 | 1 | 18 | NA | 5 |
| 6 | 93707 | 10 | 2 | 1 | 13 | NA | 5 |
| 7 | 93708 | 10 | 2 | 2 | 66 | NA | 5 |
| 8 | 93709 | 10 | 2 | 2 | 75 | NA | 4 |
| 9 | 93710 | 10 | 2 | 2 | 0 | 11 | 3 |
| 10 | 93711 | 10 | 2 | 1 | 56 | NA | 5 |
| 11 | 93712 | 10 | 2 | 1 | 18 | NA | 1 |
| 12 | 93713 | 10 | 2 | 1 | 67 | NA | 3 |
| 13 | 93714 | 10 | 2 | 2 | 54 | NA | 4 |
| 14 | 93715 | 10 | 2 | 1 | 71 | NA | 5 |

- Each row represents a person.
- Each column represents a variable.
- The first row contains the variable names. The remaining rows contain the data for each person.

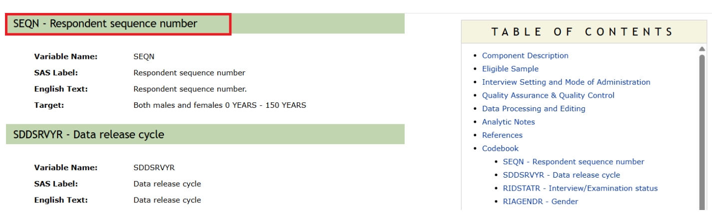
For example, SEQN is the unique identifier for each person in the dataset, and RIAGENDR is the variable for gender.

## Use of the Codebook

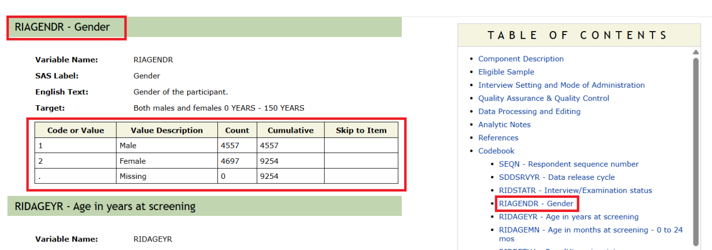
To understand the variables in the dataset, you can refer to the codebook. The codebook provides information about the variable names, labels, and values. The `Doc File` explained earlier contains the codebook for the dataset. You need to click on the `DEMO_J Doc` link to open the codebook on a new page. Let's we want to understand the variables `SEQN` and `RIAGENDR`:




You need to click on the `SEQN - Respondent sequence number` link or scroll down to find the information about the `SEQN` variable.




Similarly, you can click on the `RIAGENDR - Gender` link or scroll down to find the information about the `RIAGENDR` variable.



As you can see, the `RIAGENDR` variable has two values: 1 for Male and 2 for Female, without any missing values. You can also see the frequency of each value in the dataset. These numbers/frequencies should exactly match the frequencies in your dataset. You can use the codebook to understand other variables in the dataset. Similarly, you can use the other data files (e.g., Dietary) and their codebook to understand the variables in those datasets. Unlike the demographic file, which uses only one dataset and one codebook, the other files use multiple datasets and codebooks. As an example, if you click on the `Questionnaire Data` link, you will find multiple datasets and codebooks for questionnaire data:

## 2017-2018 Questionnaire Data - Continuous NHANES

Print

- NHANES 2017-2018 Questionnaire Variable List
- Questionnaire Instruments
- 2017-2018 Questionnaire Data Overview
- SAS Universal Viewer
- Data User Agreement

| Data File Name | Doc File | Data File | Date Published |
|---|---|---|---|
| Acculturation | ACQ_J Doc | ACQ_J Data [XPT - 396.3 KB] | February 2020 |
| Alcohol Use | ALQ_J Doc | ALQ_J Data [XPT - 434.4 KB] | December 2020 |
| Audiometry | AUQ_J Doc | AUQ_J Data [XPT - 3.9 MB] | July 2020 |
| Blood Pressure & Cholesterol | BPQ_J Doc | BPQ_J Data [XPT - 531.8 KB] | February 2020 |
| Cardiovascular Health | CDQ_J Doc | CDQ_J Data [XPT - 518.7 KB] | February 2020 |
| Consumer Behavior | CBQ_J Doc | CBQ_J Data [XPT - 435.4 KB] | April 2021 |
| Consumer Behavior Phone Follow-up | CBQPFA_J Doc | CBQPFA_J Data [XPT - 2.7 | June 2021 |

# BRFSS

The Behavioral Risk Factor Surveillance System (BRFSS) is a cross-sectional survey conducted by the US Centers for Disease Control (CDC). The survey collects information on health-related risk behaviors, chronic health conditions, and use of preventive services among US adults. BRFSS completes more than 400,000 adult interviews each year, making it the largest continuously conducted health survey system in the world.


CDC website for BRFSS

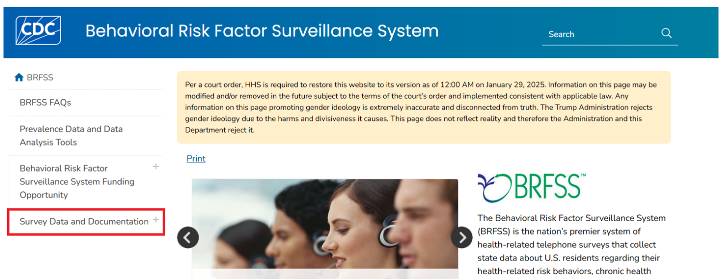
## About BRFSS

You can find more information about the BRFSS survey by clicking on BRFSS FAQs:

On the `BRFSS FAQs` page, you will find information about the BRFSS survey, including what the survey collects, how the survey is conducted, and how to use the data.

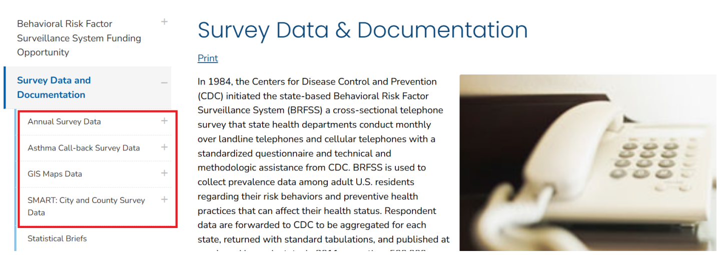
## Survey Data & Documentation

If you click on `Survey Data & Documentation` link, you will find information about the BRFSS questionnaires, datasets, and documentation:

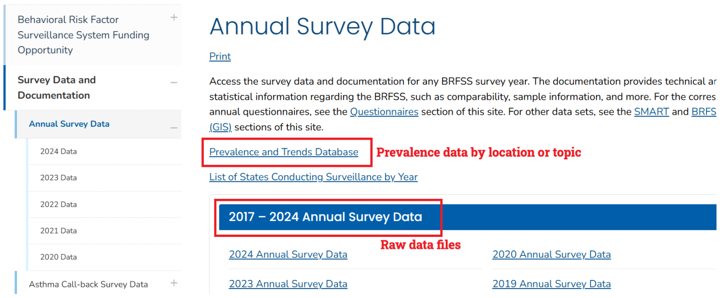



There are four types of data files available for download: 1) annual data, 2) asthma call-back data, 3) GIS map data, and 4) SMART data. The annual data files include information on health-related risk behaviors, chronic health conditions, and use of preventive services among US adults. The asthma call-back data files include information on asthma-related health outcomes and healthcare utilization among adults with asthma. The GIS map data files include information on the geographic distribution of the data. The SMART (Selected Metropolitan/Micropolitan Area Risk Trends) data provide prevalence rates for selected conditions and behaviors for cities and their surrounding counties.

BRFSS Survey Data & Documentation

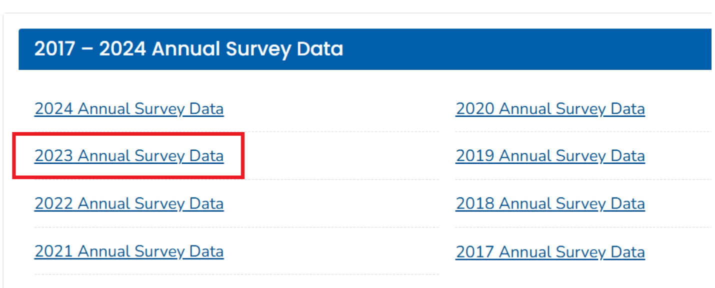


For the annual data, you can click on the `Annual Survey Data` link to find information about the annual data files, including how to access the data, how to use the data, and how to analyze the data. There are two types of annual data files available for download: 1) the raw data files and 2) the prevalence and trends database.

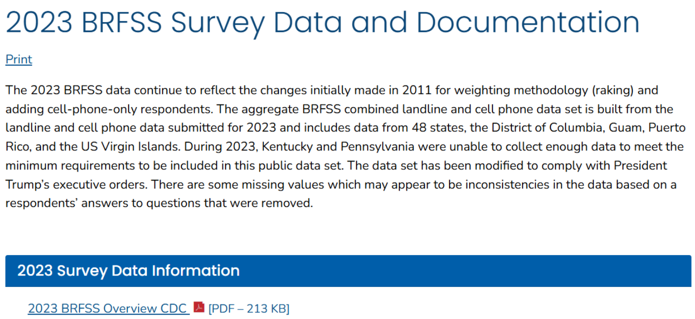



## Annual Survey Data

There are separate links for each year of data, and you can click on each link to find information about the data for that year. For example, if you click on the `2023 Annual Survey Data` link, you will find information about the 2023 annual data files, including how to access the data, how to use the data, and how to analyze the data.




Annual Survey Data



2023 BRFSS Survey Data and Documentation

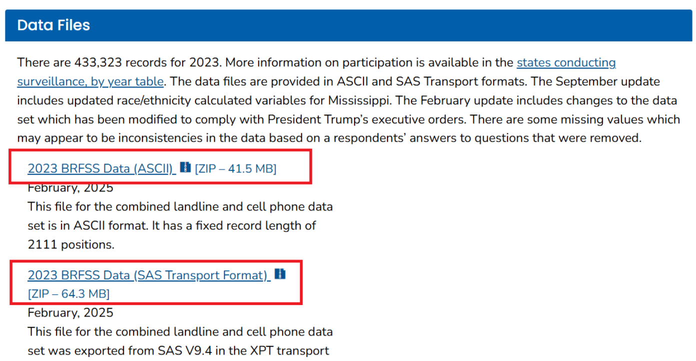
Print

The 2023 BRFSS data continue to reflect the changes initially made in 2011 for weighting methodology (raking) and adding cell-phone-only respondents. The aggregate BRFSS combined landline and cell phone data set is built from the landline and cell phone data submitted for 2023 and includes data from 48 states, the District of Columbia, Guam, Puerto Rico, and the US Virgin Islands. During 2023, Kentucky and Pennsylvania were unable to collect enough data to meet the minimum requirements to be included in this public data set. The data set has been modified to comply with President Trump's executive orders. There are some missing values which may appear to be inconsistencies in the data based on a respondents' answers to questions that were removed.

2023 Survey Data Information

2023 BRFSS Overview CDC [PDF – 213 KB]

The `2023 BRFSS Survey Data and Documentation` page provides information about three resources: 1) data information such as the codebook and sampling weights, 2) data files in ASCII and SAS XPT transport formats, and 3) SAS resources. To download/save the data files, click the specific format you want. For example, if you click on the `2023 BRFSS Data (SAS Transport Format)` link, the annual survey data for 2023 will be downloaded in a ZIP file that contains the dataset in a `.XPT` format.



Data Files

There are 433,323 records for 2023. More information on participation is available in the states conducting surveillance, by year table. The data files are provided in ASCII and SAS Transport formats. The September update includes updated race/ethnicity calculated variables for Mississippi. The February update includes changes to the data set which has been modified to comply with President Trump's executive orders. There are some missing values which may appear to be inconsistencies in the data based on a respondents' answers to questions that were removed.

2023 BRFSS Data (ASCII) [ZIP – 41.5 MB]
February, 2025
This file for the combined landline and cell phone data set is in ASCII format. It has a fixed record length of 2111 positions.

2023 BRFSS Data (SAS Transport Format) [ZIP – 64.3 MB]
February, 2025
This file for the combined landline and cell phone data set was exported from SAS V9.4 in the XPT transport

You need to unzip the file to extract the data file in `.XPT` format. However, opening the data file in Excel or Google Sheets is a bit tricky. The SAS XPT (or ASCII) data file needs to be converted/formatted appropriately before it can be opened in Excel or Google Sheets. Many statistical software packages, such as R, SAS, or STATA, can read the data file in `.XPT` format. Below is the data file, where the column names are the variable names, and the rows are the observations (i.e., survey respondents):

| | STATE FIPS CODE | FILE MONTH | INTERVIEW DATE | INTERVIEW MONTH | INTERVIEW DAY | INTERVIEW YEAR | FINAL DISPOSITION | ANNUAL SEQUENCE NUMBER | PRIMARY SAMPLING UNIT | CORRECT TELEPHONE NUMBER? | PRIVATE RESIDENCE? | DO YOU LIVE IN COLLEGE HOUSING? | RESIDENT OF STATE | CELLULAR TELEPHONE | ARE YOU YEARS O AGE OR OLDER? |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 03012023 | 03 | 01 | 2023 | 1100 | 2023000001 | 2023000001 | 1 | 1 | . | 1 | 2 | |
| 2 | 1 | 1 | 01062023 | 01 | 06 | 2023 | 1100 | 2023000002 | 2023000002 | 1 | 1 | . | 1 | 2 | |
| 3 | 1 | 1 | 03082023 | 03 | 08 | 2023 | 1100 | 2023000003 | 2023000003 | 1 | 1 | | 1 | 2 | |
| 4 | 1 | 1 | 03062023 | 03 | 06 | 2023 | 1100 | 2023000004 | 2023000004 | 1 | 1 | . | 1 | 2 | |
| 5 | 1 | 1 | 01062023 | 01 | 06 | 2023 | 1100 | 2023000005 | 2023000005 | 1 | 1 | . | 1 | 2 | |
| 6 | 1 | 1 | 01092023 | 01 | 09 | 2023 | 1100 | 2023000006 | 2023000006 | 1 | 1 | . | 1 | 2 | |
| 7 | 1 | 1 | 03212023 | 03 | 21 | 2023 | 1100 | 2023000007 | 2023000007 | 1 | 1 | . | 1 | 2 | |
| 8 | 1 | 1 | 01062023 | 01 | 06 | 2023 | 1100 | 2023000008 | 2023000008 | 1 | 1 | . | 1 | 2 | |
| 9 | 1 | 1 | 03152023 | 03 | 15 | 2023 | 1100 | 2023000009 | 2023000009 | 1 | 1 | . | 1 | 2 | |
| 10 | 1 | 1 | 01082023 | 01 | 08 | 2023 | 1100 | 2023000010 | 2023000010 | 1 | 1 | . | 1 | 2 | |
| 11 | 1 | 1 | 03082023 | 03 | 08 | 2023 | 1100 | 2023000011 | 2023000011 | 1 | 1 | . | 1 | 2 | |
| 12 | 1 | 1 | 01082023 | 01 | 08 | 2023 | 1100 | 2023000012 | 2023000012 | 1 | 1 | . | 1 | 2 | |
| 13 | 1 | 1 | 03012023 | 03 | 01 | 2023 | 1100 | 2023000013 | 2023000013 | 1 | 1 | . | 1 | 2 | |
| 14 | 1 | 1 | 01062023 | 01 | 06 | 2023 | 1200 | 2023000014 | 2023000014 | 1 | 1 | . | 1 | 2 | |
| 15 | 1 | 1 | 03212023 | 03 | 21 | 2023 | 1100 | 2023000015 | 2023000015 | 1 | 1 | . | 1 | 2 | |
| 16 | 1 | 1 | 03132023 | 03 | 13 | 2023 | 1100 | 2023000016 | 2023000016 | 1 | 1 | . | 1 | 2 | |

To understand the variables in the dataset, you can refer to the codebook:

**2023 Survey Data Information**

2023 BRFSS Overview CDC [PDF – 213 KB]
Provides information on the background, design, data collection and processing, and the statistical and analytical issues for the combined landline and cell phone data set.

2023 BRFSS Codebook CDC [ZIP – 3 MB]
Codebook for the file showing variable name, location, and frequency of values for all reporting areas combined for the combined landline and cell phone data set.

You need to unzip the file to extract the codebook in HTML format. The codebook provides information about the variable names, labels, and values. For example, if you want to understand the variable `Drinking and Driving`, you can find the variable name, label, and values in the codebook:

Label: Drinking and Driving
Section Name: Calculated Variables
Module Number: 15
Question Number: 3
Column: 2110
Type of Variable: Num
SAS Variable Name: _DRNKDRV
Question Prologue:
Question: Drinking and Driving (Reported having driven at least once when perhaps had too much to drink)

| Value | Value Label | Frequency | Percentage | Weighted Percentage |
|---|---|---|---|---|
| 1 | Have driven after having too much to drink | 6,192 | 1.43 | 1.36 |
| 2 | Have not driven after having too much to drink | 205,541 | 47.43 | 45.81 |
| 9 | Don't know/Not Sure/Refused/Missing | 221,590 | 51.14 | 52.83 |

You can similarly download the asthma call-back, GIS map, and SMART datasets. To understand the variables in those datasets, you can refer to the codebook for each dataset.

Asthma Call-back Data

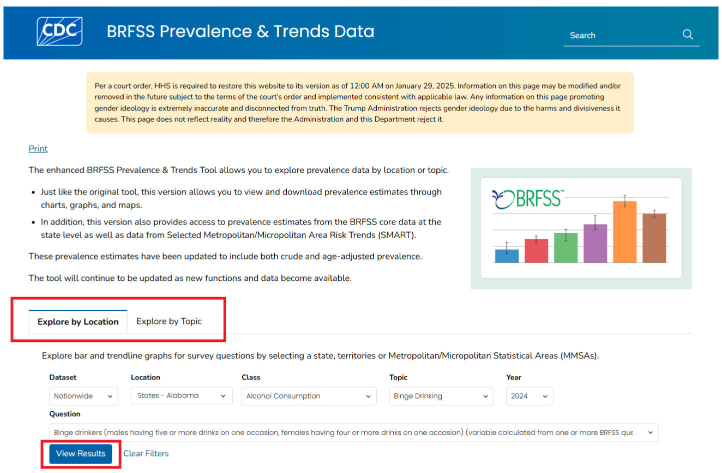
GIS Data

SMART Data

## Prevalence and Trends Database

The BRFSS prevalence and trends tool allows users to view and download prevalence estimates through charts, graphs, and maps. You can explore the estimates by selecting the topic or location. For example, if you select `by topic`, e.g., for alcohol consumption for Alabama for the year 2024, you need to select the options from the dropdown menu and click on the `View Results` button to see the results:

Prevalence and Trends Database




The results will show the (i) prevalence estimates for alcohol consumption in Alabama for the year 2024, and (ii) the trend of alcohol consumption in Alabama over time.

View Results Clear Filters

Binge drinkers (males having five or more drinks on one occasion, females having four or more drinks on one occasion) (variable calculated from one or more BRFSS questions)

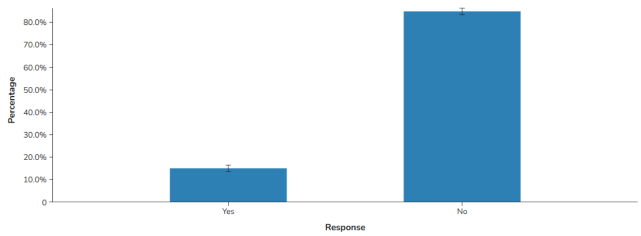
*Alcohol Consumption: Binge Drinking*

View By: Overall

Data Type: Crude Prevalence

2024, Alabama

Chart Table

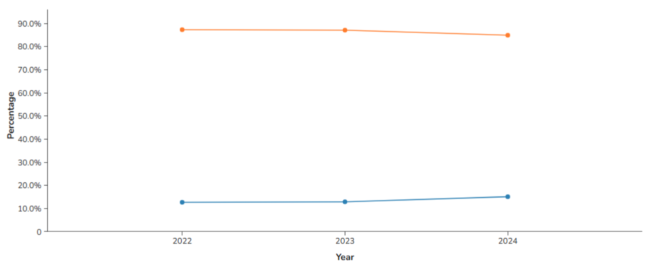



The ordering of subcategory rows in the table view may vary across responses.

All Available Years, Alabama

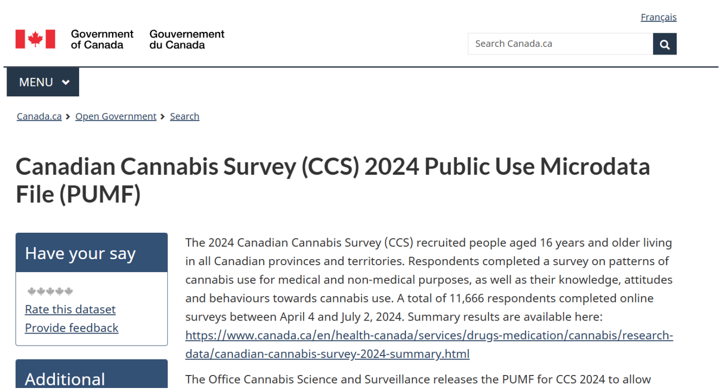
Chart Table

Response: All Responses



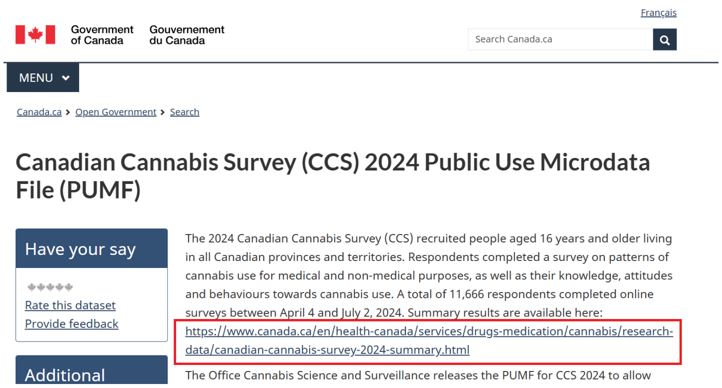
# CCS

The Canadian Cannabis Survey (CCS) is an annual survey conducted by Health Canada to gather information on cannabis use among people aged 16 years and older living in all Canadian provinces and territories. The survey collects data on various aspects of cannabis consumption, including their knowledge, attitudes and behaviours towards cannabis use.




More than 11,000 respondents participated in the 2024 survey. The study provides valuable insights into cannabis use patterns, the cannabis market, and issues of public safety. The survey results are published on Health Canada's website:

Health Canada website for CCS



Français

Government of Canada Gouvernement du Canada

Search Canada.ca

MENU

Canada.ca > Open Government > Search

Canadian Cannabis Survey (CCS) 2024 Public Use Microdata File (PUMF)

Have your say

Rate this dataset

Provide feedback

Additional

The 2024 Canadian Cannabis Survey (CCS) recruited people aged 16 years and older living in all Canadian provinces and territories. Respondents completed a survey on patterns of cannabis use for medical and non-medical purposes, as well as their knowledge, attitudes and behaviours towards cannabis use. A total of 11,666 respondents completed online surveys between April 4 and July 2, 2024. Summary results are available here: https://www.canada.ca/en/health-canada/services/drugs-medication/cannabis/research-data/canadian-cannabis-survey-2024-summary.html

The Office Cannabis Science and Surveillance releases the PUMF for CCS 2024 to allow

Français

Government of Canada Gouvernement du Canada

Search Canada.ca

MENU

Canada.ca > Health > Drug and health products > Drugs and medication > Cannabis > Cannabis research and data

Canadian Cannabis Survey 2024: Summary

On this page

- Introduction
- Results by theme
- Methods
- Considerations

## Open Licence

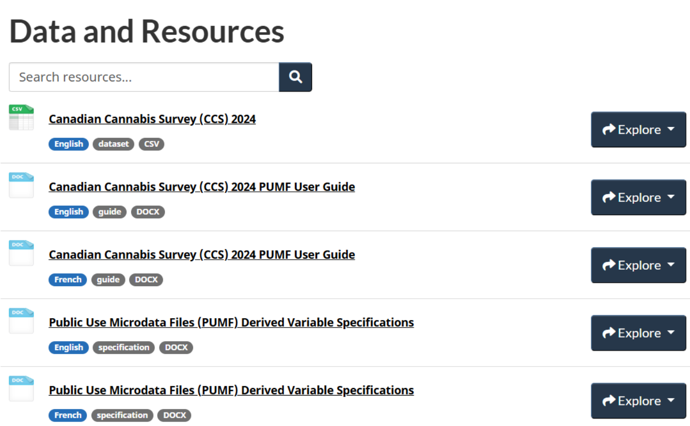
The data from the Canadian Cannabis Survey is available under the Open Government Licence - Canada (OGL-Canada). This licence allows users to freely use, modify, and distribute the data, provided that they attribute the source of the data and do not use it for commercial purposes. The OGL-Canada promotes transparency and encourages the use of government data for research, analysis, and public benefit. You can follow the `Open Government Licence - Canada` link for more information on the terms and conditions of the licence.

Open Licence

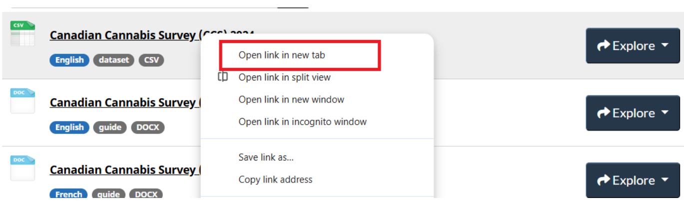


## Data and Resources

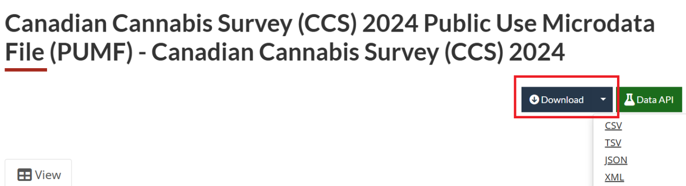
The `Data and Resources` tab on the Health Canada website provides access to the public use data, user guides, and the data dictionary.



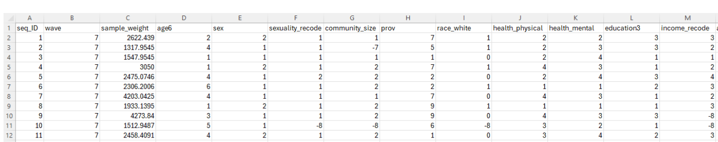

The CCS 2024 data is available to download in various formats, including CSV, TSV, JSON, and XML. The data can be accessed through the Health Canada website, where you can choose the format that best suits your needs. You need to open the data link (say, in a new tab in your browser) and select the desired format to download the data.




Under the `Download` tab, you can find the data in different formats. For example, you can download the data in CSV format by clicking on the `CSV` link:



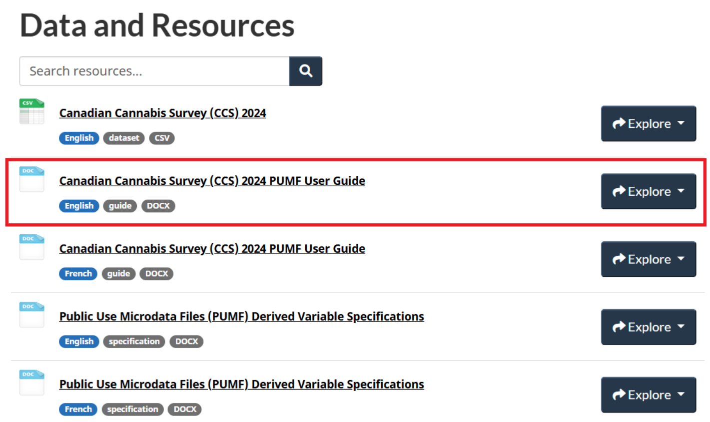

After downloading, you can open the data in Excel, Google Sheets, or any statistical software. The data is structured in a tabular format, with each row representing an individual respondent and each column representing different variables:



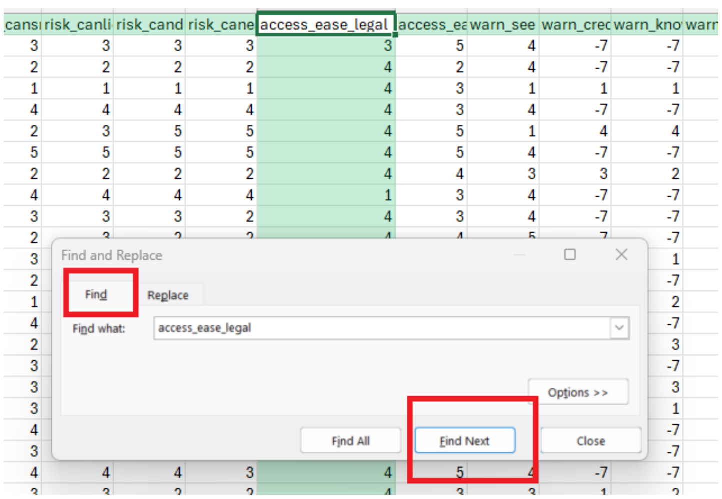
| | A | B | C | D | E | F | G | H | I | J | K | L | M |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | seq_ID | wave | sample_weight | age6 | sex | sexuality_recode | community_size | prov | race_white | health_physical | health_mental | education3 | income_recode |
| 2 | 1 | 7 | 2622.439 | 2 | 2 | 1 | 1 | 7 | 1 | 2 | 2 | 3 | 3 |
| 3 | 2 | 7 | 1317.9545 | 4 | 1 | 1 | -7 | 5 | 1 | 2 | 3 | 3 | 2 |
| 4 | 3 | 7 | 1547.9545 | 1 | 1 | 1 | 1 | 1 | 0 | 2 | 4 | 1 | 1 |
| 5 | 4 | 7 | 3050 | 1 | 2 | 1 | 2 | 7 | 1 | 4 | 3 | 1 | 2 |
| 6 | 5 | 7 | 2475.0746 | 4 | 1 | 2 | 2 | 2 | 0 | 2 | 4 | 3 | 4 |
| 7 | 6 | 7 | 2306.2006 | 6 | 1 | 1 | 2 | 2 | 1 | 1 | 1 | 2 | 3 |
| 8 | 7 | 7 | 4203.0425 | 4 | 1 | 1 | 1 | 7 | 0 | 4 | 3 | 1 | 2 |
| 9 | 8 | 7 | 1933.1395 | 1 | 2 | 1 | 2 | 9 | 1 | 1 | 1 | 1 | 3 |
| 10 | 9 | 7 | 4273.84 | 3 | 1 | 1 | 2 | 9 | 0 | 4 | 3 | 3 | -8 |
| 11 | 10 | 7 | 1512.9487 | 5 | 1 | -8 | -8 | 6 | -8 | 3 | 2 | 1 | -8 |
| 12 | 11 | 7 | 2458.4091 | 4 | 2 | 1 | 2 | 1 | 0 | 3 | 4 | 2 | 3 |
| 13 | 12 | 7 | 1860.8333 | 1 | 1 | 1 | 2 | 3 | 1 | 3 | 4 | 1 | 1 |

The `Canadian Cannabis Survey (CCS) 2024 PUMF User Guide` gives detailed information on the survey methodology, data collection process, and how to use the public use microdata file (PUMF). You can also find the variable names and their descriptions in this file.

2024 Public Use Microdata File




Let's find a variable (`access_ease_legal`) in the Excel file and check its description in the data dictionary. You can use the `Find` function in Excel (`Ctrl + F`) to search for the variable name:




Then you can see the description of the variable in the `Canadian Cannabis Survey (CCS) 2024 PUMF User Guide`, which provides information on what the variable represents and how it was coded:

| | |
|---|---|
| **access_ease_legal**<br><br>*[variable added in 2024 to replace* ***access_ease****]* | How easy or difficult is it for you to get legal cannabis?<br><br>❍ 1 Very difficult<br>❍ 2 Fairly difficult<br>❍ 3 Fairly easy<br>❍ 4 Very easy<br>❍ 5 I don't know<br>❑ -7 Missing |

In this dataset, `-7` generally represents missing. Always check the data dictionary for the specific coding of missing values, "don't know," or "refused" responses, as they can vary between datasets.

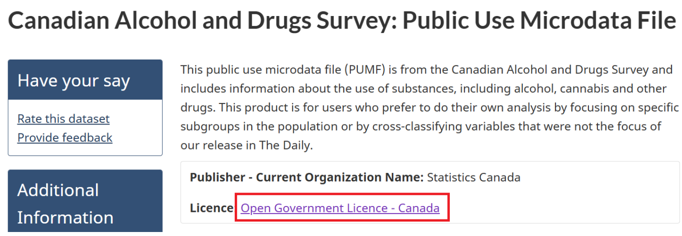
# CADS

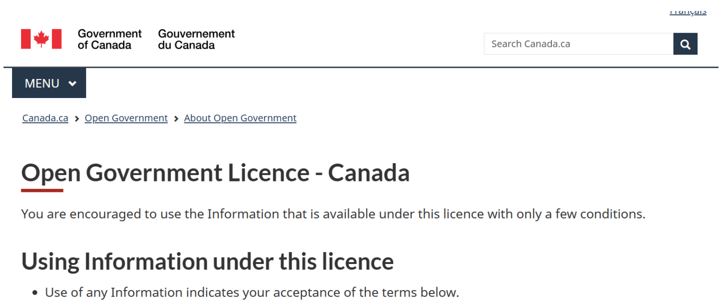
The Canadian Alcohol and Drugs Survey (CADS) is a national survey conducted by Statistics Canada that collects data on the use of substances, such as alcohol, cannabis and other drugs. The study was conducted by Statistics Canada in 2019 with the cooperation and support of Health Canada. The target population for the survey was all persons aged 15 years and over living in Canada, excluding residents of the Yukon, Northwest Territories, and Nunavut, full-time residents of institutions, and residents of Native reserves.


Health Canada website for CADS
CADS 2019 summary of results

## Open Licence

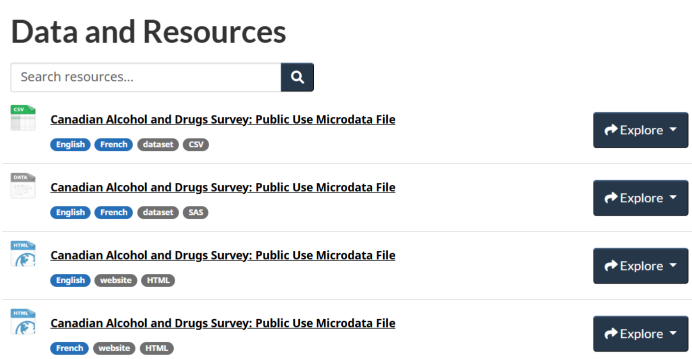
Similar to the Canadian Cannabis Survey, the CADS is also licensed under the Open Government Licence - Canada. This means that the data can be freely used, modified, and shared by anyone for any purpose, as long as they provide proper attribution to the source of the data.



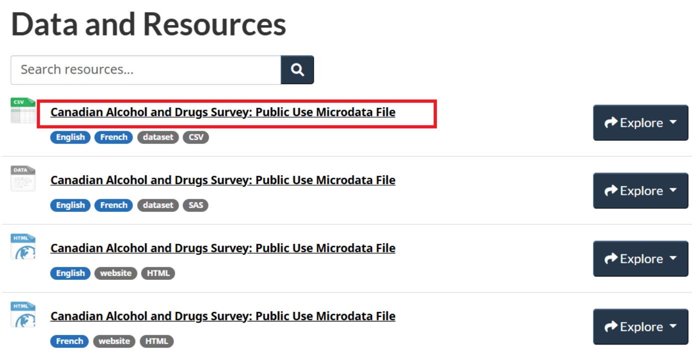




## Data and Resources

The `Data and Resources` tab provides access to the public use data, user guides, and the data dictionary.

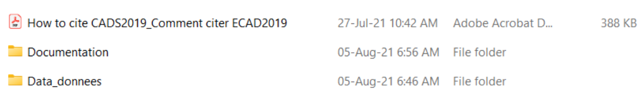



The public use data file is available in a ZIP format. You need to open the `Canadian Alcohol and Drugs Survey: Public Use Microdata File` link, and click on `Go to resource`to download the ZIP file.



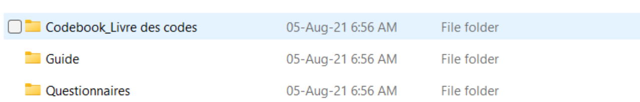


The ZIP file contains the following files: an instruction on how to cite CADS 2019, data documentation, and the dataset in CSV format.

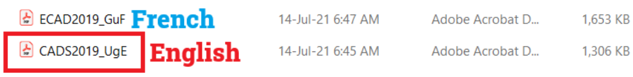



## Data Documentation

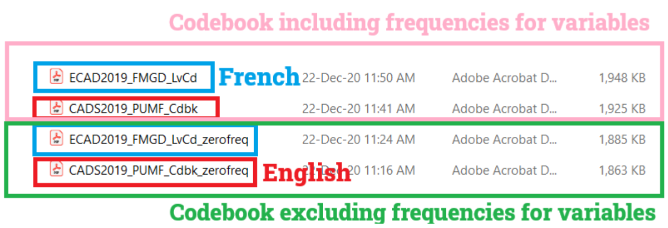
The data documentation folder includes three sub-folders: `Codebook_Livre des codes`, `Guide`, and `Questionnaires`.




The `Guide` folder contains user guides (in English and French) that provide an overview of the survey, including the methodology, data collection process, data processing, data quality, and sampling weights.



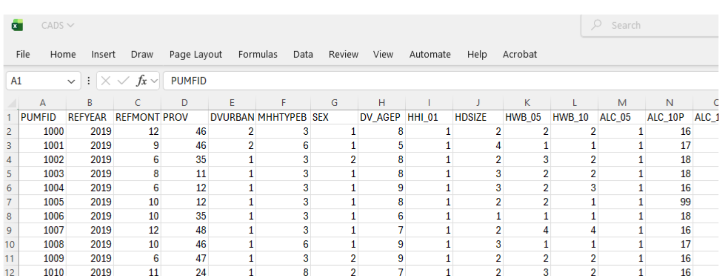

The `Codebook_Livre des codes` folder contains codebooks (in English and French) that provide detailed information about the variables in the dataset, including variable names, labels, and values. There are two types of codebooks: those with and those without variable frequencies. But both codebooks provide the same information about the variables in the dataset.




The `Questionnaires` folder contains the survey questionnaires (in English and French) that were used to collect the data.

## Data

The `Data_donnees` folder contains the dataset in CSV format. You can open the dataset in Excel or any statistical software to explore the data. The row represents the respondent, and the column represents the variable:




| | A | B | C | D | E | F | G | H | I | J | K | L | M | N |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | PUMFID | REFYEAR | REFMONT | PROV | DVURBAN | MHHTYPEB | SEX | DV_AGEP | HHI_01 | HDSIZE | HWB_05 | HWB_10 | ALC_05 | ALC_10P |
| 2 | 1000 | 2019 | 12 | 46 | 2 | 3 | 1 | 8 | 1 | 2 | 2 | 2 | 1 | 16 |
| 3 | 1001 | 2019 | 9 | 46 | 2 | 6 | 1 | 5 | 1 | 4 | 1 | 1 | 1 | 17 |
| 4 | 1002 | 2019 | 6 | 35 | 1 | 3 | 2 | 8 | 1 | 2 | 3 | 2 | 1 | 18 |
| 5 | 1003 | 2019 | 8 | 11 | 1 | 3 | 1 | 8 | 1 | 3 | 2 | 2 | 1 | 18 |
| 6 | 1004 | 2019 | 6 | 12 | 1 | 3 | 1 | 9 | 1 | 3 | 2 | 3 | 1 | 16 |
| 7 | 1005 | 2019 | 10 | 12 | 1 | 3 | 1 | 8 | 1 | 2 | 2 | 1 | 1 | 99 |
| 8 | 1006 | 2019 | 10 | 35 | 1 | 3 | 1 | 6 | 1 | 1 | 1 | 1 | 1 | 18 |
| 9 | 1007 | 2019 | 12 | 48 | 1 | 3 | 1 | 7 | 1 | 2 | 4 | 4 | 1 | 16 |
| 10 | 1008 | 2019 | 10 | 46 | 1 | 6 | 1 | 9 | 1 | 3 | 1 | 1 | 1 | 17 |
| 11 | 1009 | 2019 | 6 | 47 | 1 | 3 | 2 | 9 | 1 | 2 | 2 | 2 | 1 | 16 |
| 12 | 1010 | 2019 | 11 | 24 | 1 | 8 | 2 | 7 | 1 | 2 | 3 | 2 | 1 | 16 |

You can refer to the codebook to understand the variables that you are interested in for your analysis. For example, if you are interested in the variable `DVURBAN`, you can refer to the codebook to find out that the variable (e.g., for the English version of the codebook with frequencies):

**Variable Name:** DVURBAN **Length:** 1.0 **Position:** 14

**Question Name:**

**Concept:** Characteristics of community where the respondent lives

**Question Text:**

**Universe:** All respondents

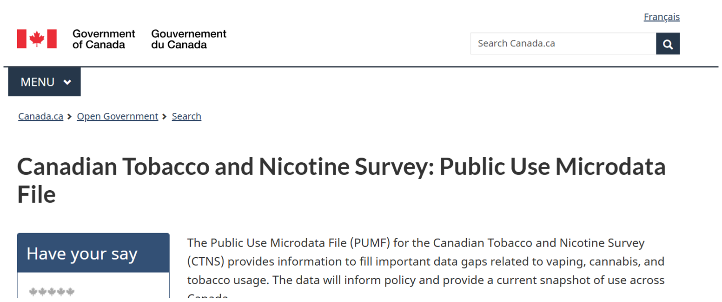
**Note:** The category 'Population centre' is historically comparable to 'Urban' which was used in previous years for CTUMS.

**Source:**

| Answer Categories | Code | Frequency | Weighted Frequency | % |
|---|---|---|---|---|
| Population centre | 1 | 7,208 | 21,321,106 | 68.8 |
| Rural | 2 | 2,438 | 5,122,120 | 16.5 |
| Not stated | 9 | 647 | 4,552,224 | 14.7 |
| **Total** | | 10,293 | 30,995,450 | 100.0 |

# CTNS

The Canadian Tobacco and Nicotine Survey (CTNS) is a cross-sectional survey that collects information on tobacco and nicotine use among Canadians aged 15 years and older. The survey is conducted by Statistics Canada on behalf of Health Canada and provides valuable insights into the prevalence of cigarette smoking, vaping, cannabis, and alcohol use.



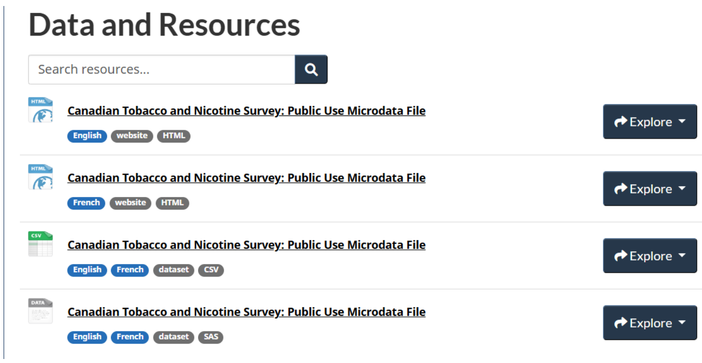

Health Canada website for CTNS
CTNS 2022 summary of results

## Open Licence

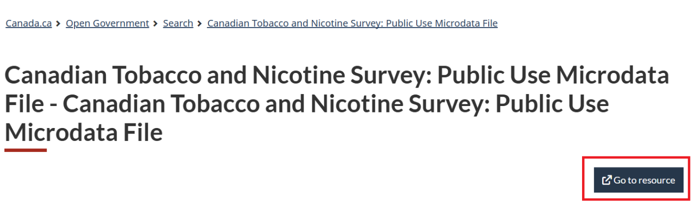
The CTNS is licensed under the Open Government Licence - Canada. This means that the data can be freely used, modified, and shared by anyone for any purpose, as long as they provide proper attribution to the source of the data.

## Data and Resources

The `Data and Resources` tab provides access to the public use data, user guides, and the data dictionary.



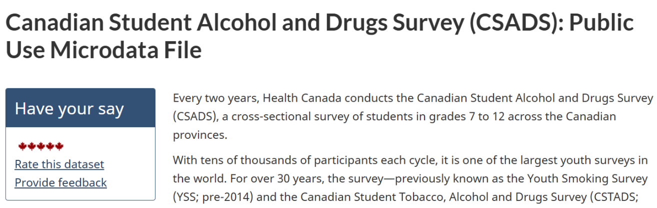

The public use data file is available in a ZIP format. You can click on the `Canadian Tobacco and Nicotine Survey: Public Use Microdata File` link, followed by `Go to resource` link to download the ZIP file.


Similar to the Canadian Alcohol and Drugs Survey (CADS) tutorial, the ZIP file for CTNS contains the following files: citation instructions, data documentation, and the dataset in CSV format. The data documentation, questionnaire, and codebook are also the same as those for CADS. The only difference is that the data documentation and questionnaire for CTNS are labelled `CTNS` rather than `CADS`.

# CSADS

The Canadian Student Alcohol and Drugs Survey (CSADS) is a national survey that collects data on the alcohol and drug use patterns of Canadian students in grades 7 to 12. The survey is conducted every two years and provides valuable insights into the prevalence and trends of substance use among Canadian youth.


Canadian Student Alcohol and Drugs Survey (CSADS): Public Use Microdata File

Have your say

Rate this dataset

Provide feedback

Every two years, Health Canada conducts the Canadian Student Alcohol and Drugs Survey (CSADS), a cross-sectional survey of students in grades 7 to 12 across the Canadian provinces.

With tens of thousands of participants each cycle, it is one of the largest youth surveys in the world. For over 30 years, the survey—previously known as the Youth Smoking Survey (YSS; pre-2014) and the Canadian Student Tobacco, Alcohol and Drugs Survey (CSTADS;

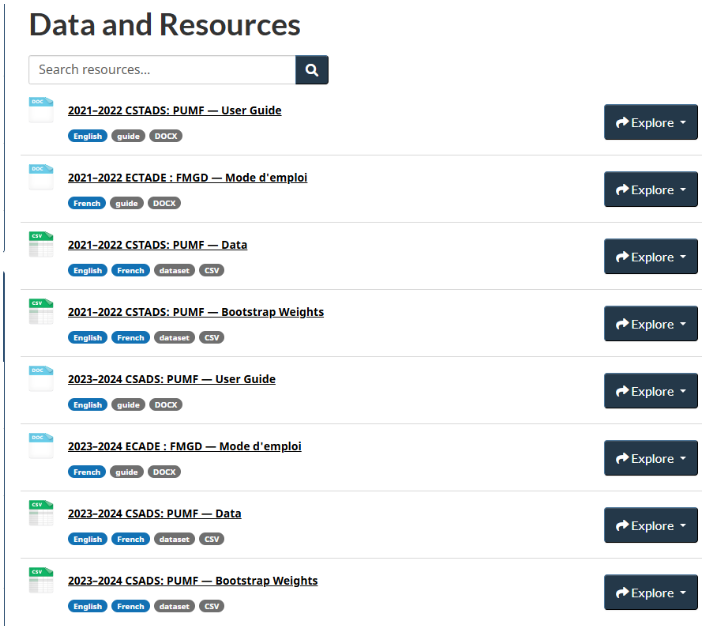
Health Canada website CSADS summary of results CTNS

## Open Licence

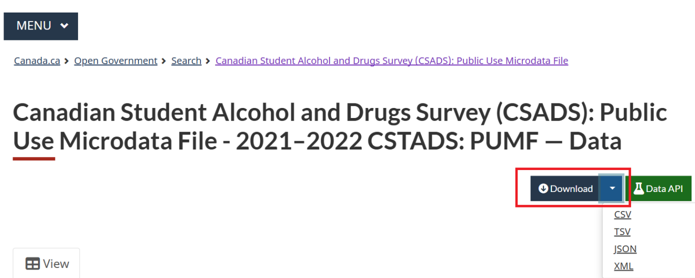
The CTNS is licensed under the Open Government Licence - Canada. This means that the data can be freely used, modified, and shared by anyone for any purpose, as long as they provide proper attribution to the source of the data.

## Data and Resources

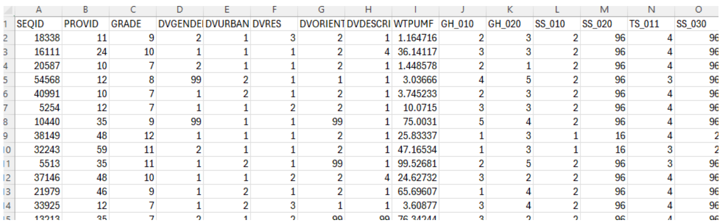
The `Data and Resources` tab provides access to the public use data, user guides, the data dictionary, and information on bootstrap weights.




To download the 2021–2022 CSTADS, you can click on the `2021-2022 CSTADS: PUMF - Dataset in CSV format` link. The data is available to download in various formats. Under the `Download` tab, you can find the formats. For example, you can download the data in CSV format by clicking on the `CSV` link:

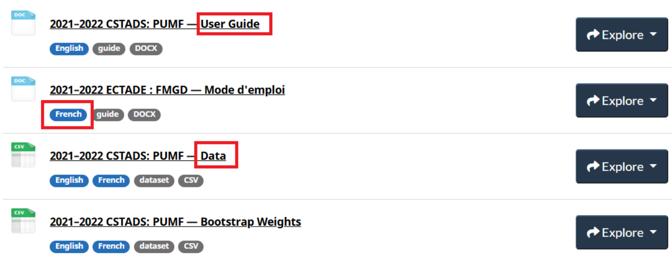


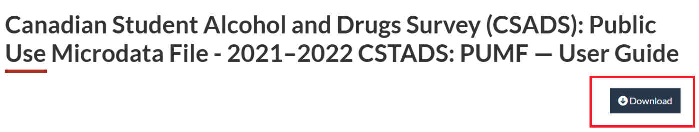

After downloading, you can open the data in Excel, Google Sheets, or any statistical software. The data is structured in a tabular format, with each row representing an individual respondent and each column representing different variables:

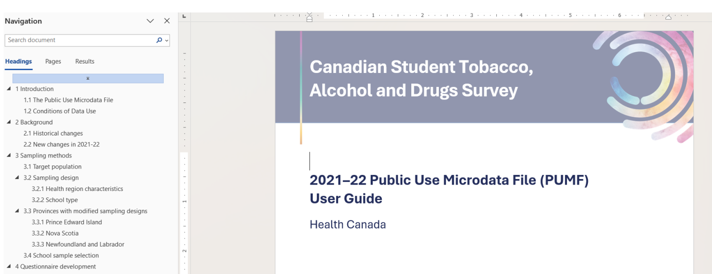


| SEQID | PROVID | GRADE | DVGENDE | DVURBAN | DVRES | DVORIENT | DVDESCRI | WTPUMF | GH_010 | GH_020 | SS_010 | SS_020 | TS_011 | SS_030 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 18338 | 11 | 9 | 2 | 1 | 3 | 2 | 1 | 1.164716 | 2 | 3 | 2 | 96 | 4 | 96 |
| 16111 | 24 | 10 | 1 | 1 | 1 | 2 | 4 | 36.14117 | 3 | 3 | 2 | 96 | 4 | 96 |
| 20587 | 10 | 7 | 2 | 1 | 1 | 2 | 1 | 1.448578 | 2 | 1 | 2 | 96 | 4 | 96 |
| 54568 | 12 | 8 | 99 | 2 | 1 | 1 | 1 | 3.03666 | 4 | 5 | 2 | 96 | 3 | 96 |
| 40991 | 10 | 7 | 1 | 2 | 1 | 2 | 1 | 3.745233 | 2 | 3 | 2 | 96 | 4 | 96 |
| 5254 | 12 | 7 | 1 | 1 | 2 | 2 | 1 | 10.0715 | 3 | 3 | 2 | 96 | 4 | 96 |
| 10440 | 35 | 9 | 99 | 1 | 1 | 99 | 1 | 75.0031 | 5 | 4 | 2 | 96 | 4 | 96 |
| 38149 | 48 | 12 | 1 | 1 | 1 | 2 | 1 | 25.83337 | 1 | 3 | 1 | 16 | 4 | 2 |
| 32243 | 59 | 11 | 2 | 1 | 1 | 2 | 1 | 47.16534 | 1 | 3 | 1 | 16 | 3 | 2 |
| 5513 | 35 | 11 | 1 | 2 | 1 | 99 | 1 | 99.52681 | 2 | 5 | 2 | 96 | 3 | 96 |
| 37146 | 48 | 10 | 1 | 1 | 2 | 2 | 4 | 24.62732 | 3 | 2 | 2 | 96 | 4 | 96 |
| 21979 | 46 | 9 | 1 | 2 | 1 | 2 | 1 | 65.69607 | 1 | 4 | 2 | 96 | 4 | 96 |
| 33925 | 12 | 7 | 1 | 2 | 3 | 1 | 1 | 3.60877 | 3 | 4 | 2 | 96 | 4 | 96 |

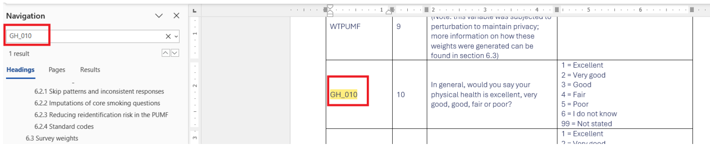
The user guide gives detailed information on the survey methodology, data collection process, and how to use the public use microdata file (PUMF). You can also find the variable names and their descriptions in this `DOCX` file.



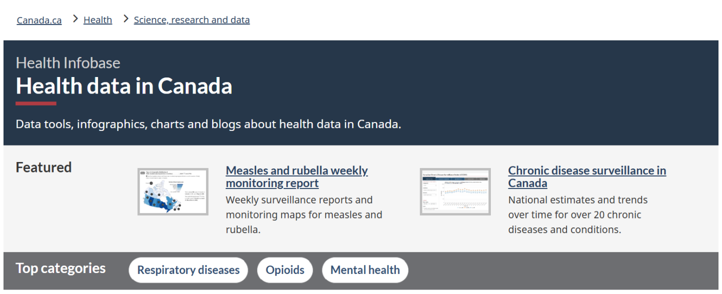




To find a variable description, you are referred to the `User Guide`. For example, the variable `GH_010` is described as follows:


Navigation
GH_010
1 result
Headings
Pages
Results
6.2.1 Skip patterns and inconsistent responses
6.2.2 Imputations of core smoking questions
6.2.3 Reducing reidentification risk in the PUMF
6.2.4 Standard codes
6.3 Survey weights
WTPUMF
9
perturbation to maintain privacy; more information on how these weights were generated can be found in section 6.3)
GH_010
10
In general, would you say your physical health is excellent, very good, good, fair or poor?
1 = Excellent
2 = Very good
3 = Good
4 = Fair
5 = Poor
6 = I do not know
99 = Not stated
1 = Excellent

# PHAC Health Infobase

The Health Infobase is a comprehensive digital platform maintained by the Public Health Agency of Canada (PHAC). The Infobase provides access to a wide range of health-related data tools, infographics, charts and blogs. There are 182 data sources available on the Infobase. The Health Infobase serves as a central hub for national surveillance data, interactive data tools, and public health reports. The platform is frequently updated with the latest surveillance data.




PHAC Health Infobase

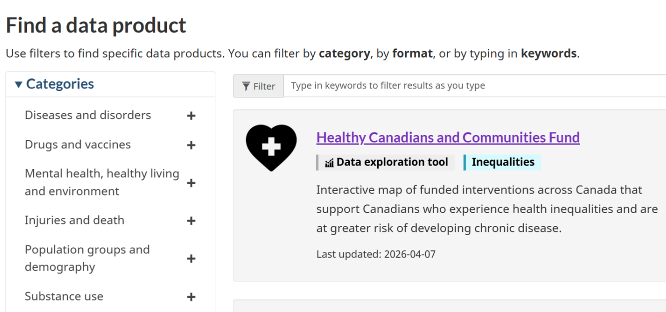
## Data Categories

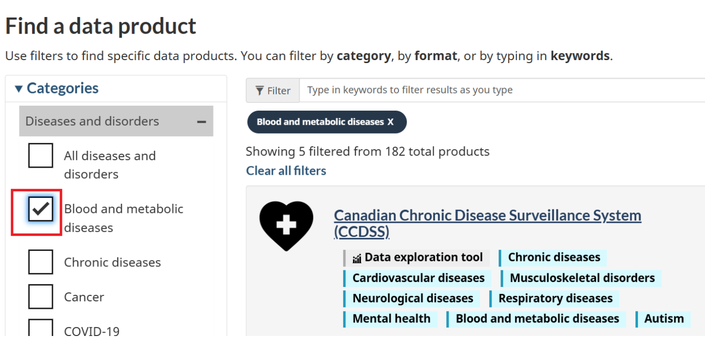
The Infobase provides access to health data under six categories:

- `Diseases and disorders`: Comprehensive tracking of chronic conditions such as cancer, cardiovascular diseases, and COVID-19.
- `Drugs and vaccines`: Data on prescription drugs, vaccine coverage, and immunization rates across the country. ‘

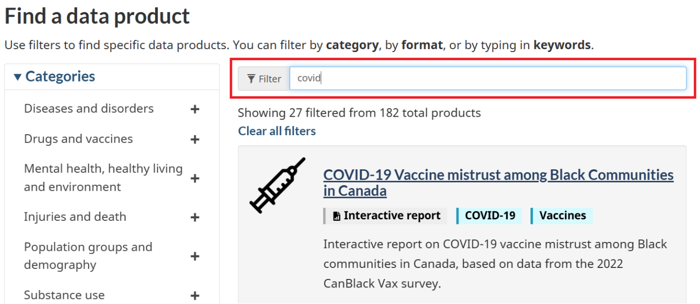
- `Mental health, healthy living and environment`: Information on mental health indicators, physical activity levels, nutrition, and environmental factors such as wildfires.
- `Injuries and death`: Data on death, suicide, violence, and drowning across regions.
- `Population groups and demography`: Data on birth, maternity, children, family, and disabilities.
- `Substance use`: Data on substance use such as alcohol, cannabis use, opioids, and smoking.

Under each category, users can select the data for specific health topics. The three top categories are `Respiratory diseases`, `Opioids` and `Mental health`.



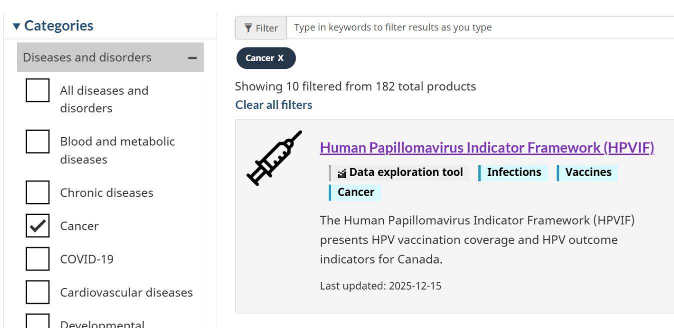

You can filter the data by category, by format, or by typing in keywords.

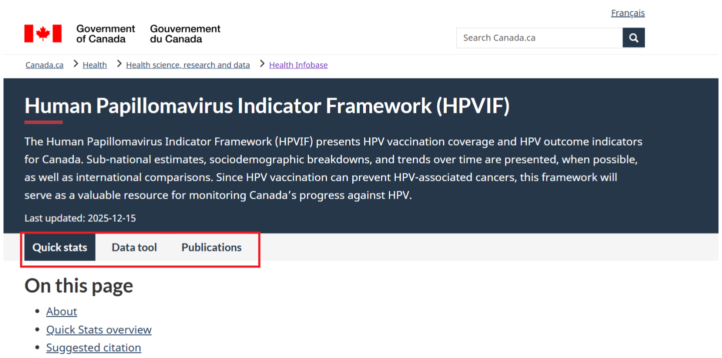



You can also filter the data by type, such as blog, infographic, or report.

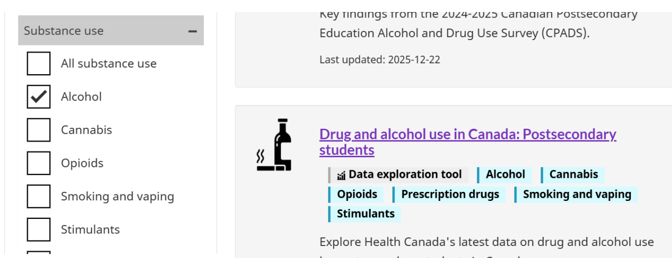

## Data Availability

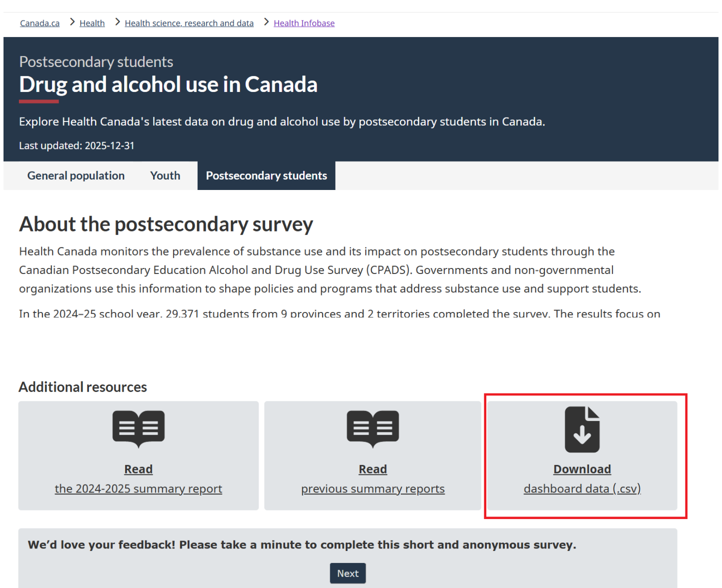
The level of information provided in the data sources varies. For some data sources/products, you can find blogs, infographics, and reports along with individual-level data. However, for some data sources/products, you may only find the summary statistics without individual-level data, i.e., only aggregated-level data. For example, the `Human Papillomavirus Indicator Framework` product provides quick statistics, data tools and a list of publications:





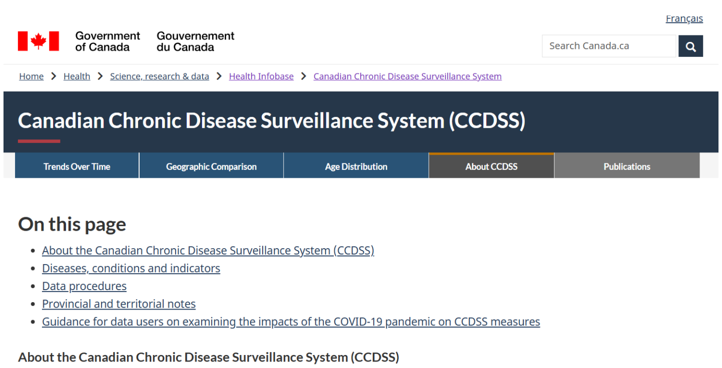

The Drug and alcohol use in Canada: Postsecondary students product provides summary statistics as well as links to download data in CSV format:



# CCDSS

The Canadian Chronic Disease Surveillance System (CCDSS) is a collaborative network of provincial and territorial chronic disease surveillance systems in Canada. The CCDSS is supported by the Public Health Agency of Canada (PHAC) and provincial/territorial governments. The CCDSS was established to provide the estimates of the prevalence and incidence of chronic diseases in Canada, as well as to monitor trends over time for more than 20 chronic diseases to help inform health policies and resource planning. The CCDSS is part of the PHAC Health Infobase that we covered in the previous tutorial on Health Infobase. The CCDSS uses health administrative data, such as hospital discharge records, physician billing claims, and prescription drug data.




CCDSS website

## Key Indicator Categories

The CCDSS website provides a variety of indicators related to chronic diseases, such as autism, cardiovascular diseases, chronic respiratory diseases, diabetes, mental health disorders, musculoskeletal disorders, neurological disorders, and multimorbidity.

Diseases, conditions and indicators

Table 1: Diseases, conditions and indicators included in the CCDSS

| | Morbidity and mortality | Health events and complications | Use of health services |
|---|---|---|---|
| | • incidence/rate of newly identified cases<br>• prevalence (cumulative and active for select diseases/conditions)<br>• all-cause mortality | • annual prevalence<br>• all-cause mortality following a health event | • annual prevalence |

Table 2: CCDSS case definitions and surveillance period reported

| Disease, condition and indicator | Age | Case definition summary | First reported year | Last reported year |
|---|---|---|---|---|
| **Autism** | | | | |
| Autism | 1-19 | One or more hospital separation records, or two or more physician claims | 2000–2001 | 2023–2024 |
| **Cardiovascular diseases** | | | | |
| Acute myocardial infarction | 20+ | One or more hospital inpatient admission records | 2000–2001 | 2023–2024 |

## Data Tool

The CCDSS website provides a data tool that allows users to explore and visualize the data on chronic diseases in Canada. The data tool provides interactive charts and tables that allow users to explore the data by province/territory.

**About the Canadian Chronic Disease Surveillance System (CCDSS)**

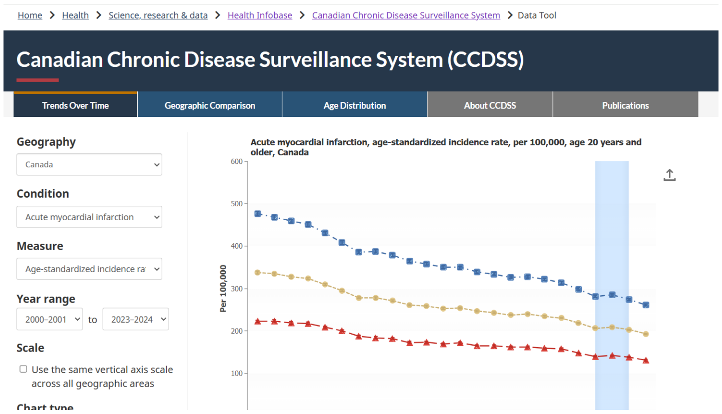
The CCDSS is a collaborative network of provincial and territorial surveillance systems, supported by the Public Health Agency of Canada (PHAC). The system collects data on all residents who are eligible for provincial or territorial health insurance. It can generate national estimates and trends over time for over 20 chronic diseases and conditions, and other selected health outcomes. To identify people with chronic diseases and conditions, provincial and territorial health insurance registry records are linked using a unique personal identifier to the corresponding physician claims, hospital separation records and prescription drug records.

Key information about the CCDSS methods and case definitions is presented below. More detailed documents are available here: CCDSS summary of methods; CCDSS case definitions; CCDSS COVID-19 guidance for data users.

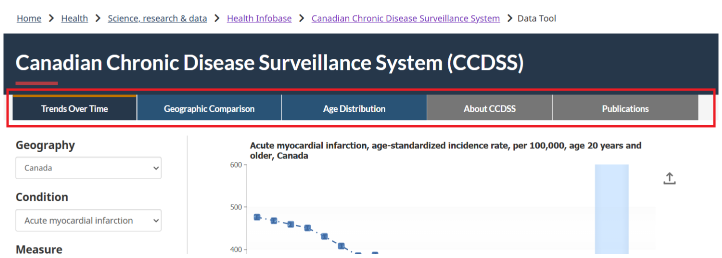
Use the CCDSS Data Tool

Users can use the data tool to explore the prevalence and incidence of cases, mortality rates, and health service utilization, and to track individuals living with two or more chronic conditions.

CCDSS Data Tool

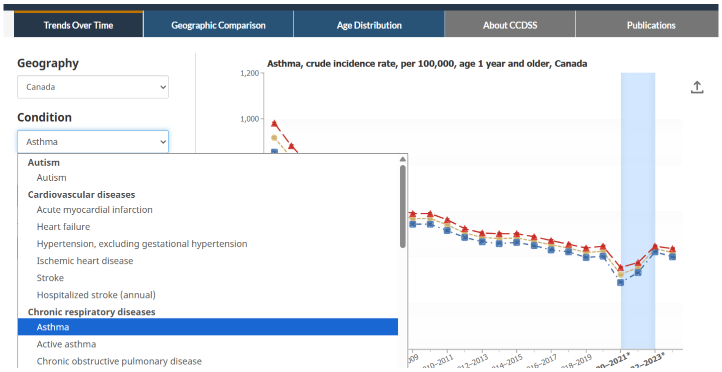



Users can explore trends over time, compare data across provinces/territories, and view the distribution of cases by age group. Under the `Publication` tab, you can find information on the repository for data-driven insights, fact sheets, peer-reviewed publications, and reports.

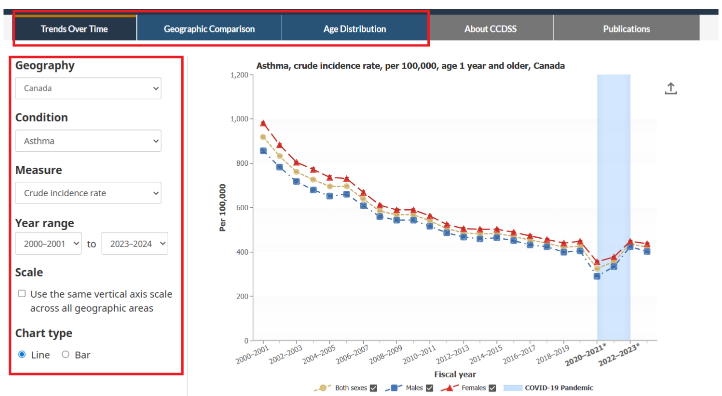



## Interactive chart example

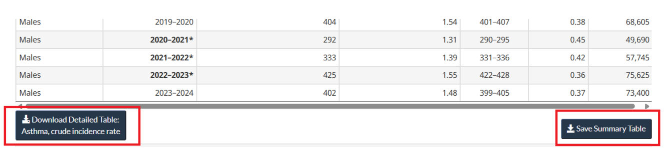
Consider `Asthma` under `Chronic respiratory diseases` as an example.




You can explore different summary measures by filtering by geography, measure, and year. Again, you can further explore trends over time, compare data across provinces/territories, and view the distribution of cases by age group.




You can also download the summary data in CSV format for further analysis. To do that, you need to scroll down to the bottom of the `Summary Table`, and click on the `Download` button.



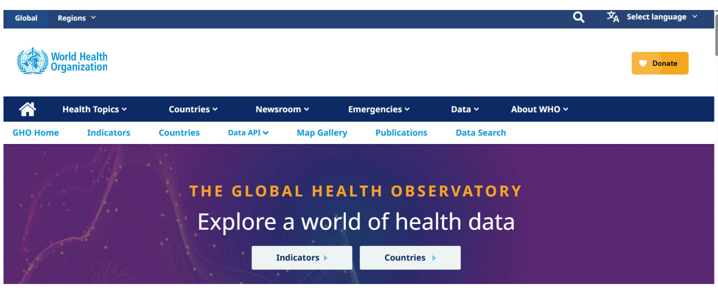

Once you open the CSV file in Excel or other software, you will find that the data is not at the individual or person level. Instead, the data are aggregated at the country and province/territory levels, and summary measures are provided for both sexes and for years. For example, see row 4. The `Geography` column says *Canada* and the `Rate (per 100,000)` column says *919*. This is an example of aggregate data.

| | A | B | C | D | E | F | G | H | I | J | K | L |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Data Table: Asthma, Crude incidence rate, per 100,000 | | | | | | | | | | | |
| 2 | | | | | | | | | | | | |
| 3 | Geography | Sex | Age group type | Age group | Fiscal year | Rate (per 100,000) | Standard Error | Lower 95%_CI | Upper 95%_CI | Rate CV (%) | Case counts | Population |
| 4 | Canada | Both sexes | Total | 1+ | 2000–2001 | 919 | 1.8 | 916 | 923 | 0.2 | 260730 | 28367890 |
| 5 | Canada | Females | Total | 1+ | 2000–2001 | 981 | 2.6206 | 976 | 986 | 0.27 | 140225 | 14289240 |
| 6 | Canada | Males | Total | 1+ | 2000–2001 | 856 | 2.4657 | 851 | 861 | 0.29 | 120505 | 14078655 |
| 7 | Canada | Both sexes | Total | 1+ | 2001–2002 | 833 | 1.7103 | 830 | 837 | 0.21 | 237405 | 28488200 |
| 8 | Canada | Females | Total | 1+ | 2001–2002 | 883 | 2.4814 | 878 | 888 | 0.28 | 126585 | 14337965 |
| 9 | Canada | Males | Total | 1+ | 2001–2002 | 783 | 2.3526 | 779 | 788 | 0.3 | 110820 | 14150240 |
| 10 | Canada | Both sexes | Total | 1+ | 2002–2003 | 762 | 1.6318 | 759 | 765 | 0.21 | 218155 | 28623720 |
| 11 | Canada | Females | Total | 1+ | 2002–2003 | 804 | 2.3637 | 800 | 809 | 0.29 | 115840 | 14399350 |
| 12 | Canada | Males | Total | 1+ | 2002–2003 | 719 | 2.2487 | 715 | 724 | 0.31 | 102310 | 14224360 |
| 13 | Canada | Both sexes | Total | 1+ | 2003–2004 | 727 | 1.5893 | 724 | 730 | 0.22 | 209140 | 28774260 |
| 14 | Canada | Females | Total | 1+ | 2003–2004 | 773 | 2.3117 | 769 | 778 | 0.3 | 111860 | 14468175 |
| 15 | Canada | Males | Total | 1+ | 2003–2004 | 680 | 2.1801 | 676 | 684 | 0.32 | 97275 | 14306080 |
| 16 | Canada | Both sexes | Total | 1+ | 2004–2005 | 696 | 1.5513 | 693 | 699 | 0.22 | 201445 | 28932495 |
| 17 | Canada | Females | Total | 1+ | 2004–2005 | 738 | 2.2531 | 734 | 743 | 0.31 | 107350 | 14541650 |
| 18 | Canada | Males | Total | 1+ | 2004–2005 | 654 | 2.1316 | 650 | 658 | 0.33 | 94095 | 14390840 |

Because this data is not at the person-level, and the summary measures are aggregated, you do not need a codebook to translate columns. However, you cannot use this data to calculate the prevalence or incidence of asthma in Canada because it is already aggregated. You can only use this data to explore the trends over time, compare data across provinces/territories, and view the distribution of cases by age group and sex. The individual patients have already been erased to protect their privacy.

# GHO

The Global Health Observatory (GHO) is the World Health Organization's (WHO) main health statistics repository. The GHO provides data on a wide range of health topics, such as mortality, morbidity, risk factors, health systems, and health equity. The GHO collects data from member states, international organizations, and other sources to provide a comprehensive picture of global health. The GHO is a valuable resource for researchers, policymakers, and public health professionals who are interested in understanding global health trends and disparities. The GHO provides various ways to interact with its data, including interactive maps and downloadable CSV files.

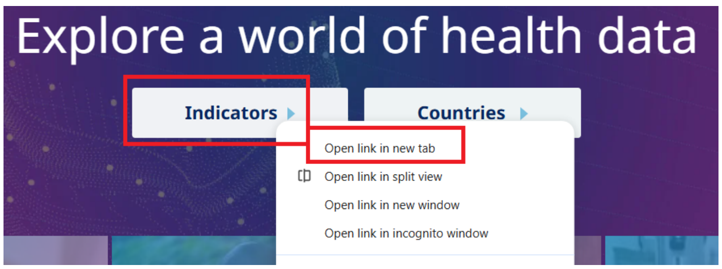


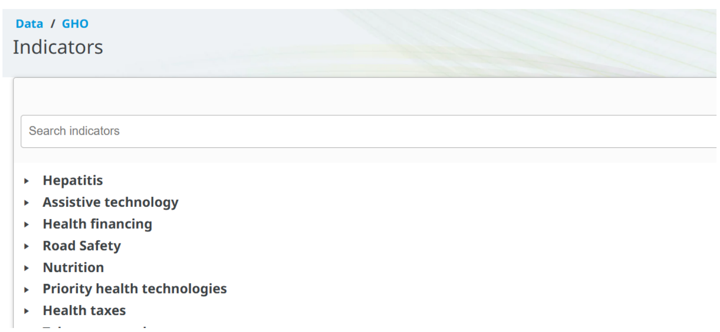

WHO Global Health Observatory

## World Health Data

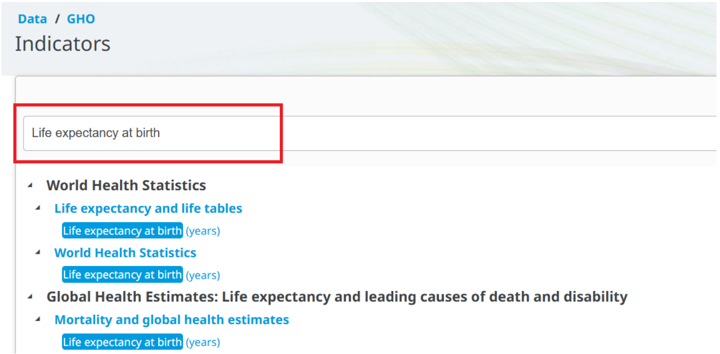
You can browse information by broader health indicators or by specific countries.


To explore the data by indicators, click on the `Indicators` tab. You can find a list of indicators across categories such as `Hepatitis`, `Nutrition`, `Mental health`, and so on.

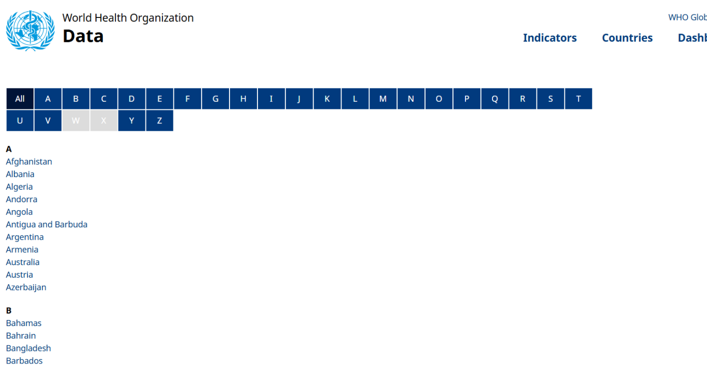


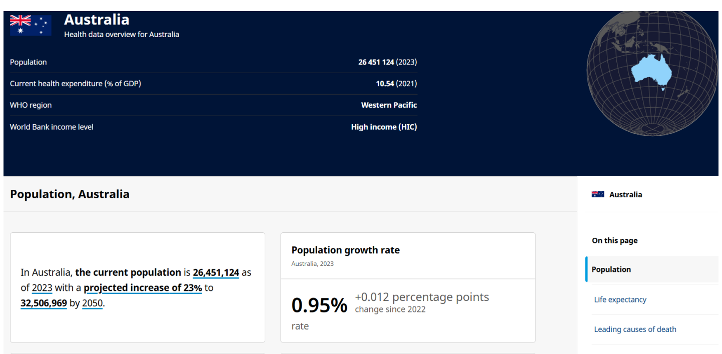




You can also use the search bar to search for specific indicators. For example, if you search for `Life expectancy at birth`, you will find a list of indicators related to life expectancy, such as `Life expectancy at birth (years)`.



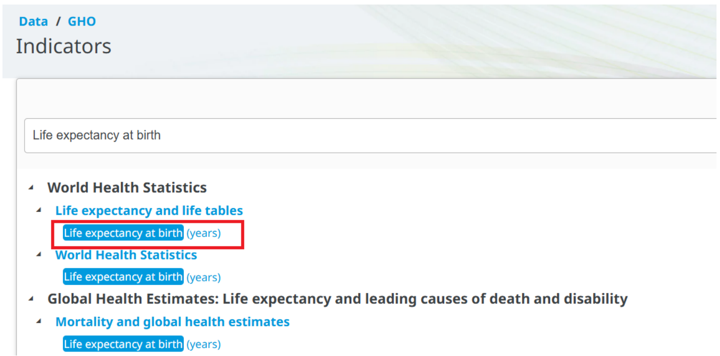

To explore the data by country, click on the `Countries` tab. You can find a list of countries and regions. You can click the country name to see the data for that country.



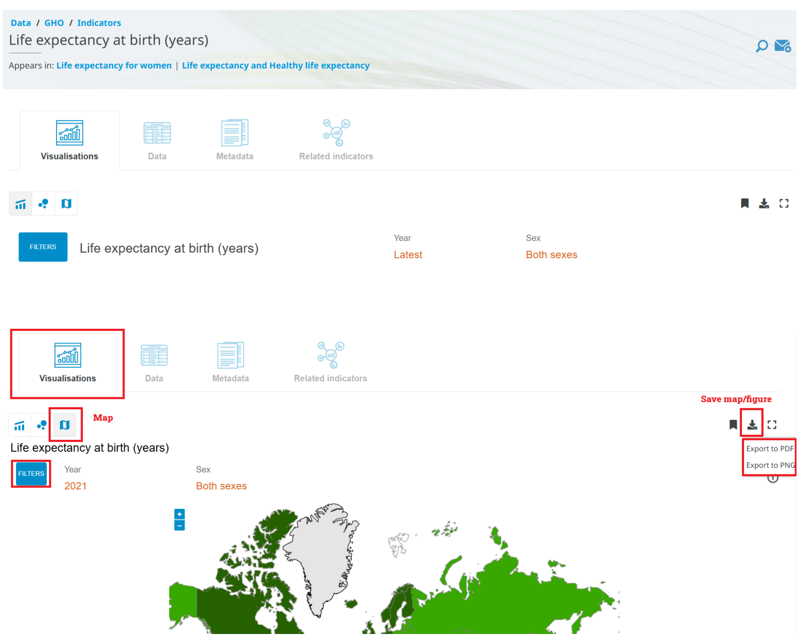
## Data by indicator example

Let's click on the `Life expectancy at birth (years)` indicator to see the details:

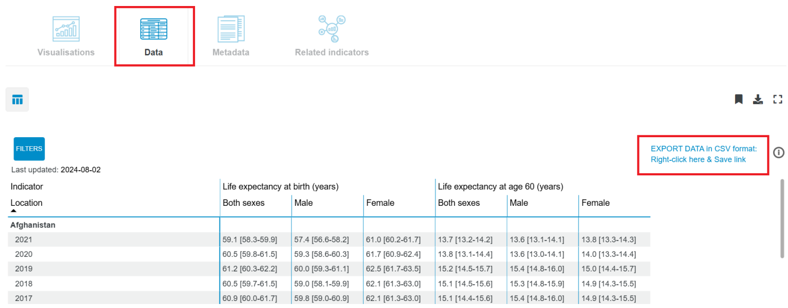



You can find different visualization options, such as charts and maps. You can also download the chart and map to your computer.

GHO Indicators

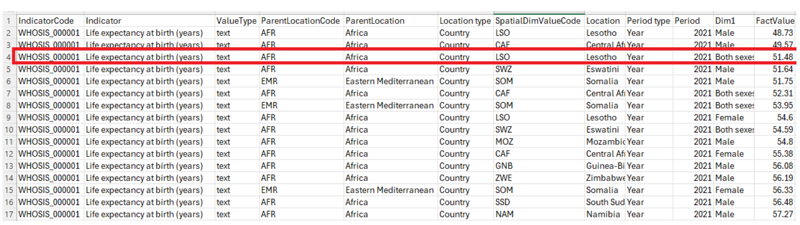


To see the data table, you can check the `Data` section. You can also download the data in CSV format for further analysis. To do that, you need to click on the `Download` button at the top right corner of the data table.



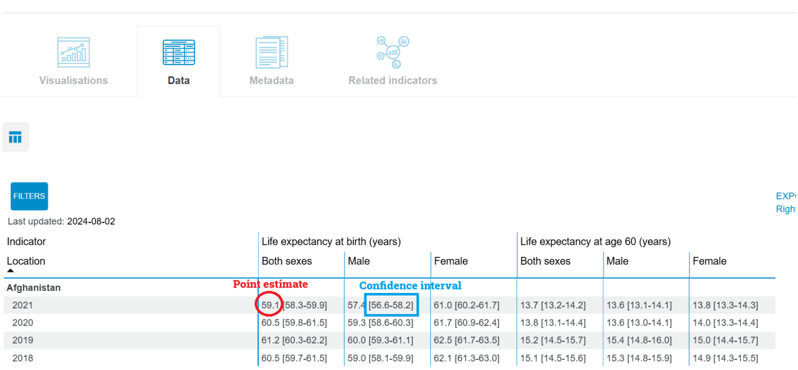

| Indicator | Life expectancy at birth (years) | | | Life expectancy at age 60 (years) | | |
|---|---|---|---|---|---|---|
| Location | Both sexes | Male | Female | Both sexes | Male | Female |
| Afghanistan | | | | | | |
| 2021 | 59.1 [58.3-59.9] | 57.4 [56.6-58.2] | 61.0 [60.2-61.7] | 13.7 [13.2-14.2] | 13.6 [13.1-14.1] | 13.8 [13.3-14.3] |
| 2020 | 60.5 [59.8-61.5] | 59.3 [58.6-60.3] | 61.7 [60.9-62.4] | 13.8 [13.1-14.4] | 13.6 [13.0-14.1] | 14.0 [13.3-14.4] |
| 2019 | 61.2 [60.3-62.2] | 60.0 [59.3-61.1] | 62.5 [61.7-63.5] | 15.2 [14.5-15.7] | 15.4 [14.8-16.0] | 15.0 [14.4-15.7] |
| 2018 | 60.5 [59.7-61.5] | 59.0 [58.1-59.9] | 62.1 [61.3-63.0] | 15.1 [14.5-15.6] | 15.3 [14.8-15.9] | 14.9 [14.3-15.5] |
| 2017 | 60.9 [60.0-61.7] | 59.8 [59.0-60.9] | 62.1 [61.3-63.0] | 15.1 [14.4-15.6] | 15.4 [14.8-16.0] | 14.9 [14.3-15.5] |

Similar to the Canadian Chronic Disease Surveillance System (CCDSS) database, the life expectancy data is also at an aggregated level, which means that the data is summarized at the country level, and does not contain individual-level data:



| 1 | IndicatorCode | Indicator | ValueType | ParentLocationCode | ParentLocation | Location type | SpatialDimValueCode | Location | Period type | Period | Dim1 | FactValue |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | LSO | Lesotho | Year | 2021 | Male | 48.73 |
| 3 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | CAF | Central Afi | Year | 2021 | Male | 49.57 |
| 4 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | LSO | Lesotho | Year | 2021 | Both sexes | 51.48 |
| 5 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | SWZ | Eswatini | Year | 2021 | Male | 51.64 |
| 6 | WHOSIS_000001 | Life expectancy at birth (years) | text | EMR | Eastern Mediterranean | Country | SOM | Somalia | Year | 2021 | Male | 51.75 |
| 7 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | CAF | Central Afi | Year | 2021 | Both sexes | 52.31 |
| 8 | WHOSIS_000001 | Life expectancy at birth (years) | text | EMR | Eastern Mediterranean | Country | SOM | Somalia | Year | 2021 | Both sexes | 53.95 |
| 9 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | LSO | Lesotho | Year | 2021 | Female | 54.6 |
| 10 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | SWZ | Eswatini | Year | 2021 | Both sexes | 54.59 |
| 11 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | MOZ | Mozambic | Year | 2021 | Male | 54.8 |
| 12 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | CAF | Central Afi | Year | 2021 | Female | 55.38 |
| 13 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | GNB | Guinea-Bi | Year | 2021 | Male | 56.08 |
| 14 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | ZWE | Zimbabwe | Year | 2021 | Male | 56.19 |
| 15 | WHOSIS_000001 | Life expectancy at birth (years) | text | EMR | Eastern Mediterranean | Country | SOM | Somalia | Year | 2021 | Female | 56.33 |
| 16 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | SSD | South Sud | Year | 2021 | Male | 56.48 |
| 17 | WHOSIS_000001 | Life expectancy at birth (years) | text | AFR | Africa | Country | NAM | Namibia | Year | 2021 | Male | 57.27 |

The data on the website, as well as the downloaded data in CSV format, both provide the point estimate with 95% confidence interval:



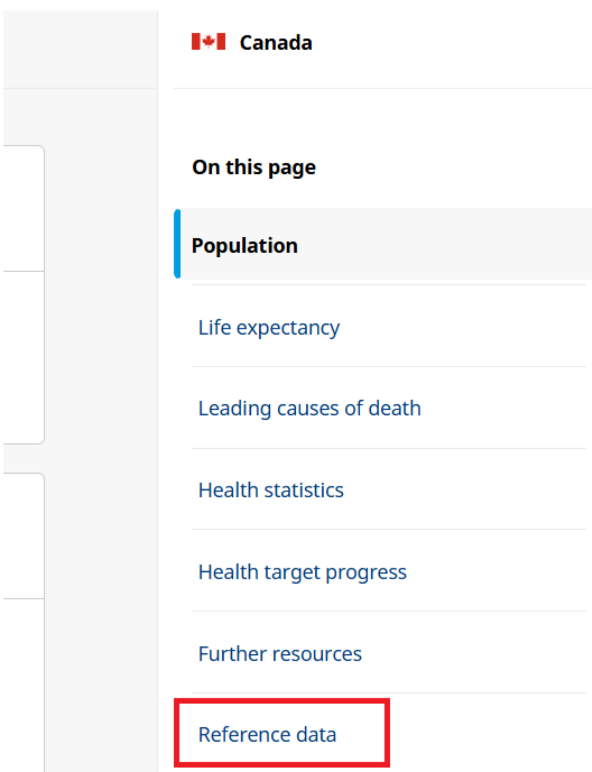

| Indicator | Life expectancy at birth (years) | | | Life expectancy at age 60 (years) | | |
|---|---|---|---|---|---|---|
| Location | Both sexes | Male | Female | Both sexes | Male | Female |
| Afghanistan | | | | | | |
| 2021 | 59.1 [58.3-59.9] | 57.4 [56.6-58.2] | 61.0 [60.2-61.7] | 13.7 [13.2-14.2] | 13.6 [13.1-14.1] | 13.8 [13.3-14.3] |
| 2020 | 60.5 [59.8-61.5] | 59.3 [58.6-60.3] | 61.7 [60.9-62.4] | 13.8 [13.1-14.4] | 13.6 [13.0-14.1] | 14.0 [13.3-14.4] |
| 2019 | 61.2 [60.3-62.2] | 60.0 [59.3-61.1] | 62.5 [61.7-63.5] | 15.2 [14.5-15.7] | 15.4 [14.8-16.0] | 15.0 [14.4-15.7] |
| 2018 | 60.5 [59.7-61.5] | 59.0 [58.1-59.9] | 62.1 [61.3-63.0] | 15.1 [14.5-15.6] | 15.3 [14.8-15.9] | 14.9 [14.3-15.5] |

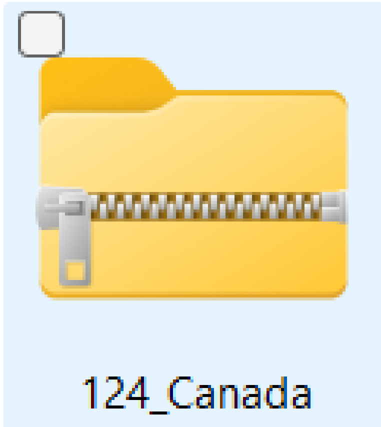
### Data by country example

Let's click on `Canada` to explore the data for Canada.


You can explore different indicators for Canada. You can also download all indicators as a ZIP file by clicking `Reference data`.

GHO Data by Country

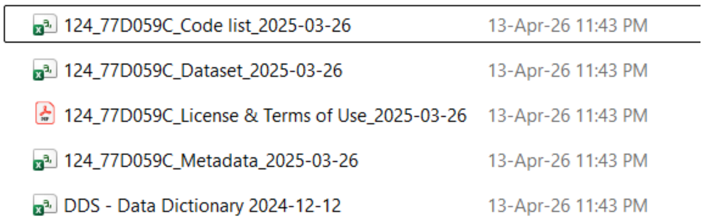





Once you unzip the folder, you will find several folders. Each folder contains a codelist, data, a data dictionary, along with a license and terms of use for different indicators.

| | | |
|---|---|---|
| 1F96863_3.4.1 - cardiovascular disease, cancer, diabetes or chronic respiratory disease | 03-Mar-26 10:33 AM | File folder |
| 2D6FBE4_3.3.5 - Neglected tropical diseases | 03-Mar-26 10:33 AM | File folder |
| 5C8435F_3.C.1 - Health worker density and distribution | 03-Mar-26 10:33 AM | File folder |
| 6A64C9A_7.1.2 - Clean fuels | 03-Mar-26 10:33 AM | File folder |
| 12EE54A_6.2.1 - Safely managed sanitation & hand-washing | 03-Mar-26 10:33 AM | File folder |
| 16BBF41_3.4.2 - Suicide | 03-Mar-26 10:33 AM | File folder |
| 45CA7C8_3.C.1 - Health worker density and distribution | 03-Mar-26 10:33 AM | File folder |
| 75DDA77_3.A.1 - Age-standardized prevalence of tobacco use | 03-Mar-26 10:33 AM | File folder |
| 77D059C_3.3.1 - HIV infections | 03-Mar-26 10:33 AM | File folder |
| 84FD3DE_3.9.3 - Unintentional poisoning | 03-Mar-26 10:33 AM | File folder |
| 608DE39 | 03-Mar-26 10:33 AM | File folder |
| 1548EA3_6.1.1 - Safely managed drinking water | 03-Mar-26 10:33 AM | File folder |
| 217795A_3.C.1 - Health worker density and distribution | 03-Mar-26 10:33 AM | File folder |
| 361734E_16.1.1 - Intentional homicide | 03-Mar-26 10:33 AM | File folder |
| 1772666_3.1.2 - Births attended by skilled health personnel | 03-Mar-26 10:33 AM | File folder |

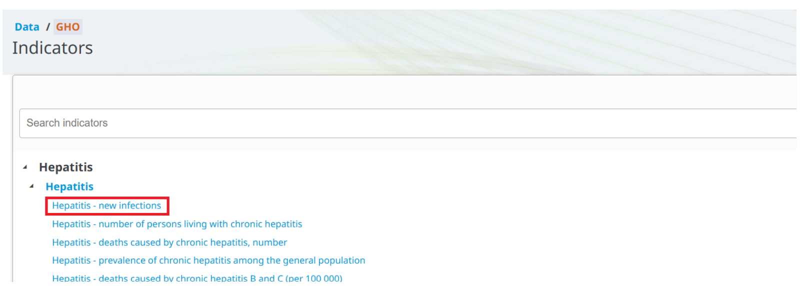
For example, the `77D059C_3.3.1 - HIV infections` folder contains the following files:

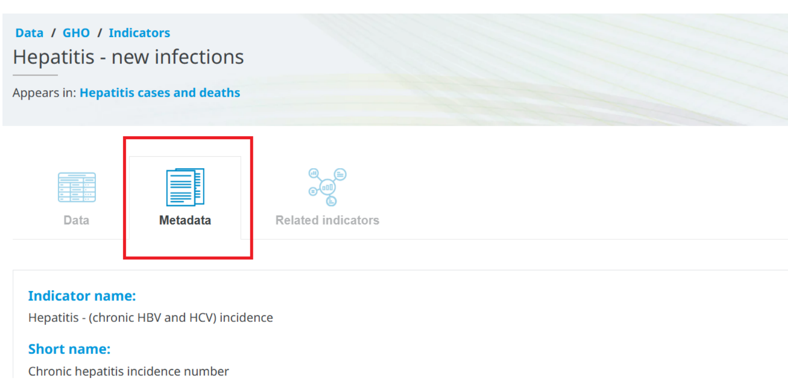



## Metadata

The GHO provides metadata for each indicator, providing important context for understanding the data and its limitations. For example, the metadata for the `Hepatitis - new infections` indicator includes information on the definition of the indicator, the methods used to calculate the indicator, data sources, and other relevant information.

Indicator Metadata Registry List



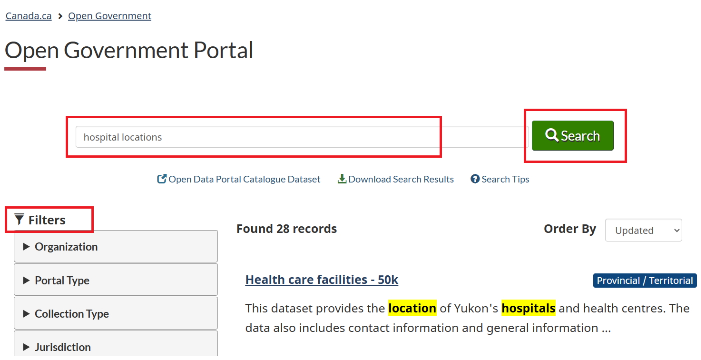
# Open Portal

Open Government Portal Canada is a platform that provides access to searching for a wide range of public data. Users can explore and download data in different formats under the Open Government Licence - Canada.

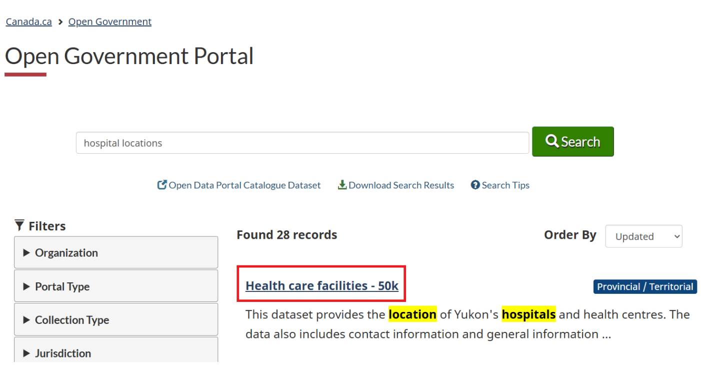

Open Government Portal Canada

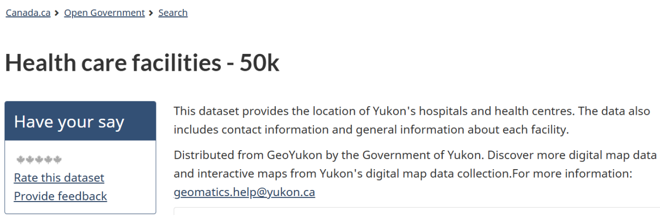
## Search for Open Data

To search for open data, you can use the search bar on the Open Government Portal Canada homepage. For example, if you are interested in finding data on "hospital locations", you can enter that term in the search bar and click the search icon. There are many filter options to narrow your search results, such as format and organization.

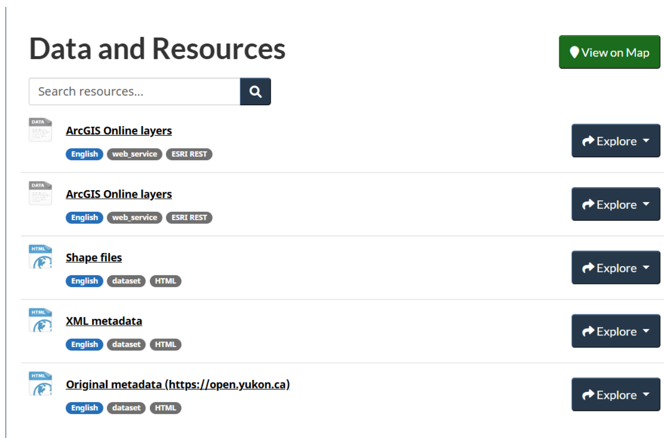




To explore a specific dataset, click its title. This will take you to a page with more information about the dataset, including its description, format options, and download links. For example, if you click on the "Health care facilities - 50k" dataset, you will find detailed information about the location of Yukon's hospitals and health centres.

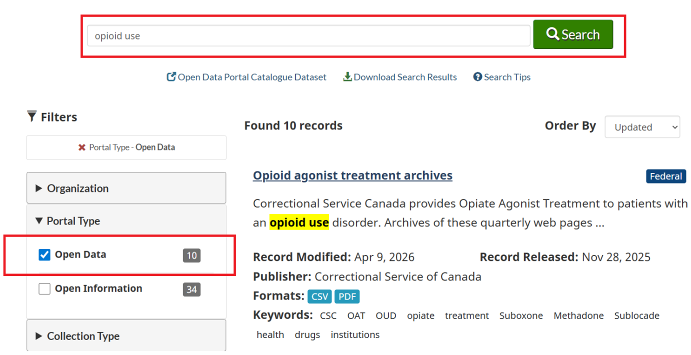




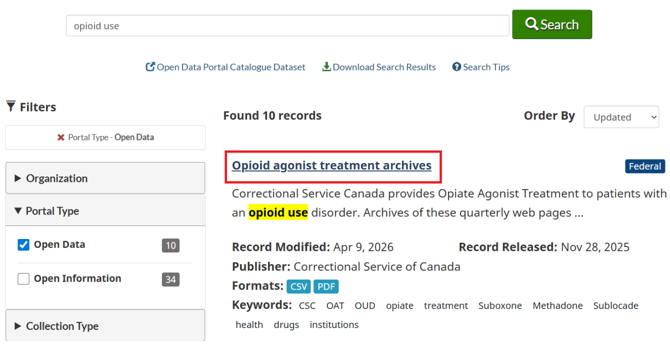

Under the `Data and Resources` tab, you can find the available formats for downloading the location of Yukon's hospitals and health centres.

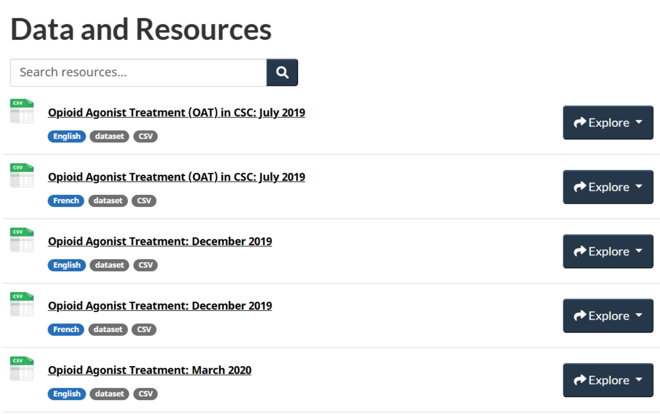



Similarly, say we are interested in "opioid use" and want to find open data related to that topic. We can enter "opioid use" into the search bar and filter by `Open data`:



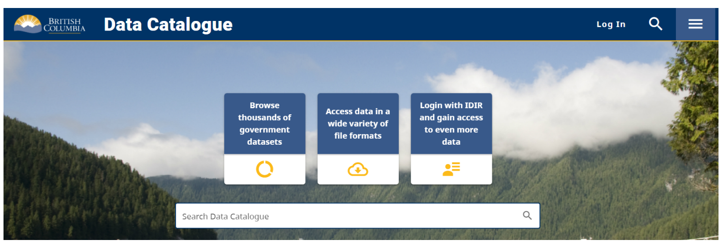
To explore the data on opioid agonist treatment, we can click the dataset title. This will take us to a page with more information about the dataset, including its description, format options, and download links.



# BC Data Catalogue

The British Columbia Data Catalogue is a central hub provided by the Government of British Columbia (BC). Its primary purpose is to provide public access to thousands of datasets managed by the provincial government and its partners. Essentially, it is a digital library designed to make government information transparent and usable for citizens, researchers, and developers.

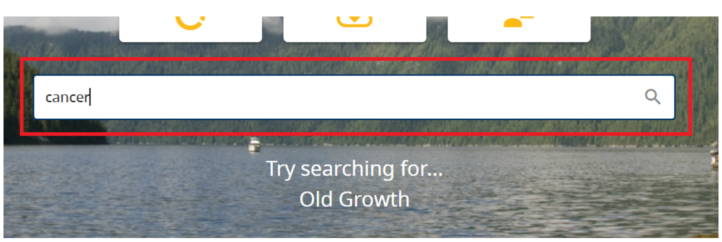


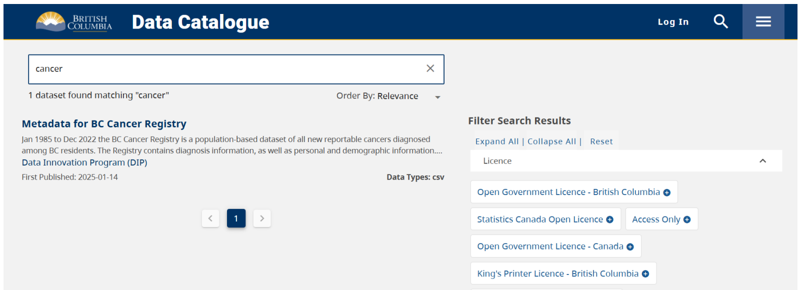

British Columbia Data Catalogue

## Search for Data

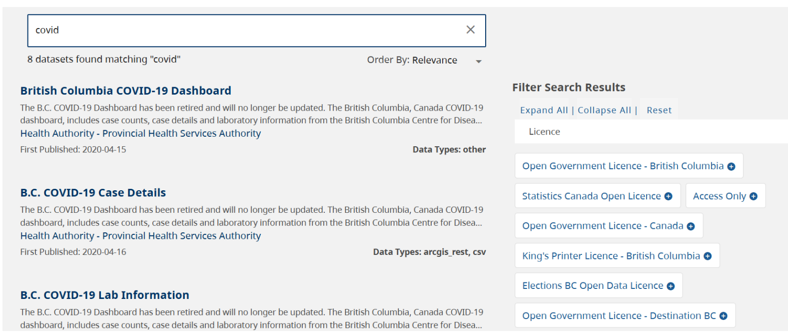
To search for data, you can use the search bar on the BC Data Catalogue homepage. For example, if you are interested in finding data on "cancer", you can enter that term in the search bar and click the search icon.

Search Data Catalogue

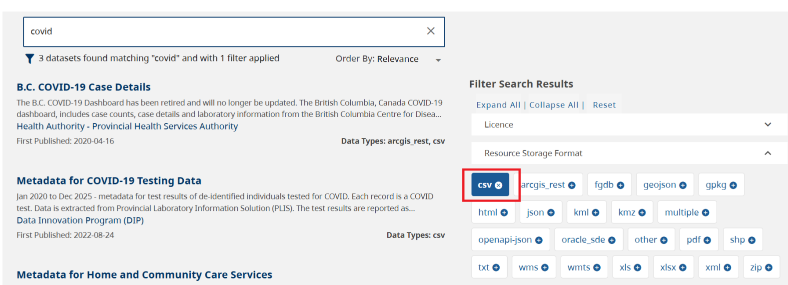




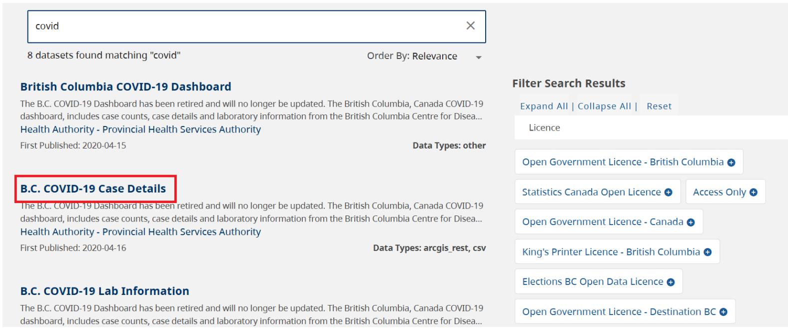

Similarly, say we are interested in "COVID-19" and want to find open data related to that topic. We can enter "covid" into the search bar:



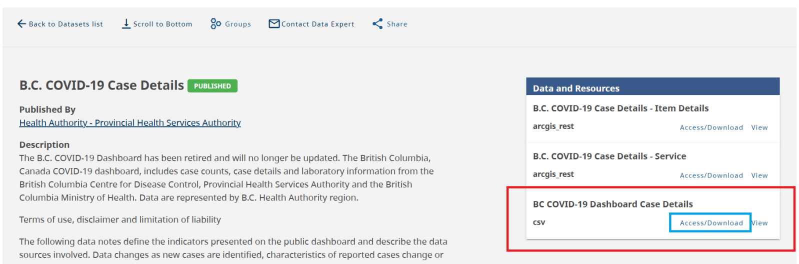

You can filter your search by license, format, group, and so on:



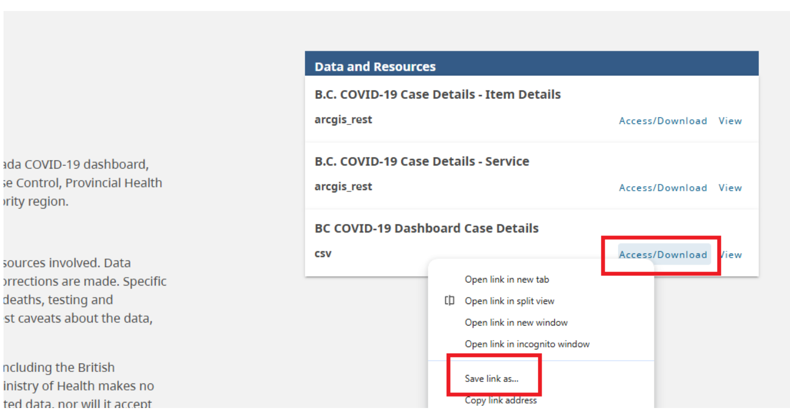

Let's explore the "COVID-19 Case Details" dataset. To do that, we can click the dataset title. This will take us to a page with more information about the dataset, including its description and related links.

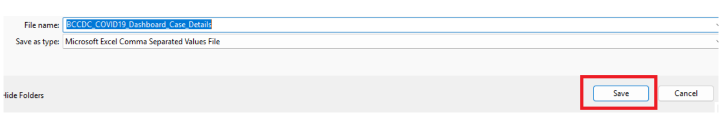



You will find information on COVID-19 case details as well as a link to download the data:




To download the dataset, right-click on the `Access/Download` link and select `Save link as...`. This will allow you to save the dataset to your computer.



After downloading, you can open the data in Excel, Google Sheets, or any statistical software. The data is structured in a tabular format, with each row representing an individual case and each column representing different variables.

| 1 | Reported_Date | HA | Sex | Age_Group | Classification_Reported |
|---|---|---|---|---|---|
| 2 | 29-01-20 | Out of Canada | M | 40-49 | Lab-diagnosed |
| 3 | 06-02-20 | Vancouver Coastal | F | 50-59 | Lab-diagnosed |
| 4 | 10-02-20 | Out of Canada | M | 30-39 | Lab-diagnosed |
| 5 | 10-02-20 | Out of Canada | F | 20-29 | Lab-diagnosed |
| 6 | 18-02-20 | Interior | F | 30-39 | Lab-diagnosed |
| 7 | 24-02-20 | Fraser | F | 30-39 | Lab-diagnosed |
| 8 | 24-02-20 | Fraser | M | 40-49 | Lab-diagnosed |
| 9 | 03-03-20 | Fraser | M | 50-59 | Lab-diagnosed |
| 10 | 03-03-20 | Vancouver Coastal | F | 60-69 | Lab-diagnosed |
| 11 | 03-03-20 | Vancouver Coastal | F | 30-39 | Lab-diagnosed |
| 12 | 03-03-20 | Vancouver Coastal | F | 80-89 | Lab-diagnosed |
| 13 | 03-03-20 | Vancouver Coastal | M | 60-69 | Lab-diagnosed |
| 14 | 05-03-20 | Out of Canada | F | 50-59 | Lab-diagnosed |
| 15 | 05-03-20 | Vancouver Coastal | M | 60-69 | Lab-diagnosed |
| 16 | 05-03-20 | Fraser | F | 50-59 | Lab-diagnosed |
| 17 | 05-03-20 | Vancouver Coastal | M | 30-39 | Lab-diagnosed |

To learn more about BC Data Catalogue, click on the three bars on the right side of the page: